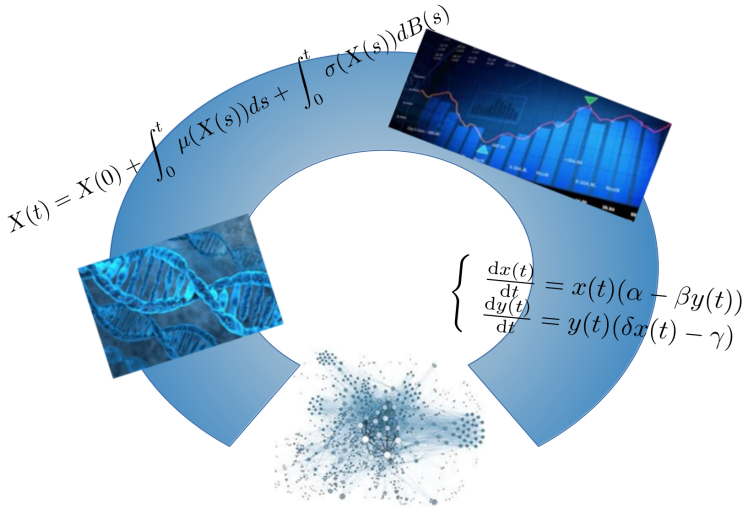
Apprentissage en ligne pour la finance quantitative

4 juin 2025

Synthèse finale



Equipe n° A(S6+S7) :
Vinicius DA MATA E MOTA
Quentin FRETAULT
Nicolas LAFAILLE
Martin PUPAT
Zein SAKKOUR
Philémon WEHBE

Equipe n° B (S8) :
Nabil ELYAZIDI
Victor LETOURNEL
Javier TORO BERRIOS
Philémon WEHBE
Seif ZAAFOURI

Client et Référent :
Damien CHALLET
MICS

Pôle n° 15
Modélisation Mathématique des Systèmes Complexes

Table des matières

1 Introduction **1**

2 État de l'art **1**

I Semestre 6 : Découverte de l'online learning et premiers pas. **2**

3 Optimisation convexe en ligne (OCO) **2**
3.1 Définition du problème . . . 2
3.2 Analyse du regret . . . 2

4 Marche aléatoire gaussienne **3**

5 Algorithme Follow-The-Leader (FTL) **4**
5.1 Description de l'algorithme . . . 4
5.2 Follow-The Leader dans le contexte des marchés financiers . . . 4
5.3 Les succès et les échecs de Follow-The-Leader . . . 5

6 Les variantes de Follow-The-Leader **7**
6.1 FTL avec un seuil : Follow-The-Leader-Epsilon (FTL$\mathcal{E}$) . . . 7
6.1.1 Description de l'algorithme . . . 7
6.1.2 Exemple de Follow-The-Leader-Epsilon . . . 7
6.2 FTL avec un facteur d'oubli . . . 8
6.3 FTL Moving Average . . . 9

7 Régularisation **10**
7.1 Techniques de régularisation . . . 10
7.2 Régularisation L1 . . . 10
7.3 Régularisation L2 . . . 10

8 L'algorithme Follow-The-Regularized-Leader (FTRL) **11**
8.1 Définition du problème . . . 11
8.2 Description de l'algorithme . . . 11
8.3 Borne du regret FTRL . . . 11
8.4 FTRL en pratique . . . 12
8.5 Variations de FTRL : . . . 13
8.6 FTRL avec l'approche de n-SIGMA pour redémarrer l'apprentissage . . . 13

9 Redémarrage des algorithmes : l'approche de l'écart-type **14**
9.1 Idée de l'approche . . . 14
9.2 Exemples avec FTL$\mathcal{E}$ . . . 14

II Semestre 7 : Portefeuilles universels et Applications. **16**

10 Introduction à l'algorithme des portefeuilles universels **16**
10.1 Contexte . . . 16
10.2 Algorithme *Universal Portfolios* . . . 16
10.3 Implémentation, applications et difficultés . . . 17
10.4 Une première approche du redémarrage . . . 19

11 L'algorithme VB-FTRL-qN **21**
11.1 Principe de l'algorithme . . . 21
11.1.1 Approche . . . 21
11.1.2 Algorithme . . . 22
11.1.3 Approximation de Universal Portfolios . . . 23
11.1.4 Implémentation théorique et méthode(s) de quasi-Newton . . . 24
11.1.5 Performance Théorique . . . 25
11.2 Implémentation en python . . . 25

12 Prétraitement des données **27**

13 Tests et résultats **28**
13.1 Tests sur des données synthétiques . . . 28
13.2 Tests sur des données réelles . . . 29

14 Redémarrage de l'algorithme **30**
14.0.1 Approche de l'écart-type . . . 30
14.0.2 Approche de la variation du portefeuille . . . 30

15 Tests et résultats du redémarrage **31**

16 Conclusion **33**
16.1 Résumé . . . 33
16.2 Ouverture . . . 33

III Semestre 8 : Optimisation et Redémarrage. **35**

17 Introduction du S8 **35**

18 Optimisation du code **35**
18.1 Rappels mathématiques du Universal Portfolio . . . 36
18.1.1 Définition générale . . . 36
18.1.2 Utilité logarithmique . . . 36
18.1.3 Fenêtre glissante (rolling window) . . . 37
18.2 Implémentation naïve en Python . . . 37
18.2.1 Jeu de données et notations . . . 37
18.2.2 Structure de l'implémentation naïve . . . 37
18.2.3 Problèmes principaux de performance . . . 38
18.3 Optimisation et vectorisation NumPy . . . 38
18.3.1 Philosophie générale de la vectorisation . . . 38
18.3.2 Recalcul matriciel de l'utilité . . . 38
18.3.3 Calcul vectorisé des poids et du portefeuille universel . . . 40
18.3.4 Rééquilibrage de la richesse . . . 41
18.4 Code optimisé . . . 41
18.5 Analyse de complexité et gains de performance . . . 43
18.5.1 Complexité de l'implémentation optimisée . . . 43
18.5.2 Comparaison avec l'approche naïve . . . 43
18.6 Optimisation GPU avec CuPy . . . 43
18.7 Gains de temps mesurés . . . 44
18.8 Résumé et perspectives . . . 44
18.8.1 Perspectives et améliorations possibles . . . 44

19 Vérification de l'approximation **45**
19.1 Données et protocole expérimental . . . 45
19.1.1 Source et nettoyage des données . . . 45
19.1.2 Effet des hyper-paramètres . . . 45
19.1.3 Buckets et valeurs fixées . . . 45
19.1.4 Plan d'expérience par sous-échantillonnage . . . 46
19.2 Résultats et discussion . . . 46
19.2.1 Impact de la dimension . . . 46
19.2.2 Richesse cumulée des portefeuilles . . . 48
19.3 Résumé . . . 49
19.4 Stratégie de redémarrage de VB-FTRL . . . 50
19.4.1 Classements et corrélations . . . 50
19.5 Premiers tests — Seuil = 0 . . . 51
19.6 Optimisation de μ et λ . . . 52
19.7 Optimisation du seuil de redémarrage . . . 52
19.7.1 Kendall . . . 53
19.7.2 Kendall pondéré . . . 53
19.7.3 Blest . . . 53
19.8 Tableau des seuils optimaux . . . 55
19.9 Tests et Comparaisons Finales . . . 56
19.9.1 Blest . . . 56
19.9.2 Kendall . . . 57
19.9.3 Kendall pondéré . . . 58
19.10Comparaison globale . . . 59
19.11Tableau des redémarrages par stratégie . . . 60
19.12Résumé et perspectives . . . 60

20 Conclusion finale **61**

1 Introduction

Dans le domaine dynamique de la finance quantitative, la capacité de s'adapter et d'apprendre à la volée à partir des données diffusées en continu est de plus en plus vitale. Pour être toujours plus compétitifs et performants, les hedge funds font appel à des algorithmes d'apprentissage en ligne, des algorithmes rapides capables de deviner le comportement du marché à partir des données reçues en temps réel. Mais les algorithmes d'apprentissage en ligne présentent un défaut : plus le temps passe, moins l'algorithme sera efficace pour s'adapter aux changements brusques du marché. Notre objectif est de remédier à ce comportement.

Dans cet article, nous étudierons la manière dont les algorithmes d'apprentissage en ligne s'adaptent pour faire des prédictions sur des séries temporelles à évolution rapide, comme en finance quantitative. Nous tenterons également de régler le défaut mentionné plus haut, et pour ce faire nous étudierons des méthodes « douces » telles que des algorithmes modifiés pour essayer de contourner le problème, ainsi qu'une méthode « dure » : le redémarrage des algorithmes.

Dans la seconde partie de ce rapport, nous étudierons ensuite une autre idée pour l'allocation de portefeuille optimale : l'algorithme des portefeuilles universels. Nous y verrons les avantages de cette solution mais aussi les difficultés auxquelles nous avons fait face pour l'implémenter. Nous chercherons et expliciterons des solutions potentielles. Nous essayerons enfin de réfléchir au problème du redémarrage dans ce nouveau cas.

2 État de l'art

Il existe deux principaux paradigmes d'apprentissage pour le machine learning : l'apprentissage hors ligne qui s'appuie sur l'analyse des données collectées à l'avance et l'apprentissage en ligne qui se concentre sur la prise de décision à l'aide de données collectée en temps réel.

Il n'existe aucun mécanisme pour les algorithmes entraînés hors-ligne leur permettant d'apprendre à la volée des nouvelles informations. Pour apprendre de nouvelles informations, les algorithmes devraient normalement être entraînés de nouveau en utilisant le nouvel échantillon de données en plus des échantillons précédents. Contrairement à l'apprentissage hors-ligne, un algorithme d'apprentissage en ligne est capable de s'adapter à des environnements en évolution, par exemple pour réaliser un contrôle plus optimale des feux de signalisation [3].

De nos jours, les algorithmes d'online learning tendent à se développer dans la recherche mais aussi différentes industries ayant besoin d'analyser une large quantité de données (généralement des séries temporelles comme en finance par exemple). On peut donc y retrouver des cours divers et variés sur lesquels nous nous sommes appuyé pour construire ce rapport [2].

Première partie

Semestre 6 : Découverte de l'online learning et premiers pas.

3 Optimisation convexe en ligne (OCO)

3.1 Définition du problème

Dans les scénarios d'apprentissage en ligne, nous rencontrons le problème de l'optimisation convexe en ligne, où des décisions doivent être prises séquentiellement en réponse aux données observées. Ici, nous examinons le contexte dans lequel les décisions sont prises de manière itérative à chaque tour, et notre objectif est de minimiser les pertes cumulées subies au cours de l'ensemble de ces tours.

Description du problème :
Considérons une séquence de tours $t = 1, 2, \ldots, T$, où à chaque tour, un décideur doit sélectionner un vecteur ω_t dans l'espace $\mathbb{R}^n$. Par la suite, une fonction de perte convexe f_t est révélée, traduisant l'écart entre le point choisi et une décision optimale dans le contexte de ce tour. La perte subie au tour t est alors $f_t(\omega_t)$.

3.2 Analyse du regret

Notre mesure de performance en optimisation convexe en ligne est souvent quantifiée par la notion de regret, qui évalue la performance du décideur par rapport à un vecteur de référence ou de comparaison $\mathring{\omega}$.

Le regret est alors défini comme la différence cumulée entre les pertes subies par nos vecteurs choisis ω_t et les pertes subies par le vecteur de référence $\mathring{\omega}$ sur les T tours :

$$\text{Regret} = \sum_{t=1}^{T} f_t(\omega_t) - \sum_{t=1}^{T} f_t(\mathring{\omega})$$

Un bon vecteur de référence pour évaluer la performance est le vecteur minimisant perte cumulée, donnée par :

$$\mathring{\omega} = \arg\min_{w \in W} \sum_{t=1}^{T} f_t(w)$$

On définit alors le regret par :

$$R(T) = \sum_{t=1}^{T} f_t(w_t) - \min_{w \in W} \sum_{t=1}^{T} f_t(w)$$

Dans le cadre de l'apprentissage en ligne :
— Si le regret augmente linéairement, $R(T) = \Omega(T)$, et donc le joueur n'apprend pas.
— Si le regret augmente de manière sous-linéaire, $R(T) = o(T)$, cela signifie que le joueur apprend et que la précision de ses prédictions s'améliore.

4 Marche aléatoire gaussienne

Afin de modéliser l'évolution du marché boursier, nous avons décidé d'utiliser une approche de marche aléatoire : nous avons modélisé les log-returns (les logarithmes de l'évolution des cours) avec une marche aléatoire gaussienne avec des paramètres qui suivent également une marche aléatoire. Plus précisément, si nous désignons les log-returns par R_t avec $t \in \mathbf{N}$, nous avons :

$$\forall t \in \mathbf{N},\ R_{t+1} - R_t \sim \mathcal{N}(\mu_t,\ \sigma_t^2)$$

où pour tout $t \in \mathbf{N}$

$$\mu_{t+1} - \mu_t \sim \mathcal{N}(m_\mu,\ s_\mu 2)$$
$$\sigma_{t+1}^2 - \sigma_t^2 \sim \mathcal{N}(m_\sigma,\ s_\sigma^2)$$

avec $m_\mu,\ s_\mu, m_\sigma$ et s_σ des constantes.

Même si ce modèle pourrait être critiqué quant à son réalisme, la capacité d'avoir des tendances (μ_t) et des volatilités (σ_t) qui peuvent évoluer sur le long terme et être mesurées avec exactitude nous a permis de mieux comprendre comment les différents algorithmes s'adaptent à différentes situations, par exemple avec des actions donc les retours sont négatifs puis positifs.

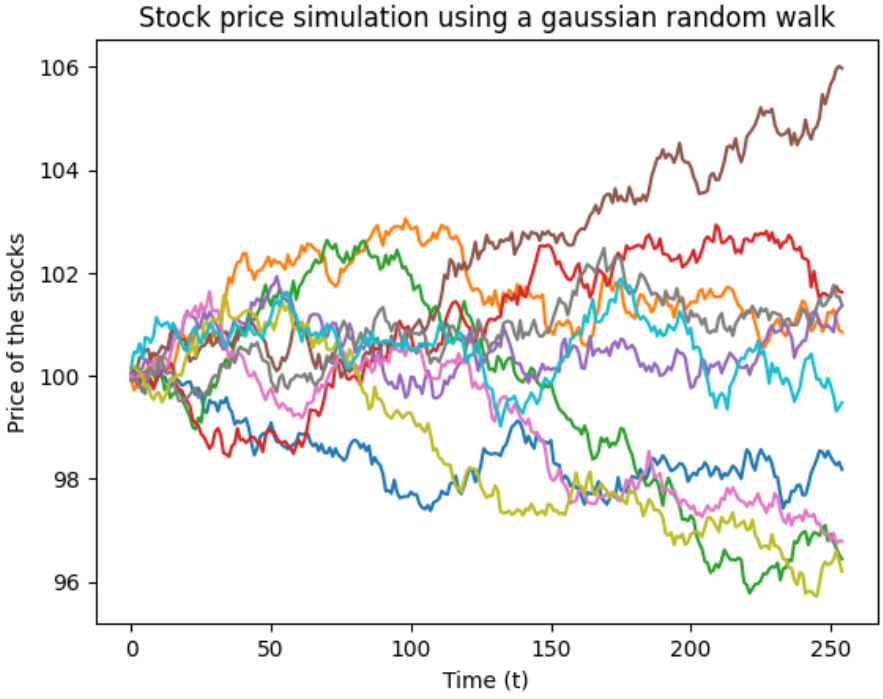


FIGURE 1 – Simulation du cours de 10 actions

5 Algorithme Follow-The-Leader (FTL)

5.1 Description de l'algorithme

Le premier algorithme que nous appliquerons dans le cadre de l'OCO est Follow-The-Leader, qui est le plus simple et naturel. Son idée est la suivante : s'il était possible de prédire exactement la meilleure action, il serait possible de gagner des bénéfices arbitrairement importants. Bien sûr, cela est impossible en pratique. Mais si on suppose que le marché évolue suffisamment lentement pour pouvoir mesurer la performance des actifs de telle manière à ce qu'elle ne change pas après la mesure, on peut s'approcher de cette stratégie en investissant dans les meilleurs actifs mesurés (les leaders).

Avec cette idée en tête et avec les notations de la section 3, l'algorithme Follow-The-Leader peut être exprimé de la façon suivante :

Follow-The-Leader
ω_0 est fixé arbitrairement.
Pour $T \geq 1$, on fixe ω_T tel que : $w_t \in \arg\min_{\omega \in \Omega} \left(\sum_{t=1}^{T-1} f_t(\omega) \right)$

TABLE 1 – L'algorithme Follow-The-Leader

On peut alors déterminer [2] la borne du regret de l'algorithme.

Lemme 1. ***Borne du regret de FTL*** *Soient $w_1, \ldots, w_T$ les vecteurs choisis par Follow-The-Leader. Alors, pour tout $u \in X$:*

$$Regret(u) \leq \sum_{t=1}^{T} [f_t(w_t) - f_t(u)]$$

Le plus gros problème avec FTL est qu'en général, la borne du regret est $O(T)$. Cela est particulièrement vrai pour les fonctions de perte linéaires, et peut conduire à des résultats décevants. Pour voir comment les fonctions de perte linéaire sont sous-performantes dans le cas général, nous allons poursuivre avec un exemple.

5.2 Follow-The Leader dans le contexte des marchés financiers

On considère un marché composé de n actifs différents qui ne sont pas correlés et dont on connaît les rendements logarithmiques à chaque pas de temps t : $R_{i,t}$ pour $i \in [\![1, n]\!]$. On pose alors l'ensemble des portefeuilles :

$$\Omega = \left\{ w \in (\mathbb{R}^+)^n \mid \sum_{i=1}^{n} w_i = 1 \right\}$$

Nous souhaitons maximiser nos gains en déterminant le meilleur portefeuille w_t au temps t. On considère donc la fonction de perte suivante [1] :

1. D'autres fonctions de perte ont été envisagées et testées. Par exemple, $f_t(\omega) = \sum_{i=1}^{n} \omega_i \frac{1}{R_i} \frac{\sigma_i}{\mu_i}$ avec μ_i et σ_i la moyenne du prix et la volatilité de l'actif i. Cette fonction de perte alternative vise à avoir un gain maximum tout en minimisant le risque.

$$f_t(\omega) = -\sum_{i=1}^{n} \omega_i R_{i,t} = -w_t \cdot R_t$$

Cette fonction exprime bien l'idée que le meilleur portfolio est celui qui rapporte le plus de rendement.

Il est aisé de vérifier que pour maximiser $w_t \cdot r_t$, sous la contrainte $\sum_{i=1}^{n} |w_{t,i}| = 1$, nous devons choisir

$$w_t = (0, \ldots, 0, 1, 0, \ldots, 0) = e_k \qquad k = \arg\max_{i=1,\ldots,n} r_{t,i}$$

Ainsi dans le cadre de FTL, minimiser $\sum_{s=1}^{t-1} f_s(w)$ est simple en utilisant la linéairité du produit scalaire : $\sum_{s=1}^{t-1} f_s(w) = \sum_{s=1}^{t-1} w \cdot r_s = w \cdot \sum_{s=1}^{t-1} r_s$.

En utilsant ce qui a été fait ci-dessus, on trouve

$$w = e_k \qquad k = \arg\max_{i=1,..,n} \left(\sum_{s=1}^{t-1} r_s \right)_i$$

Tout le capital du portefeuille est donc dans l'action la plus performante au tour précédent.

Nous savons donc maintenant implémenter FTL, il faut maintenant s'interesser à des métriques pour mesurer sa performance.

Pour faire des mesures de comparaison, nous allons calculer le gain avec un portefeuille uniforme : $w_t = (\frac{1}{n}, ..., \frac{1}{n})$. Nous calculerons ensuite le portefeuille constant (ou statique) optimal a posteriori,c'est-à-dire que nous utilisons la connaissance du futur pour faire notre investissement, mais celui-ci ne pourra pas changer au fil du temps. Nous devons être attentifs aux indices dans le code de nos algorithmes pour ne pas que FTL et FTRL utilisent d'informations futures.

Pour un portefeuille w constant au cours du temps, on définit le regret : avec $S_T(w)$ le gain du portefeuille w après T pas de temps,

$$Regret(w) = S_T(w) - S_T(FTL)$$
$$Regret(T) = \max_{w} \; Regret(w)$$

Le regret est la différence de performance entre notre algorithme (portefeuille qui change au fil du temps) et ce qu'on aurait pu espérer en choisissant un portefeuille et en ne ne modifiant pas (si on avait misé avec le meilleur portefeuille statique).

5.3 Les succès et les échecs de Follow-The-Leader

Les expériences nous donnent quelques nouvelles prometteuses : FTL est capable de détecter et de suivre une tendance lorsqu'elle existe. En effet, FTL va suivre le meilleur actif avec un décalage d'un pas de temps. Voici un exemple lorsqu'un actif domine le marché :

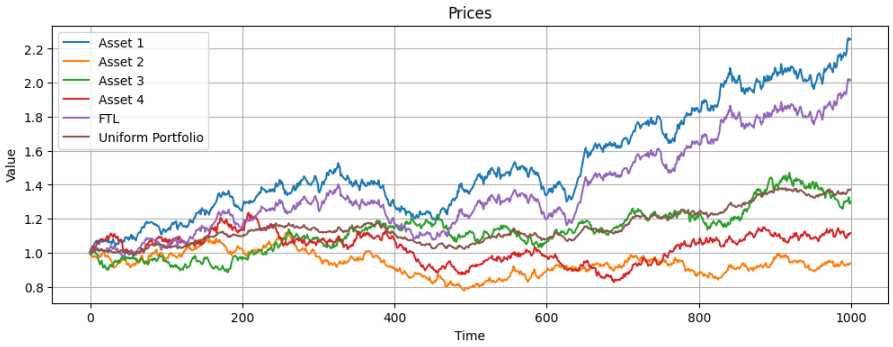


FIGURE 2 – FTL dans un cas efficace

Le regret est négligeable puisque FTL suit le meilleur portefeuille constant dès $t = 10$. Cependant, cette situation n'est pas vraiment représentative des marchés financiers. Regardons maintenant un cas plus intéressant : que se passe-t-il lorsque les actifs croissent de la même manière (nous prendrons ici une croissance nulle pour des questions de représentation graphique) ?

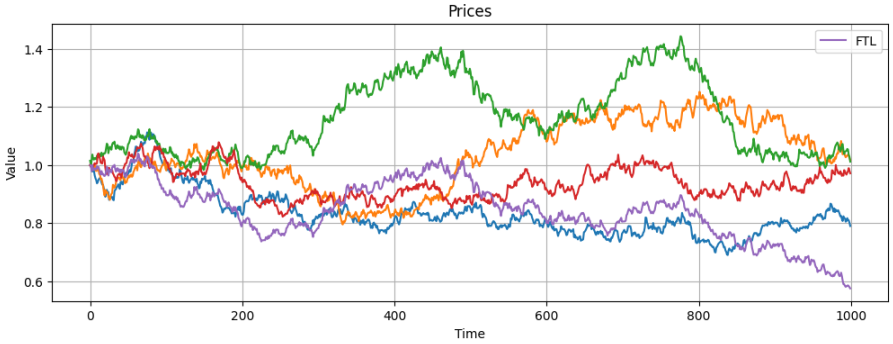


FIGURE 3 – FTL dans un cas catastrophique

Nous constatons que chaque fois que les courbes des actifs se croisent, FTL perd. En effet, FTL change de position trop souvent (59 fois dans la simulation ci-dessus), et alterne en misant toujours tout son capital sur un unique actif. Cette stratégie est néfaste pour le gain, et FTL sous-performe par rapport au meilleur portefeuille statique. De plus, le regret est en $O(T)$. L'intuition derrière cette situation est que FTL essaie d'apprendre là où il n'y a rien à apprendre : aucune tendance ne se dégage du marché.

Voyons un dernier exemple pour FTL. Nous simulons 2 actifs qui ont des tendances qui s'inversent au fil du temps.

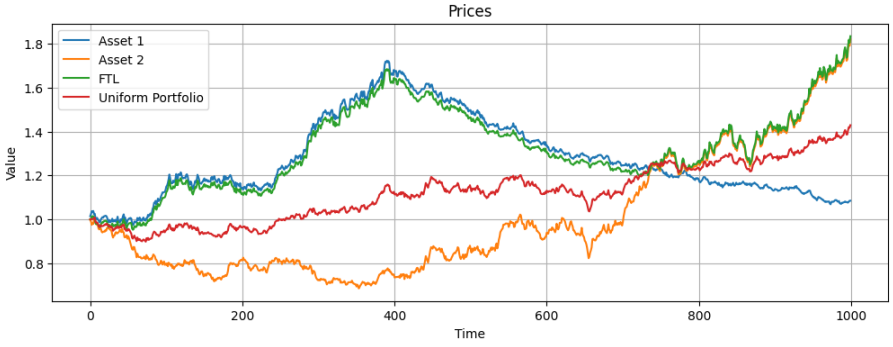


FIGURE 4 – Les limites de FTL

On constate qu'au long terme, le portefeuille donné par la stratégie FTL performe mieux que le portefeuille uniforme, et puisque les actifs 1 et 2 ne se croisent que très peu, il a presque la même performance que le meilleur portfeuille statique, ce qui est raisonnable.

Cependant, un investisseur attentif remarquerait qu'il est possible dr faire beaucoup mieux. À partir de $t = 40$, l'actif 2 a un prix plus faible, mais un rendement bien meilleur, et donc est plus intéressant que l'actif 1. La stratégie optimale ici est de tout miser sur l'actif 1 les 400 premiers tours, puis de tout miser sur l'actif 2. Or FTL a appris durant les 400 premiers pas de temps que l'actif 1 était plus rentable, et ainsi le portefeuille continue à être uniquement constitué de l'actif 1 ensuite même si l'actif 2 est plus rentable. Ce problème s'explique par le fait que FTL accorde autant d'importance aux informations passées il y a longtemps qu'aux informations présentes. Cette remarque nous amène donc à considérer des variantes de l'algorithme Follow-The-Leader.

6 Les variantes de Follow-The-Leader

6.1 FTL avec un seuil : Follow-The-Leader-Epsilon (FTL$\mathcal{E}$)

6.1.1 Description de l'algorithme

Dans cette sous-section, nous explorerons une version légèrement différente de Follow-The-Leader que nous avons imaginée et qui vise à réduire les pertes de Follow-The-Leader lorsque les actifs se croisent plusieurs fois. Nous désignerons ce nouvel algorithme par Follow-The-Leader-Epsilon ou FTL$\mathcal{E}$. Étant donné un seuil $\epsilon > 0$, il s'écrit comme suit :

Follow-The-Leader-Epsilon
ω_0 est fixé arbitrairement.
Pour $T \geq 1$, si on écrit $\chi = \arg\min_{\omega\in\Omega}(\sum_{t=1}^{T-1} f_t(\omega))$ alors on définit $\omega_T = \begin{cases} \chi & \text{si } \lvert \sum_{t=1}^{T-1} (f_t(\omega_{T-1}) - f_t(\chi)) \rvert > \epsilon \\ \omega_{T-1} & \text{sinon} \end{cases}$

Table 2 – L'algorithme Follow-The-Leader-Epsilon

La différence avec Follow-The-Leader est que pour que l'algorithme change de portefeuille pour celui du leader, il faut que ce dernier entraîne une amélioration suffisament intéressante par rapport à la fonction de perte. En particulier, cela signifie que l'algorithme ne changera d'actif que si les retours du meilleur actif sont suffisament meilleurs, afin d'éviter d'éventuelles pertes si jamais il ne s'agissait que de bruit.

6.1.2 Exemple de Follow-The-Leader-Epsilon

Pour cet exemple, on considère un marché composé de 10 actifs différents dont les prix sont générés selon la méthode exposée en section 4.

Bien que cet algorithme semble avoir résolu le problème de la perte d'argent lorsque les valeurs de plusieurs actifs se croisent, il ne résout pas l'un des principaux problèmes de l'algorithme Follow-The-Leader : il cesse d'apprendre avec le temps et devient de moins en moins performant. Par exemple, si l'algorithme avait choisi l'actif orange entre $t = 220$ et $t = 260$, il aurait gagné de l'argent au lieu d'en perdre.

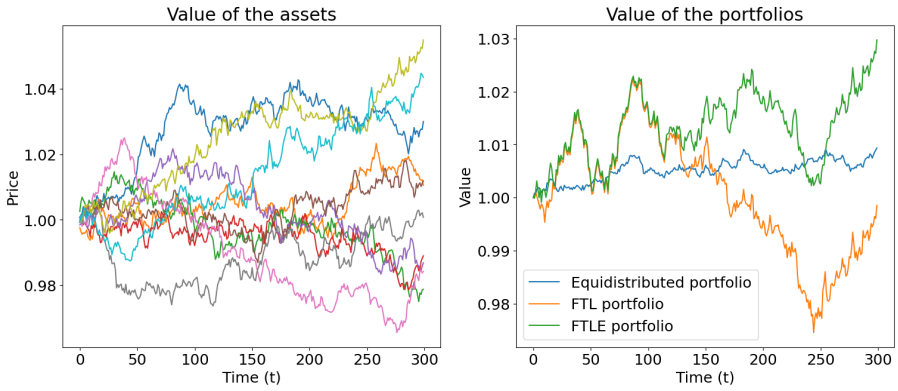


FIGURE 5 – Exemple - FTL$\mathcal{E}$

6.2 FTL avec un facteur d'oubli

L'idée logique après la remarque précédente est d'introduire $\lambda \in [0,1]$ facteur d'« oubli » de dimension $[T^{-1}]$. Le pricipe est le même que FTL, mais les informations se verront attribuer des poids selon leur temporalité. On effectue une *exponential moving average*. Au temps t, le portefeuille est donc le suivant :

$$w_t = e_k \qquad k = \arg\max_{i=1,..,n} \left(\sum_{s=1}^{t-1} \lambda^{t-1-s} r_s \right)_i$$

Voyons ci-dessous une comparaison de FTL avec oubli et FTL dans le cas où le marché est constitué de deux actifs dont les tendances changent.

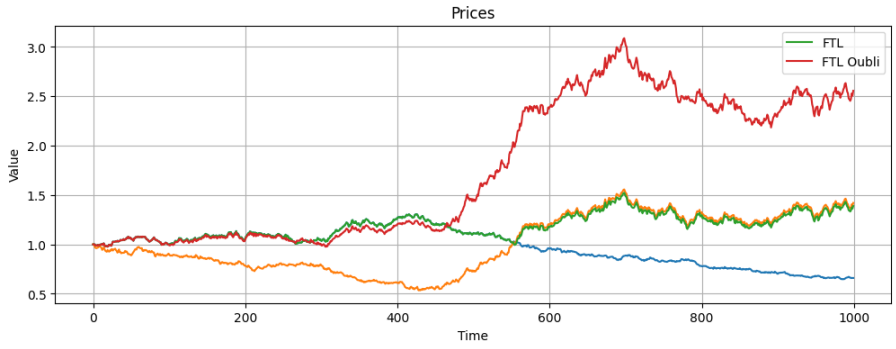


FIGURE 6 – Comparaison FTL Oubli - FTL, $\lambda = 0.95$

On constate que FTL Oubli résout la problématique soulevée à la fin de la partie 7. En effet, il détecte au bout d'un temps raisonnable que l'actif 1 n'est plus intéressant après le temps 400. Lorsque $\lambda = 1$, FTL Oubli se comporte exactement comme FTL. Inversement, si $\lambda = 0$, FTL Oubli regarde uniquement le dernier rendement, il va donc changer de position en permanence et retomber dans les travers de la stratégie naïve. Il s'agit donc de faire un bon compromis sur le facteur d'oubli. Intuitivement, $\lambda = 1 - \frac{1}{T_0}$ où T_0 est la durée moyenne d'une tendance semble être optimal.

6.3 FTL Moving Average

L'idée est assez similaire à FTL Oubli. On prend en compte les m dernières informations (rendements) du marché avec un poids égal. On a donc

$$w_t = e_k \qquad k = \arg\max_{i=1,..,n} \left(\sum_{s=t-1-m}^{t-1} r_s \right)_i$$

La difficulté est encore une fois de choisir le bon paramètre m. Il faut éviter les oscillations de portefeuille mais aussi être capable de réagir rapidement aux tendances du marché. Une étude approfondie des séries temporelle pourrait ensuite permettre de déterminer les paramètres optimaux pour ces algorithmes. Dans le cas ci-dessous, FTL Moving Average est très semblable à FTL Oubli. Nous avons ici choisi de considérer les 20 derniers pas de temps. Ce choix peut être mis en défaut si le marché a une très forte volatilité : le marché va évoluer plus vite que l'apprentissage de notre algorithme.

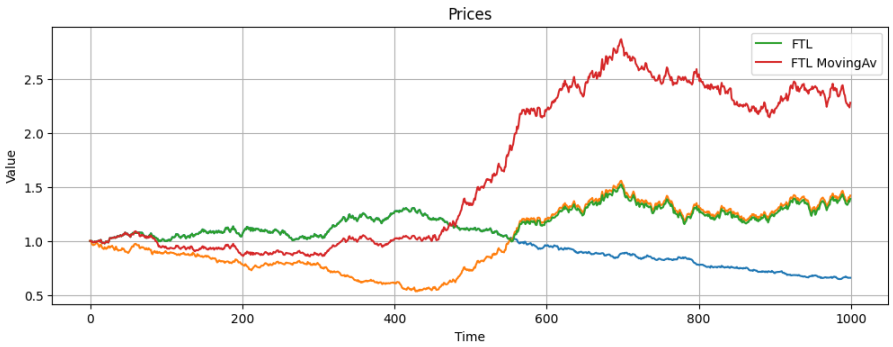


FIGURE 7 – FTL Moving Average, $m = 20$

7 Régularisation

L'ajout d'un facteur de régularisation est l'une des techniques essentielles en apprentissage et optimisation. L'idée générale est d'ajouter un terme de pénalité au problème d'optimisation. Le problème fondamental des algorithmes proposés jusqu'à présent est leur instabilité : FTL choisit toujours un portefeuille de la forme $w_t = (0, .., 1, ..0) = e_k$, c'est-à-dire qu'il mise tout sur une seule action. On sait pourtant qu'un investisseur souhaite minimiser le risque, et pour cela il faut se diversifier. Comparons les volatilités σ des portefeuilles FTL et homogène :

$$w = e_k \implies \sigma_k$$
$$w = \left(\frac{1}{n}, .., \frac{1}{n}\right) \implies \sigma_P = \frac{1}{n}\sqrt{\sum_{i=1}^{n} \sigma_i}$$

Au lieu de passer d'un portefeuille $w_t = e_k$ à $w_{t+1} = e_{k'}$, nous souhaiterions donc une transition plus lisse. L'adaptation sera potentiellement plus lente mais évitera les instabilités.

7.1 Techniques de régularisation

Dans notre étude, nous testerons à la fois les techniques de régularisation $L1$ (*Lasso*) et $L2$ (*Ridge*). Ces techniques ajoutent des termes de pénalité à la fonction de perte pour diminuer la complexité des modèles.

7.2 Régularisation L1

La régularisation $L1$ (*Lasso*) ajoute la norme L_1 de la décision à la fonction de perte. Mathématiquement, elle est définie par :

$$R_{L1}(w) = \lambda \sum_{i=1}^{n} |w_i|$$

où λ est l'hyperparamètre de régularisation qui contrôle sa force.

7.3 Régularisation L2

La régularisation $L2$, aussi connue sous le nom de *Ridge*, est une fonction proportionnelle à la norme euclidienne de notre portefeuille. Mathématiquement, on a

$$R_{L2}(w) = \lambda \sum_{i=1}^{n} w_i^2$$

où λ est le paramètre contrôlant la puissance de la régularisation.

En pratique, le choix entre la régularisation $L1$ et $L2$ dépend des caractéristiques spécifiques du problème et des propriétés souhaitées de la solution. La régularisation $L1$ a tendance à produire des solutions clairsemées en ramenant certaines composantes de ω à zéro, tandis que la régularisation $L2$ a tendance à répartir la pénalité plus uniformément entre tous les composantes de ω.

8 L'algorithme Follow-The-Regularized-Leader (FTRL)

Appliquons la théorie précédemment vue dans le cadre suivant : posons $R : \Omega \to \mathbb{R}^+$ terme de régularisation. Lorsque l'on considère des prédicteurs linéaires, la littérature suggère R de la forme $R(w) = \frac{1}{2\eta}\|w\|_2^2$ où $\eta > 0$ est une constante. Intuitivement, cela va tendre à équilibrer notre portefeuille puisqu'en minimisant $R(w)$ sous la contrainte $\sum_{s=1}^{t-1} w_i = 1$, on trouve aisément $w = (\frac{1}{n}, .., \frac{1}{n})$.

8.1 Définition du problème

Considérons une séquence de tours $t = 1, 2, \ldots, T$, où à chaque tour un décideur doit sélectionner un vecteur ω_t dans un espace Ω. Par la suite, une fonction $f_t : \Omega \to \mathbb{R}$ de perte convexe est révélée, capturant l'écart entre le vecteur choisi et une décision optimale dans le contexte de ce tour. L'objectif est de minimiser la perte cumulée sur T tours.

8.2 Description de l'algorithme

À chaque tour t, l'algorithme FTRL selectionne une décision ω_t en résolvant le problème d'optimisation suivant :

$$\omega_t = \underset{\omega \in \Omega}{\operatorname{argmin}} \left(\sum_{\tau=1}^{t-1} f_\tau(\omega) + R(\omega) \right)$$

Sans perte de généralité nous pouvons considérer que $R(0) = 0$ minimise la fonction de régularisation. Toute fonction de régularisation peut être translatée pour qu'elle soit minimum en 0.

$$\omega_1 = \underset{\omega \in \Omega}{\operatorname{argmin}} R(x) = 0$$

où R est un terme de régularization qui pénalise les décisions trop complexes. Le choix de la fonction de régularisation dépend du contexte et des propriétés désirées de la solution.

Follow-The-Regularized-Leader
ω_0 est fixé arbitrairement.
Pour $T \geq 1$, on fixe ω_T tel que : $w_t \in \arg\min_{\omega \in \Omega}(\sum_{t=1}^{T-1} f_t(\omega) + R(\omega))$

TABLE 3 – L'algorithme Follow-The-Regularized-Leader

NB. On peut monter [2] que sous l'hypothèse $R(w) = \frac{1}{2\eta}\|w\|_2^2$ où $\eta > 0$, alors $\omega_{t+1} = \omega_t + \eta R_t$ (notations de la sous-section 5.2)

8.3 Borne du regret FTRL

Lemme 2. ***Borne du regret FTRL [2]*** *Soient $\omega_1, \ldots, \omega_T$ les vecteurs choisis par l'alrogithme FTRL. Alors, pour tout $u \in \Omega$:*

$$\text{Regret}(u) \leq R(u) + \sum_{t=1}^{T} [f_t(\omega_t) - f_t(\omega_{t+1})]$$

Preuve. Exécutons FTL avec la séquence de fonctions de perte suivante : $f_0 = R, f_1, f_2, \ldots, f_T$. De la manière dont nous avons défini les x_t, nous pouvons garantir que dans cette situation, FTL jouera la même séquence $x_1, x_2, \ldots, x_T$ que FTRL. Grâce au Lemme 1, nous savons que

$$\sum_{t=0}^{T}[f_t(x_t) - f_t(u)] \leq \sum_{t=0}^{T}[f_t(x_t) - f_t(x_{t+1})]$$

En faisant le changement de variable $f_0 = R$, il vient :

$$R(x_0) - R(u) + \sum_{t=1}^{T}[f_t(x_t) - f_t(u)] \leq R(x_0) - R(x_1) + \sum_{t=1}^{T}[f_t(x_t) - f_t(x_{t+1})]$$

Nous savons par ailleurs que $R(x_1) = 0$. En remplaçant et en simplifiant, il vient :

$$\sum_{t=1}^{T}[f_t(x_t) - f_t(u)] \leq R(u) + \sum_{t=1}^{T}[f_t(x_t) - f_t(x_{t+1})]$$

$$\Rightarrow \text{Regret}(u) \leq R(u) + \sum_{t=1}^{T}[f_t(x_t) - f_t(x_{t+1})]$$

8.4 FTRL en pratique

De manière générale, FTRL performe mieux que FTL lorsque les actifs ont la même moyenne.

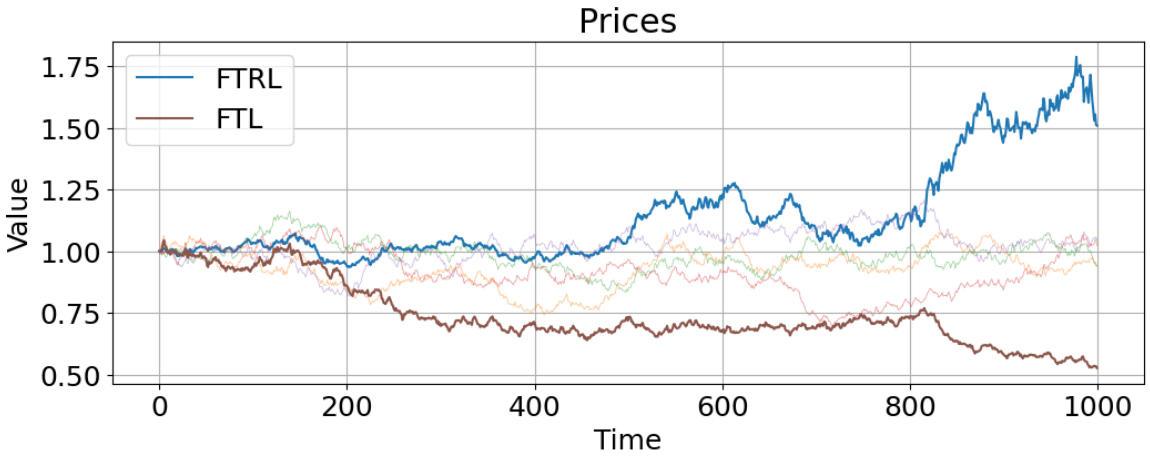


FIGURE 8 – Comparaison de FTRL avec FTL Fonction de perte linéaire

Cependant, ce n'est pas le cas dans la vraie vie car on a toujours une action qui domine les autres.

Comme il est possible de voir sur la figure ci-dessous, en général FTRL performe mieux que le portefeuille équiréparti mais moins bien que FTL. Il faut donc trouver d'autres manières d'améliorer FTL.

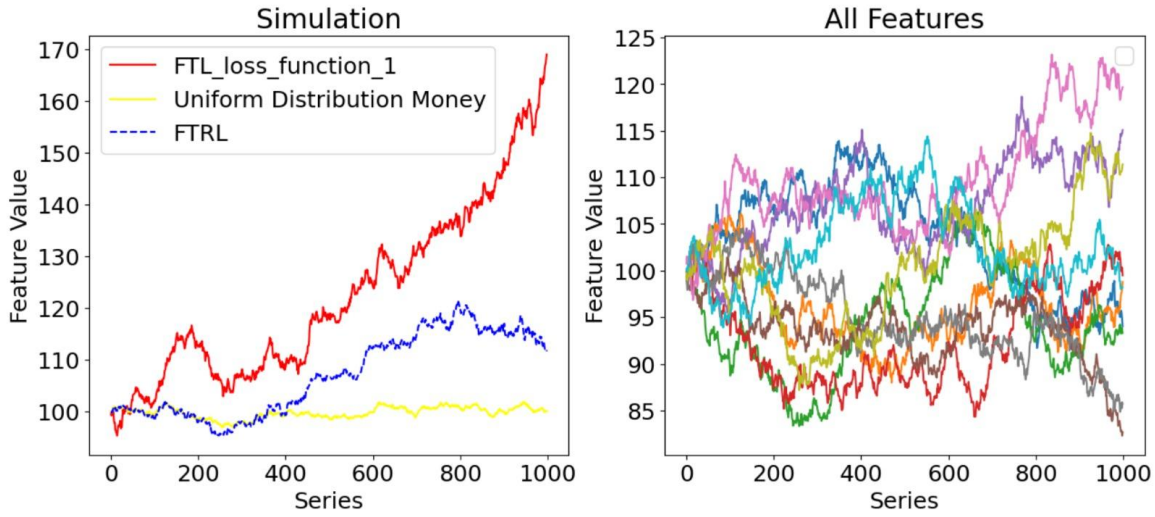


FIGURE 9 – Comparaison de FTRL avec FTL

8.5 Variations de FTRL :

8.6 FTRL avec l'approche de n-SIGMA pour redémarrer l'apprentissage

L'idée de cette approche est de redémarrer l'apprentissage lorsqu'une action sort du domaine (explose ou domine les autres). Dans ce cas, on repart d'un portefeuille qui répartit les investissements sur les actions qui dominent les autres (hors du range de 3 sigma).

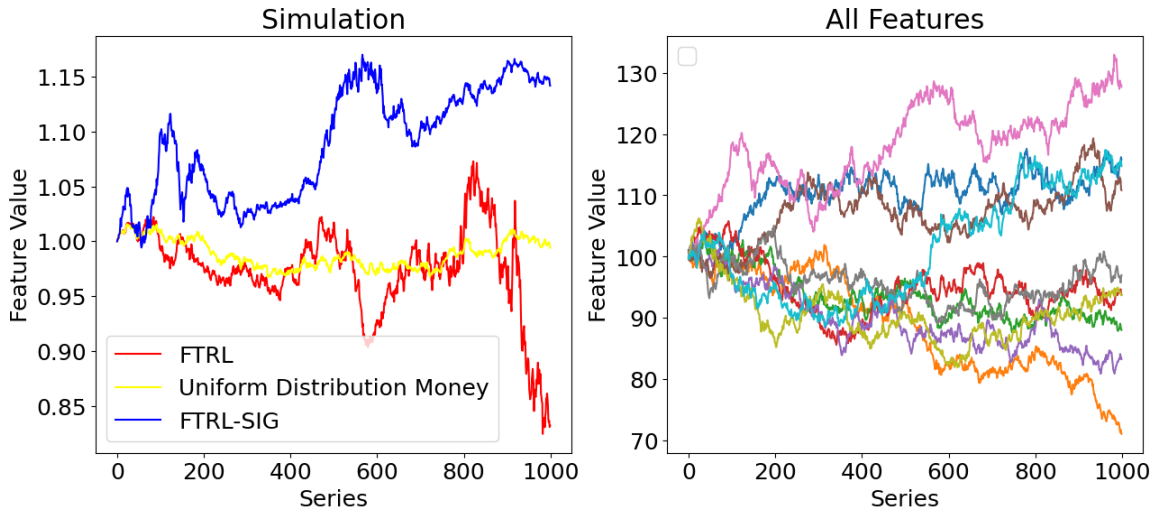


FIGURE 10 – Comparaison de FTRL avec FTRL-SIG

Comme on le remarque ici, quand il y a une action qui domine, FTRL ne performe pas bien. Cependant, avec l'approche de redémarrer l'apprentissage avec la méthode 3 sigma, en se basant sur un portefeuille uniforme des actions qui dominent (hors du domaine de 3 sigma), FTRL performe beaucoup mieux.

9 Redémarrage des algorithmes : l'approche de l'écart-type

Comme vu précédemment, l'un des problèmes principaux de Follow-The-Leader et de ses dérivés est qu'ils arrêtent d'apprendre avec le temps : les nouvelles informations ont de moins en moins d'impact sur le vecteur de poids ω. Dans cette section, nous allons détailler un criètre pour déterminer quand redémarrer l'algorithm afin que les nouvelles informations aient plus d'impact sur les prédictions.

9.1 Idée de l'approche

L'intuition justifiant le redémarrage est la suivante : si l'algorithme a une bonne performance, il n'est pas très important de prendre en compte les nouvelles informations. Cependant, si l'algorithme perd en performance, il faut plutôt oublier les anciennes informations, qui sont vraisemblablement devenues dépassées et erronnée. L'objectif qui en découle est le suivant : il faut détecter lorsque les algorithmes arrêtent d'être performants afin de les « redémarrer », c'est-à-dire leur faire oublier toutes les anciennes informations.

Pour cela, on considère la moyenne du portefeuille de l'algorithme depuis sont lancement μ ainsi que son écart type σ, et on redémarre l'algorithme lorsque la valeur du portefeuille de l'algorithme sort du segment $[\mu - \alpha_1 \cdot \sigma, \mu + \alpha_2 \cdot \sigma]$, où α_1 et α_2 sont des réels strictement positifs.

Nous pourrions noter une limite à cette approche : bien que ce critère permet de détecter des changements rapides de la valeur du portefeuille de l'algorithme, ces changements pourraient être causés par une très bonne performance de l'algorithme. Nous notons que ce cas ne saurait avoir un impact négatif puisque dès lors que l'algorithme redémarre, il choisit l'actif avec les meilleurs retours, et devrait donc avoir une performance au moins aussi bonne que précédemment.

9.2 Exemples avec FTL$\mathcal{E}$

Voyons ce que cela donne en pratique. Pour ces exemples, nous considérons 10 actifs différents simulés comme décrit dans la section 4. Les prix sont normalisés et il n'y a pas de frais de transaction pour l'achat d'un actif. On obtient les résultats suivants :

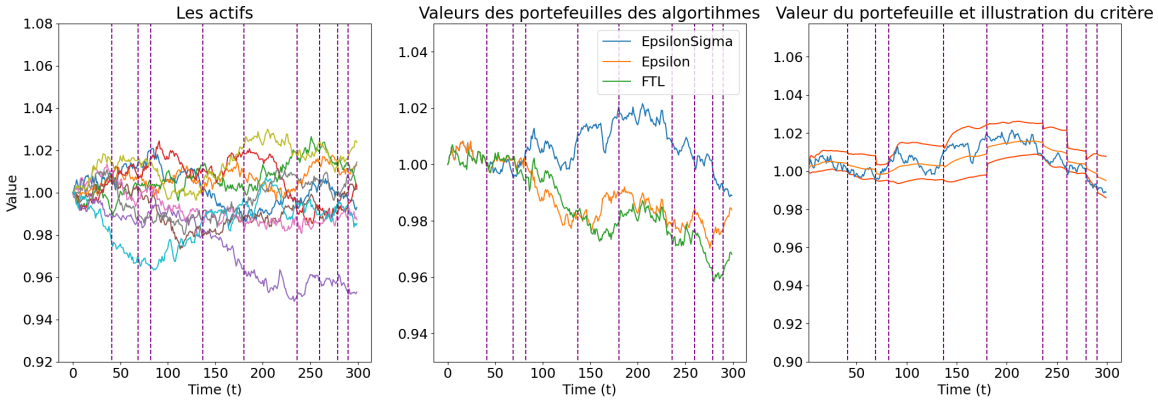


FIGURE 11 – Approche de l'écart type avec FTL$\mathcal{E}$

Ici, « EpsilonSigma » représente FTL$\mathcal{E}$ auquel on a appliqué le critère et « Epsilon » représente FTL$\mathcal{E}$. On a pris $\alpha_1 = \alpha_2 = 1.8$. les barres verticales représentent les instants auquels l'algorithme redémarre.

On observe dans cet exemple (figure 11) que le critère permet d'éviter une importante perte. Cepenandant, ce dernier n'a pas empêché l'algorithme FTL$\mathcal{E}$ de perdre de l'argent.

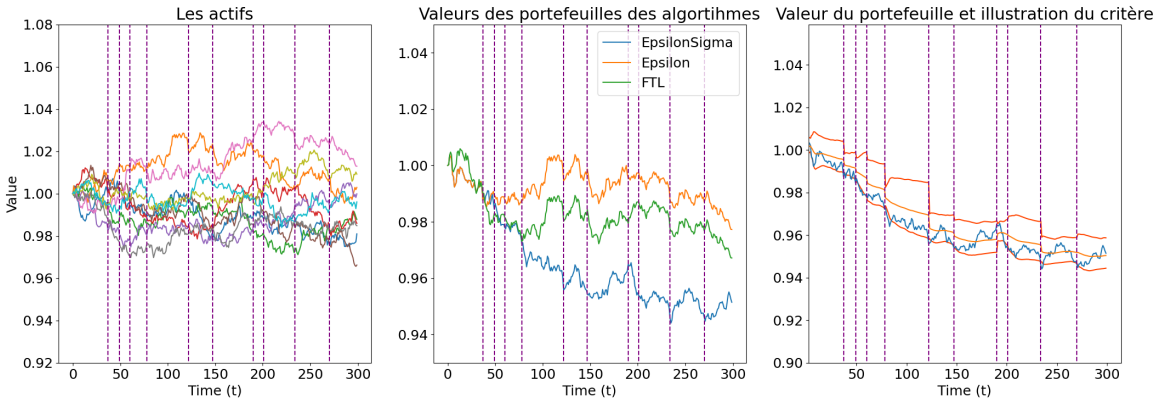


FIGURE 12 – Approche de l'écart type avec FTL$\mathcal{E}$

Cependant, le redémarrage n'est pas toujours aussi intéressant. Dans l'exemple suivant (figure 12), le critère provoque des pertes : bien que l'algorithme redémarre il, n'est pas capable de déterminer quel est actuellement le meilleur actif à choisir pour maximiser les gains. Par ailleurs, ce critère semble parfois trop sensible et d'autres fois pas suffisament : le redémarrage s'effectue sur de petites flucutations fréquentes, mais pas sur de plus longues pertes.

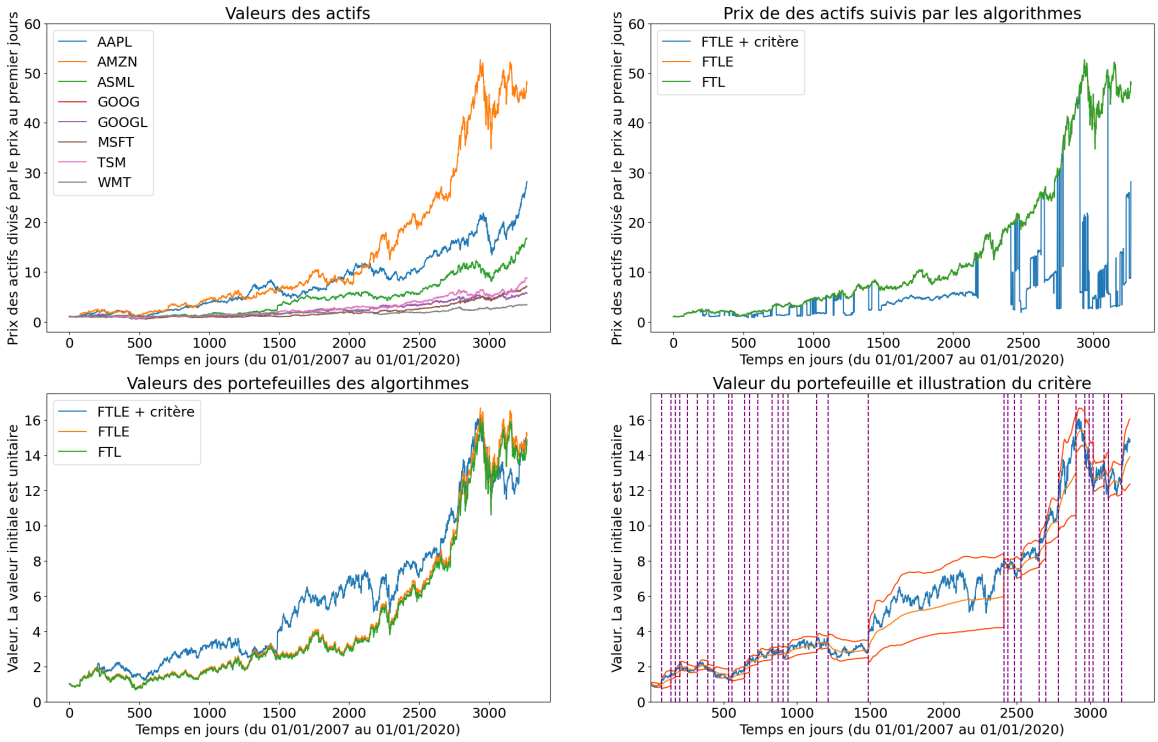


FIGURE 13 – Approche de l'écart type avec FTL$\mathcal{E}$ sur des données réelles

Finalement, sur le dernier exemple (figure 13), nous avons respectivement appliqué FTL$\mathcal{E}$ avec le critère de redémarrage, sans le critère de redémarrage et FTL à des données réelles. On observe que contrairement aux exemples précédents, les différents algorithmes se comportent très bien. Cela s'explique par le fait que sur les marchés financiers, un actif sort souvent du lot sur le long terme et performe mieux que les autres. Ceci a pour conséquence que les différents problèmes mis en évidence dans notre étude n'ont qu'un impact minime sur de longues échelles de temps.

Deuxième partie

Semestre 7 : Portefeuilles universels et Applications.

10 Introduction à l'algorithme des portefeuilles universels

10.1 Contexte

Le contexte est le suivant.

On considère un marché avec d actifs, discrétisé en T tours. Le cadre est celui de l'apprentissage en ligne (comme formalisé au semestre précédent). À chaque tour, on prédit un portefeuille $w_t \in \Delta_d$, avec

$$\Delta_d = \left\{ w \in \mathbb{R}^d \mid \sum_{i=1}^{d} w_i = 1 \right\}.$$

Le marché engendre un vecteur $x_t \in \mathbb{R}^d$ de rendements (*returns*), où $x_{i,t}$ indique la performance de l'actif i au temps t. On calcule alors une perte, définie comme une fonction convexe :

$$\ell_t(w) := -\log(x_t^\top w).$$

Ces opérations se répètent pendant T tours. L'objectif du joueur est de minimiser son regret :

$$\mathcal{R}_T(w_{1:T}, x_{1:T}) := \sum_{t=1}^{T} \ell_t(w_t) - \min_{w \in \Delta_d} \sum_{t=1}^{T} \ell_t(w).$$

10.2 Algorithme *Universal Portfolios*

En 1996, Thomas Cover publie un article révolutionnaire : *Universal Portfolios* [9]. Il y décrit un algorithme répondant au problème ci-dessus, garantissant que la richesse créée au tour n notée $\hat{S}_n$, a le même exposant au premier ordre que la richesse créée par le meilleur portefeuille constant ($S_n^* = \max_{w \in \Delta_d} \prod_{t=1}^{n} (w^\top x_t)$), et ce sans aucune hypothèse sur les rendements du marché !

En général, cet algorithme assure une croissance exponentielle de la richesse. Son fonctionnement est expliqué ci-dessous.

Pour $n \in [\![1, T]\!]$, on note

$$S_n(w) = \prod_{t=1}^{n} (w_t^\top x_t).$$

la richesse créée au temps n par la suite de portefeuilles $(w_t)_{1 \le t \le n}$.

Le premier tour, nous n'avons aucune information à notre disposition. Il est donc assez naturel de choisir le portefeuille uniforme $w_1 = \left(\frac{1}{d}, \ldots, \frac{1}{d}\right)$. L'idée intéressante vient ensuite. Pour $t \in [\![1, T]\!]$, on choisit le portefeuille

$$w_{t+1} = \frac{\int_{\Delta_d} w\, S_t(w)\, dw}{\int_{\Delta_d} S_t(w)\, dw}$$

On intègre sur l'ensemble des portefeuilles possibles $\Delta_d = \left\{ w \in \mathbb{R}^d \mid \sum_{i=1}^{d} w_i = 1 \right\}$ afin de visualiser cette formule. Le dénominateur est un facteur de normalisation. Il permet à notre

nouveau portefeuille de bien se situer dans Δ_d. Aussi, l'intégrale au numérateur exprime la moyenne pondérée par $S_t(w)$, c'est-à-dire le gain du portefeuille w jusqu'à présent. Il est aussi possible de le voir comme une espérance sur Δ_d, où la distribution est donnée par S_t.

On remarque que l'on définit w_{t+1} à l'aide des données obtenues jusqu'au temps t, ce qui est bien endémique d'un contexte d'Online Learning. Mais pourquoi cette idée fonctionne-t-elle ? Intuitivement, on accorde plus de poids aux portefeuilles qui auraient bien performé jusqu'à présent, et l'on pense donc qu'ils vont continuer à bien performer. À l'inverse, ceux qui ont très peu performé sont effacés par la pondération, et l'on se retrouve donc avec un portefeuille "gagnant".

L'article de Thomas Cover [9] montre rigoureusement cette estimation : on observe

$$\frac{1}{n}\ln \hat{S}_n - \frac{1}{n}\ln S_n^* \to 0, \tag{1}$$

pour une suite de rendements arbitraire bornée $\mathbf{x}_1, \mathbf{x}_2, \ldots$. Ainsi, $\hat{S}_n$ et S_n^* ont le même exposant au premier ordre.

Une analyse plus précise donne

$$\hat{S}_n \sim \sqrt{\frac{2\pi}{nJ_n}} S_n^*, \tag{2}$$

où J_n est le déterminant d'une jacobienne liée à la volatilité du marché.

On peut réécrire l'équation (1) sous la forme

$$\frac{1}{n}\left[\sum_{t=1}^{n} \log(w_t^\top x_t) - \min_{w\in\Delta_d} \sum_{t=1}^{n} \log(w^\top x_t)\right] \to 0.$$

Cela signifie, en reprenant la définition du regret,

$$\mathcal{R}_T(w_{1:T}, x_{1:T}) = o(n) \quad \text{lorsque } n \to \infty.$$

Ainsi, le regret est sous-linéaire, ce qui signifie que l'algorithme est capable d'apprendre au fil du temps.

Généralement, les algorithmes d'apprentissage statistique admettent qu'une loi de probabilité régit la distribution des rendements et que la suite des rendements (x_t) est i.i.d. On cherche donc à estimer la loi de probabilité. Dans ce cas, on connaît la stratégie optimale, sous le nom de *Constant Rebalanced Portfolios*. Ce qui est intéressant dans l'algorithme UP, c'est que la distribution des rendements est complètement arbitraire et peut être choisie par le marché de manière contradictoire à notre portefeuille et offre quand une garantie sur le regret. C'est de là que provient le nom de « portefeuilles universels ».

10.3 Implémentation, applications et difficultés

Un algorithme offrant des estimations aussi impressionnantes sur le regret avec autant de généralité (on demande seulement que les rendements soient bornés) se doit d'être implémenté, testé et mis en production ! On remarque immédiatement une première difficulté : chaque itération nécessite le calcul d'une intégrale. Notre ordinateur n'est pas capable de faire ca ; il faut donc compter sur des approximations. Heureusement, il existe de nombreuses méthodes pour calculer des intégrales (méthodes des rectangles, des trapèzes ou de Monte-Carlo). Mais un autre problème se pose : l'intégration se fait sur un sous-ensemble (certes connexe et convexe), mais dans $\mathbb{R}^d$! Le fléau de la dimension nous guette, et l'on se voit déjà difficilement appliquer ces algorithmes à 50 titres financiers. En effet, l'implémentation la plus efficace connue à ce jour est due à Kalai et Vempala [5], et a pour complexité $O(d^4(T+d)^{14})$ ce qui rend inenvisageable la mise en production.

Nous avons implémenté *Universal Portfolios* sur Python. L'idée est la suivante : nous discrétisons l'intégrale en calculant des portefeuilles qui remplissent uniformément Δ_d, puis nous calculons le gain pour chaque portefeuille jusqu'au temps t et enfin, nous moyennons. Évidemment, le temps de calcul augmente exponentiellement (en la dimension) en fonction du niveau de granularité (c.-à-d. la précision à laquelle on calcule notre intégrale).

En nous inspirant fortement de [4], nous avons implémenté puis appliqué *Universal Portfolio* sur 5 titres financiers du *S&P500* entre 2016 et 2020 qui ne sont pas complètement corrélés.

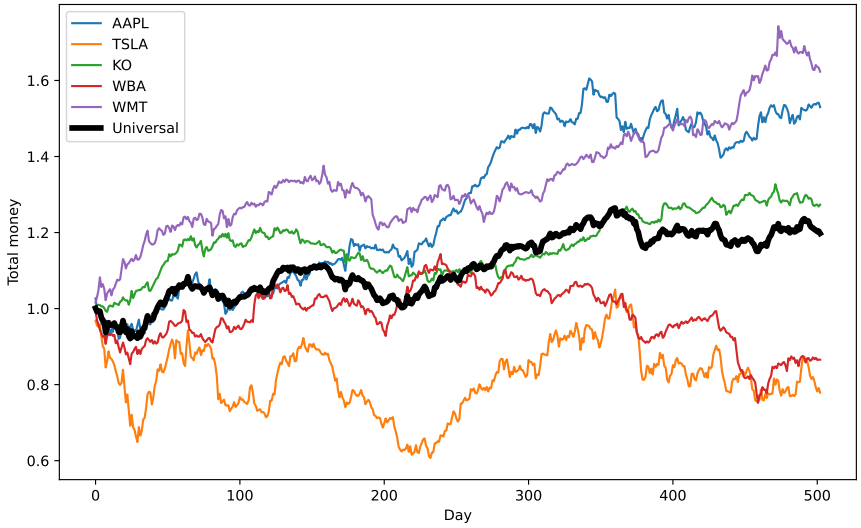


FIGURE 14 – Performance (en noir) de Universal Portfolio ($S_0 = 1$)

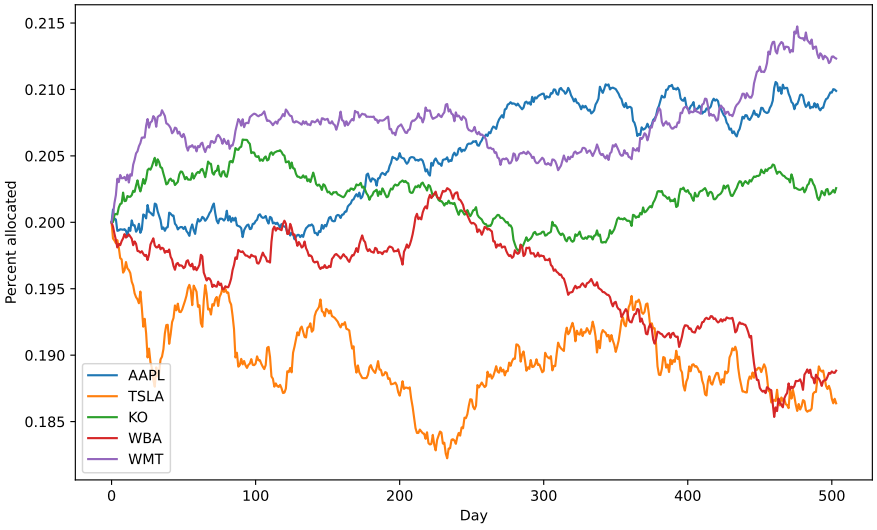


FIGURE 15 – Allocation du portefeuille en fonction du temps.

N.B. Les informations sur le temps de calcul sont les suivantes. Pour $d = 5$ actifs, $T = 500$ périodes de temps il a fallu, à un ordinateur muni d'un Intel i5-7500 4 coeurs à 3.8 GHz, 36 secondes lorsque les intégrales sont calculées en moyennant sur 10 000 portefeuilles, et 7 minutes lorsqu'elles le sont sur 135 000 portefeuilles.

L'implémentation est très naïve, et il est possible d'être bien plus efficace, mais on perd énormément en précision si l'on veut quelque chose d'un peu efficace et que cet algorithme ne pourra jamais être mis en production pour $d = 50$ actifs.

On observe un phénomène intéressant : le portefeuille suit correctement les actifs ayant généré les meilleurs rendements jusqu'à présent, mais le facteur d'apprentissage semble trop faible. Cela entraîne une difficulté à s'éloigner d'un portefeuille uniforme, traduisant un apprentissage trop lent. Ce problème est particulièrement critique dans un marché soumis à de fortes fluctuations.

Universal Portfolio se concentre sur la croissance linéaire des rendements. Pour qu'un actif prédomine dans l'allocation, il doit croître de manière exponentielle en valeur. Une solution consiste à ajouter un facteur dans le calcul des gains,

$$S_t(w) = \exp\left(\lambda \sum_{k=1}^{t-1} \log(w_k^\top x_k)\right)$$

avec $\lambda > 0$ une constante. Plus λ est élevé, plus le portefeuille varie rapidement. Une idée pour déterminer une valeur optimale de λ pourrait être de s'intéresser à la volatilité ou au volume du marché. En effet, un volume de marché élevé reflète des ajustements fréquents des portefeuilles par les acteurs, ce qui implique que notre modèle doit également être capable de réagir rapidement.

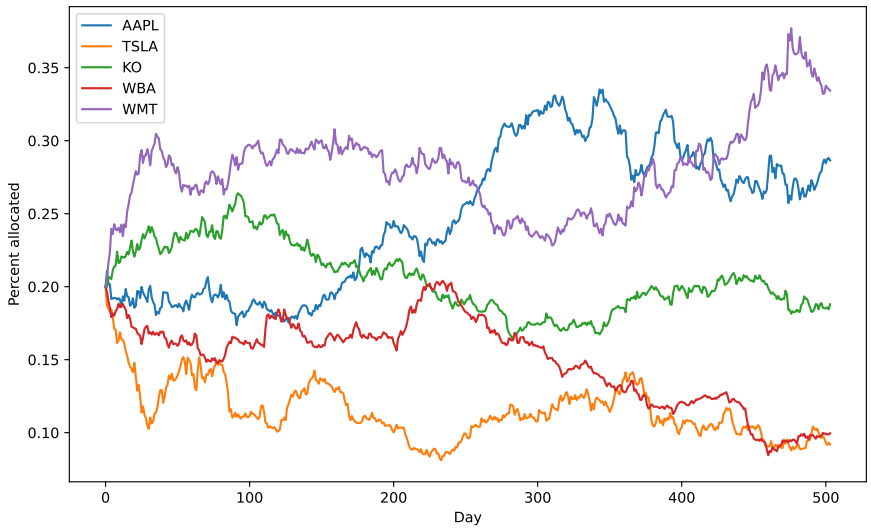


FIGURE 16 – Allocation du portefeuille en fonction du temps ($\lambda = 10$).

10.4 Une première approche du redémarrage

Dans le chapitre 9, nous avons étudié l'intérêt du redémarrage, ainsi que les moyennes glissantes et exponentielles appliquées à FTL/FTRL (chapitre 6). Dans le cas de l'Universal Portfolio, le portefeuille au temps $t + 1$ est calculé en moyennant les gains des instants 0 à t. Cependant, sur des marchés dynamiques, il peut être avantageux d'accorder davantage d'importance aux instants proches de t plutôt qu'aux instants plus éloignés.

Une solution consiste à utiliser une moyenne mobile exponentielle. On introduit une constante $0 < \lambda < 1$, et on redéfinit le calcul des gains comme suit :

$$S_t(w) = \exp\left(\sum_{k=1}^{t-1} \lambda^{t-1-k} \log(w_k^\top x_k)\right)$$

Ce mécanisme permet de pondérer les gains récents de manière plus significative, car $\lambda^n \to 0$ lorsque $n \to +\infty$. Ainsi, les gains importants proches de t influencent davantage le calcul du portefeuille, ce qui améliore son adaptabilité à des marchés volatils.

Une autre idée serait de faire une moyenne glissante : On ne considère que les temps entre $t-m$ et $t-1$ où m est une fenêtre fixée. Le calcul des gains est donc

$$S_t(w) = \exp\left(\sum_{k=t-m}^{t-1} \log(w_k^\top x_k)\right)$$

Voici ci-dessous l'exécution de cette nouvelle méthode avec $m = 10$ et le même exemple que précédemment.

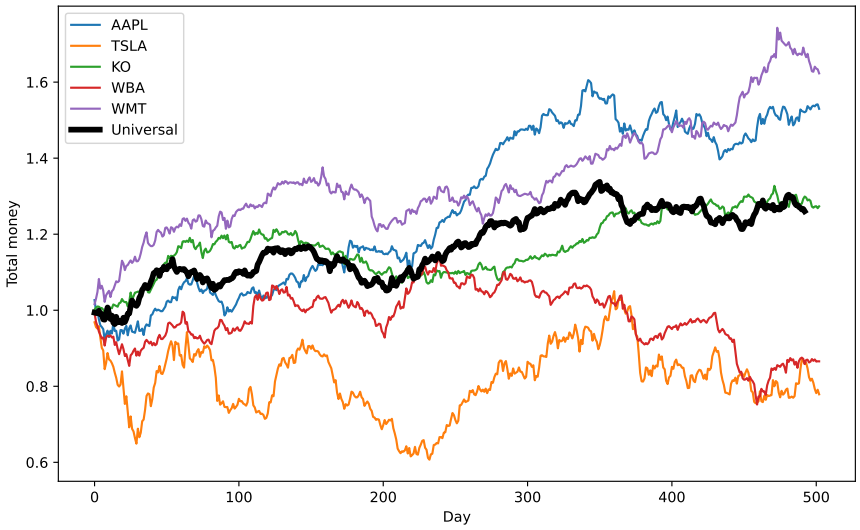


FIGURE 17 – Performance (en noir) de *Universal Portfolio* avec moyenne glissante ($m = 10$)

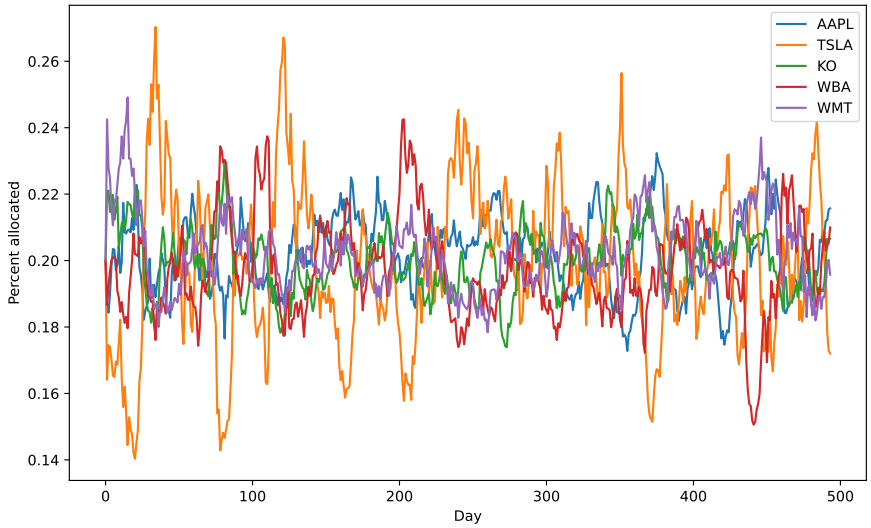


FIGURE 18 – Allocation du portefeuille en fonction du temps

On constate que les gains augmentent légèrement, mais surtout que notre portefeuille varie beaucoup plus, ce qui est très cohérent. Cependant, nous n'avons pas pris en compte les frais de transaction, et dans le monde réel, avoir des mouvements aussi brusques de portefeuille peut entraîner de nombreux frais.

11 L'algorithme VB-FTRL-qN

11.1 Principe de l'algorithme

11.1.1 Approche

Nous avons vu les difficultés rencontrées lors de l'application réelle de *Universal Portfolio*. Cependant, l'idée reste toujours très intéressante. Une approche alternative consiste à utiliser des méthodes d'approximation pour trouver un compromis entre le temps de calcul et la précision. C'est précisément ce que propose l'article de Jézéquel, Ostrovskii et Gaillard [8], que nous avons étudié en détail.

L'idée de Kalai et Vempala [5] est la suivante : pour approximer une intégrale sur Δ_d, on effectue un passage à un espace de dimension inférieure. Concrètement, on passe de

$$\Delta_d := \left\{ w \in \mathbb{R}_+^d : \mathbf{1}_d^\top w = 1 \right\}$$

au simplexe solide dans $\mathbb{R}^{d-1}$ défini par :

$$\Delta_{d-1} := \left\{ v \in \mathbb{R}_+^{d-1} : \mathbf{1}_{d-1}^\top v \leq 1 \right\}$$

En passant de Δ_d à Δ_{d-1}, on libère une place pour la composante w_d, ce qui simplifie les calculs.

On définit une bijection linéaire entre Δ_d et Δ_{d-1}, donnée par

$$v \mapsto Av + e_d$$

où

$$A = \begin{pmatrix} I_{d-1} \\ -\mathbf{1}^\top \end{pmatrix}, \quad e_d = (0, 0, \ldots, 1)$$

avec e_d le $d$ième vecteur de la base canonique. On peut obtenir la réciproque de cette bijection à l'aide de la pseudo-inverse de A, calculée aisément : $A^+ = (A^\top A)^{-1}A^\top$. Cette transformation permet déjà de passer de la dimension d à la dimension $d-1$, mais surtout d'avoir un simplexe solide qui permet un calcul probabiliste par une marche aléatoire. L'idée de [5] reste tout de même impossible à mettre en production, c'est pourquoi nous allons nous intéresser à des approximations de cette dernière. Un exemple d'une telle approximation est VB-FTRL-qN.

11.1.2 Algorithme

La perte $\ell_t(w) := -\log(x_t^\top w)$ étant convexe, on se ramène à un problème d'optimisation convexe. Nous avons donc, pour $t \in [\![1, T]\!]$,

$$w_{t+1} = \operatorname*{argmin}_{w \in \Delta_d} P_t(w). \tag{VB-FTRL}$$

où P_t est un potentiel. On reconnaît le cas FTRL exploré dans la partie 8.

Barrière logarithmique R

Soit $\lambda, \mu > 0$. Nous allons introduire différentes quantités. La fonction $R : \mathbb{R}^d \to \mathbb{R} \cup \{+\infty\}$ est appelée *barrière logarithmique*. Intuitivement, elle empêche que le portefeuille ne sélectionne qu'une seule action. En effet, si $w = e_i = (0, \dots, 0, 1, 0, \dots, 0)$, alors $R(w) = +\infty$.

Sans rentrer dans les détails théoriques, on peut voir R comme une fonction définie sur Δ_{d-1}. De plus, $R(w)$ diverge lorsque w s'approche de la frontière de Δ_{d-1}. Cela assure une certaine stabilité de l'algorithme, de même que l'interdiction de poids négatifs.

$$R(w) := \begin{cases} -\sum_{i=1}^d \log(w[i]) & \text{si } w \in \mathbb{R}^d_{++}, \\ +\infty & \text{sinon.} \end{cases}$$

Fonction de perte régularisée L_t

La fonction $L_t(w)$ est la perte cumulative classique à laquelle on a ajouté la régularisation $R(w)$. Cette approche est standard en Machine Learning, comme dans les cas de la régression Ridge ou Lasso.

$$L_t(w) := \sum_{\tau=1}^{t} \ell_\tau(w) + \lambda R(w).$$

Barrière volumétrique V_t

La fonction $V_t : \mathbb{R}^d \to \mathbb{R} \cup \{+\infty\}$ est appelée *barrière volumétrique*. Elle est définie par :

$$V_t(w) := \begin{cases} \frac{1}{2}\log\det(A^\top \nabla^2 L_t(w) A) & \text{si } w \in \mathbb{R}^d_{++}, \\ +\infty & \text{sinon.} \end{cases}$$

Interprétation physique de V_t

Puisque L_t est convexe et deux fois différentiable, sa hessienne $\nabla^2 L_t(w)$ est une matrice symétrique dont toutes les valeurs propres sont strictement positives. Cela garantit que $V_t(w)$ est bien définie (en posant $\log(0) = +\infty$).

- On sait que $\det(M)$ correspond au produit des valeurs propres de M. Par conséquent, on peut interpréter $V_t(w)$ comme

$$V_t(w) = \sum_{i=1}^{d-1} \log(\lambda_i)$$

où les λ_i sont les valeurs propres de la hessienne $\nabla^2 L_t(w)$, après passage dans Δ_{d-1} via les matrices $A^\top$ et A.

En théorie du calcul différentiel, si a est un minimum local de f, alors $\nabla^2 f(a)$ a ses valeurs propres strictement positives. Minimiser V_t permet de contrôler ces valeurs propres, ce qui est essentiel pour :

1. Assurer la convergence des algorithmes de Newton (liée au conditionnement de la matrice hessienne).
2. Résoudre des problèmes de stabilité numérique, car l'algorithme nécessite d'inverser $\nabla^2 L_t(w)$. Une valeur propre nulle rendrait cette inversion impossible, mais la présence de V_t empêche ce cas.

Potentiel P_t

Enfin, nous définissons le potentiel P_t, qui est une fonction convexe. Cela garantit l'existence d'un minimum global

$$P_t(w) := L_t(w) + \mu V_t(w)$$

11.1.3 Approximation de Universal Portfolios

Pour mieux comprendre VB-FTRL-qN, nous allons montrer que l'algorithme est en fait une approximation de *Universal Portfolios*.

Rappelons que dans le cadre de *Universal Portfolios*,

$$w_{t+1} \propto \int_{\Delta_d} w\, \phi_t(w)\, dw \quad \text{avec} \quad \phi_t(w) \propto \exp\left(-\sum_{\tau=0}^{t-1} \ell_\tau(w)\right), \quad w \in \Delta_d$$

Nous allons donc, après avoir introduit $\lambda, \mu > 0$, poser

$$w_{t+1} \propto \int_{\Delta_d} w\, \phi_t(w)\, dw \quad \text{avec} \quad \phi_t(w) \propto \exp\left(-\frac{1}{\mu} L_{t-1}(w)\right)$$

On rappelle que le paramètre λ est présent dans : $L_t(w) := \sum_{\tau=1}^{t} \ell_\tau(w) + \lambda R(w)$.

Que signifient physiquement ces paramètres ?

— Lorsque λ est grand, le poids accordé à $R(w)$, la barrière logarithmique est amplifiée. On va donc biaiser le portefeuille w vers le portefeuille uniforme $\frac{1}{d}\mathbf{1}_d = (\frac{1}{d}, ..., \frac{1}{d})$

— Lorsque μ est grand, on a $\frac{1}{\mu} L_{t-1}(w)$ qui va tendre vers 0, et donc ϕ_t va avoir tendance à être uniformisé sur Δ_d (c'est en fait le cas, à une constante prêt, ϕ_t est la distribution de probabilité uniforme sur Δ_d, c'est à dire qu'il va donner plus de chances à d'autres portefeuilles et moins s'intéresser aux retours passés.

On sait déjà que *Universal Portfolios* assure un regret sous-linéaire (et en fait logartihmique en fonction du temps) [9]. Nous ne rentrons pas dans les détails mathématiques, mais l'article [8] montre exactement que pour tout $\lambda, \mu > 0$, l'algorithme dérivé VB-FTRL-qN assure aussi un regret logarithmique. C'est ce que dit le théorème ci-dessous :

Théorème 1. Pour tout $T \in \mathbb{N}$ et toute suite de rendements bornées $x_{1:T}$, l'algorithme VB-FTRL-qN donne une suite de portefeuilles $\widetilde{w}_{1:T}$ tels que

$$\mathcal{R}_T(\widetilde{w}_{1:T} | x_{1:T}) \leq (\lambda + 2\mu)\left[(d-1)\log(T + \lambda d) + 1\right]$$

Dire que le regret est sous-linéaire c'est dire que l'on apprend. Un regret logarithmique est d'autant plus remarquable, rappelons que FTRL donnait un regret en $O(\sqrt{T})$

11.1.4 Implémentation théorique et méthode(s) de quasi-Newton

Ce qui précède nous donne une extension du cadre de *Universal Portfolios*. Il faut maintenant pouvoir implémenter en pratique cet algorithme, avec une complexité assez raisonnable pour pouvoir le mettre en production.

On cherche donc à appliquer des méthodes d'optimisation convexe pour caculer $w_{t+1} = \text{argmin}_{w\in\Delta_d} P_t(w)$. Parmi celles-ci, il existe les méthodes de descente de gradient, aussi connues sous le nom de méthodes de Newton. On va ici utiliser une méthode de quasi-Newton expliquée dans [10].

On sait que si f est différentiable et $\nabla f(x) = 0$ alors x est un extremum local de f. De plus si f est convexe, cet extremum est un minimum.

La méthode de Newton classique est : $x_{k+1} = x_k - H(x_k)^{-1}\nabla f(x_k)$ où $H(x_k)$ est la hessienne de f en x_k.

Par calcul, on trouve facilement :

$$\nabla V_t(w) = \sum_{\tau=0}^{t} \pi_\tau(w)\nabla\ell_\tau(w)$$

En revanche, le calcul de la hessienne est coûteux en calcul ($O(d^2(T+d)^2)$). On utilise donc des méthodes de quasi-Newton qui consistent à approximer la hessienne. C'est pour cela que l'on a l'estimation suivante :

On pose la relation d'ordre suivante sur les matrices symétriques : $A \preceq B \iff$ les coefficients de $B - A$ sont tous positifs. La hessienne de $V_t(w)$ vérifie les bornes $Q_t(w) \preceq \nabla^2 V_t(w) \preceq 3Q_t(w)$ où $Q_t(w)$ est définie par

$$Q_t(w) = \sum_{\tau=0}^{t} \pi_\tau(w)\nabla^2\ell_\tau(w) \quad \text{calculable en } O(d^2(T+d))$$

Le théorème 1 garantit qu'il suffit d'effectuer $O(\log(T+d))$ étapes de Newton pour assurer la borne de regret, jusqu'à un facteur constant. L'article indique que l'algorithme a un temps d'exécution de $O(d^2T + d^3)$ par étape de Newton, ce qui donne un temps d'exécution global de $O(d^2T + d^3)$ par tour. On peut verifier l'estimation avec une vérification du algorithme. Nous analysons le temps d'exécution à chaque tour t :

1. **Passage dans Δ_{d-1}** : Traduire w_{t-1} en v_{t-1} prend $O(d)$ de temps.
2. **Calcul du gradient** : Le calcul des gradients $\nabla_\tau = \nabla\ell_t^{\mathbb{1}}(v_t^{(s)})$ pour t points de données et d termes barrières prend $O(dt + d^2)$.
3. **Calcul de la Hessienne** : La formation de $H_{t-1}^{(s)}$ comme somme de $O(t+d)$ matrices de rang un prend $O(d^2T + d^3)$.
4. **Inversion de matrice** : L'inversion de $H_{t-1}^{(s)}$ prend $O(d^3)$.
5. **Calcul des scores de levier** : Le calcul des scores de levier avec $H_{t-1}^{(s)^{-1}}$ coûte $O(td^2)$.
6. **Matrice $M_{t-1}^{(s)}$** : La construction et l'inversion de $M_{t-1}^{(s)}$ prend $O(d^3)$.
7. **Multiplication matrice-vecteur** : La multiplication finale prend $O(td^2 + d^3)$.

Donc, on a que le tour t prend $O(td^2 + d^3)$. De cela, nous pouvons déduire que le temps d'exécution moyen par tour pour T tours est $O(d^2T + d^3)$.

11.1.5 Performance Théorique

La borne de regret présente une croissance logarithmique, ce qui signifie qu'elle augmente de manière sous-linéaire avec le temps. Cette caractéristique est cruciale car elle démontre la capacité de l'algorithme à apprendre de manière efficace au fil du temps. Puisque le regret croît plus lentement que le temps lui-même, le regret par itération finit par converger vers zéro à mesure que le temps progresse. Cela met en évidence la capacité de l'algorithme à prendre des décisions de plus en plus précises à mesure que l'horizon temporel s'étend, confirmant sa capacité à apprendre à long terme.

Cette analyse montre que l'algorithme VB-FTRL-qN conserve les propriétés d'apprentissage souhaitables de VB-FTRL, même avec l'approximation employée dans le processus d'optimisation pour le Hessien de la barrière volumétrique.

11.2 Implémentation en python

Sur un conseil de notre encadrant M. Challet, nous avons tout d'abord tenter d'implémenter l'algorithme en python à l'aide de ChatGPT. Malheureusement l'algorithme était trop complexe pour lui : il n'était pas capable de l'écrire en entier, et ce qu'il écrivait était très peu optimisé. Cela est en particulier causé par le fait que le pseudo-code était (peut-être volontairement de la part de l'auteur du papier) compliqué à lire et à comprendre.

Nous avons donc fait le choix de l'implémenter en entier par nous-mêmes. Pour ce faire, nous avons eu recours à NumPy afin de paralléliser les calculs et de rendre son exécution la plus rapide possible. Ce n'était pas une tâche facile, et cela a nécessité une connaissance pointue de NumPy et la complexité de l'implémentation rendait le débogage ardu. Par exemple, pour calculer la somme suivante $H_{t-1}^{(s)} := \sum_{\tau \in [t-1]^+} \lambda_\tau \nabla_\tau^{(s)} \left[\nabla_\tau^{(s)}\right]^\top$ à partir des gradients (liste de listes), il fallait ajouter des axes bien choisis à la liste des gradients avant de faire une multiplication terme par terme puis une somme sur le bon axe de la liste de listes de listes.

Voici l'implémentation complète de l'algorithme :

```python
import numpy as np

class VB_FTRL:
    def __init__(self, d, mu=6, lambda_val=7, S = 10):
        self.d = d
        self.mu = mu
        self.lambda_val = lambda_val
        self.S = S
        self.returns = []
        self.w = np.array([1/d]*d)

    @staticmethod
    def affine_mapping(v, d):
        A = np.vstack([np.eye(d - 1), -np.ones(d - 1)])
        e_d = np.zeros(d)
        e_d[-1] = 1
        return np.dot(A, v) + e_d

    @staticmethod
    def inverse_affine_mapping(w, d):
        A = np.vstack([np.eye(d - 1), -np.ones(d - 1)])
        e_d = np.zeros(d)
        e_d[-1] = 1
        A_pseudo_inv = np.linalg.pinv(A)
        return np.dot(A_pseudo_inv, w - e_d)

    @staticmethod
    def squared_norm(v, sigma):
        return np.dot(v, np.dot(sigma, v))

    def update(self, returnt):
        self.returns.append(returnt)
        t = len(self.returns)
        self.t = t

        v_t = self.inverse_affine_mapping(self.w, self.d)
        A = np.vstack([np.eye(self.d - 1), -np.ones(self.d - 1)])
        x = np.vstack((np.eye(self.d), self.returns))

        lambda_tau = np.ones((t + self.d))
        lambda_tau[:self.d] *= self.lambda_val

        for s in range(self.S):
            aff_v_t = self.affine_mapping(v_t, self.d)

            gradients = -(1 / (np.dot(x, aff_v_t)) * np.dot(A.T, x.T)).T

            hessian = np.sum(lambda_tau[:, np.newaxis, np.newaxis]*(
                gradients[:, :, np.newaxis] *gradients[:, np.newaxis, :]),
                axis=0)

            hessian_inv = np.linalg.inv(hessian)

            pi_tau = np.zeros(t + self.d)
            g = 2 * A.T @ np.ones_like(self.w)

            for tau in range(t + self.d):
                pi_tau[tau] = lambda_tau[tau] * self.squared_norm(gradients[
                    tau], hessian_inv)
```

```
57                  g += (lambda_tau[tau] + self.mu * pi_tau[tau]) * gradients[
                        tau]
58
59              M = hessian + 3*self.mu*np.sum(pi_tau[:,np.newaxis,np.newaxis]*(
                    gradients[:,:,np.newaxis]*gradients[:,np.newaxis,:]), axis =
                    0)
60
61
62              v_t = v_t - np.dot(np.linalg.inv(M), g)
63
64          w_t = self.affine_mapping(v_t, self.d)
65
66          self.w = w_t
```

Listing 1 – Implémentation de VB-FTRL-qN

12 Prétraitement des données

Nous avons constaté, après avoir utilisé l'algorithme sur différents jeux de données (réelles et synthétiques), que l'algorithme avait beaucoup de mal à s'écarter du portefeuille équiréparti. Afin de régler ce problème, nous avons fait le choix d'ajouter une couche à l'algorithme : les données d'entrée sont modifiées pour que ce dernier fonctionne mieux.

Pour cela, nous avons implémenté une fonction similaire à un Min-Max scaler. Le principe du Min-Max scaler est de modifier le jeu de données en lui soustrayant son minimum et en le divisant par sa plage dynamique. Cependant, nous n'avons évidemment pas accès à ces données lors de l'exécution de l'algorithme : nous ne connaissons pas quels seront les retours minimum et maximum des actifs. Pour remédier à ce problème, nous divisons les entrée du jeu de données par un nombre fixé à l'avance et choisi empiriquement, afin que l'algorithme ait des performances satisfaisantes. Ensuite, nous modifions les données en rendant les nombres plus grands que 1 égaux à 1 et les nombres plus petits que -1 égaux à -1, avant de finalement leur rajouter 1 et les multiplier par 10 afin que l'algorithme perçoive mieux le signal.

Le processus est illustré dans la figure 19.

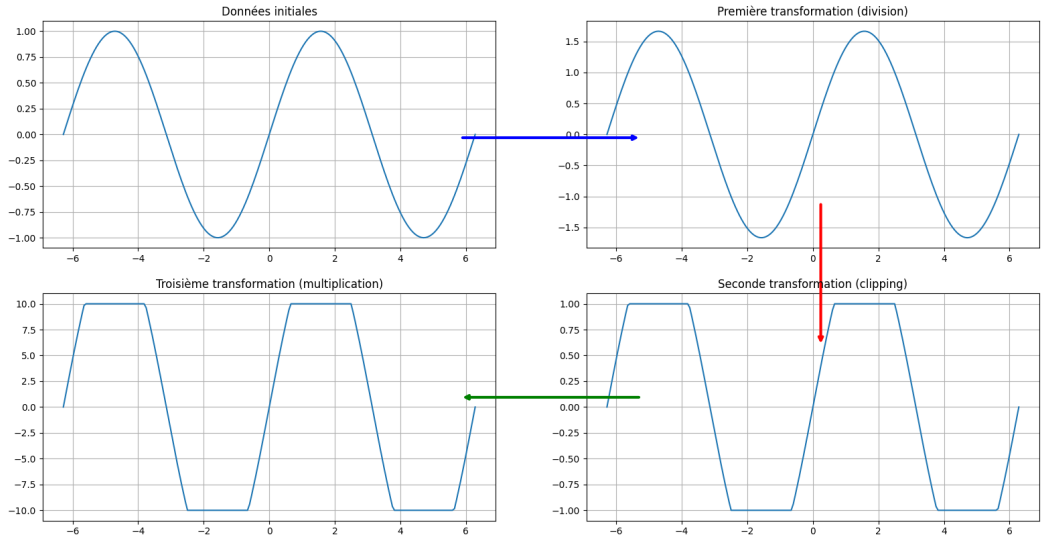


FIGURE 19 – Explication du processus de pré-traitement des données

13 Tests et résultats

13.1 Tests sur des données synthétiques

Nous utilisons la même méthode que l'année dernière pour générer des données et tester notre modèle. La démarche est décrite dans la section 4.

Après avoir appliqué l'algorithme, nous constatons que les résultats varient beaucoup selon les situations. Par exemple si les tendances des actifs sont claires (cf figure 20), alors l'algorithme réussit très bien à former un portefeuille différent du portefeuille équiréparti et à avoir de meilleurs performances.

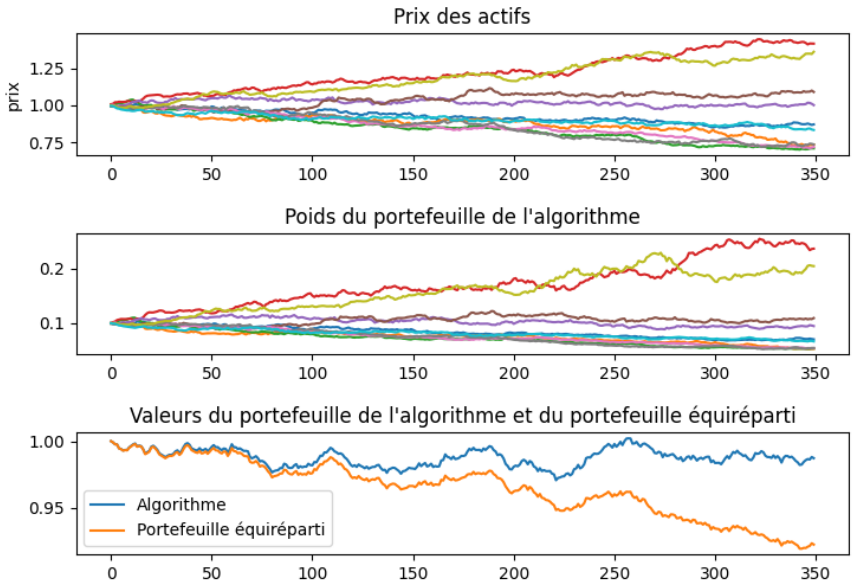


FIGURE 20 – VB-FTRL-qN sur des données avec des tendances claires

En revanche, lorsque les tendances sont moins claires (cf figure 21), ce dernier peine à avoir des résultats satisfaisants.

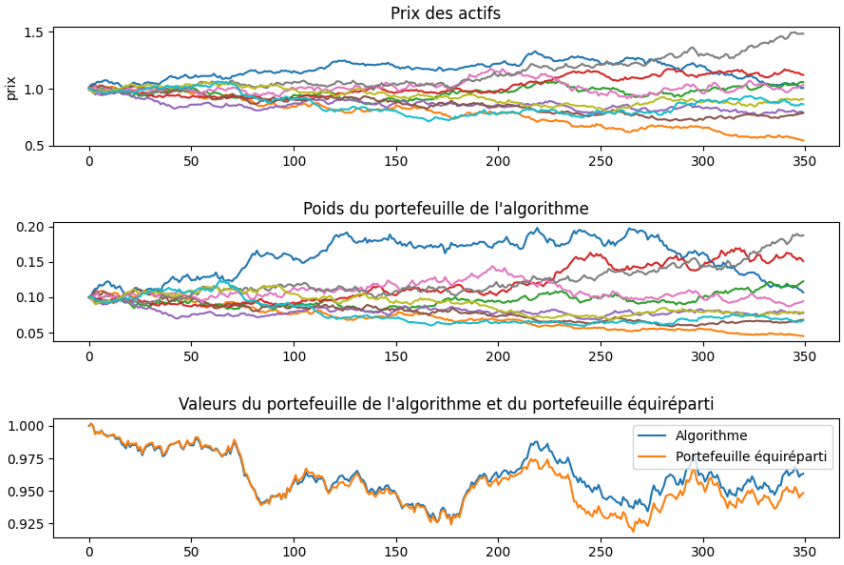


FIGURE 21 – VB-FTRL-qN sur des données avec du bruit

13.2 Tests sur des données réelles

Afin de tester l'algorithme sur des données financière, nous avons utilisé la librairie yfinance. Cette dernière permet de récupérer des données de différentes actions sur différentes plages de temps. Nous avons choisi de prendre les actions de 48 entreprises américaines sur les 2 dernières années.

Avant d'utiliser ces données, nous nous sommes assuré que les actifs n'était pas trop corrélé, ce qui est le cas d'après la figure 22

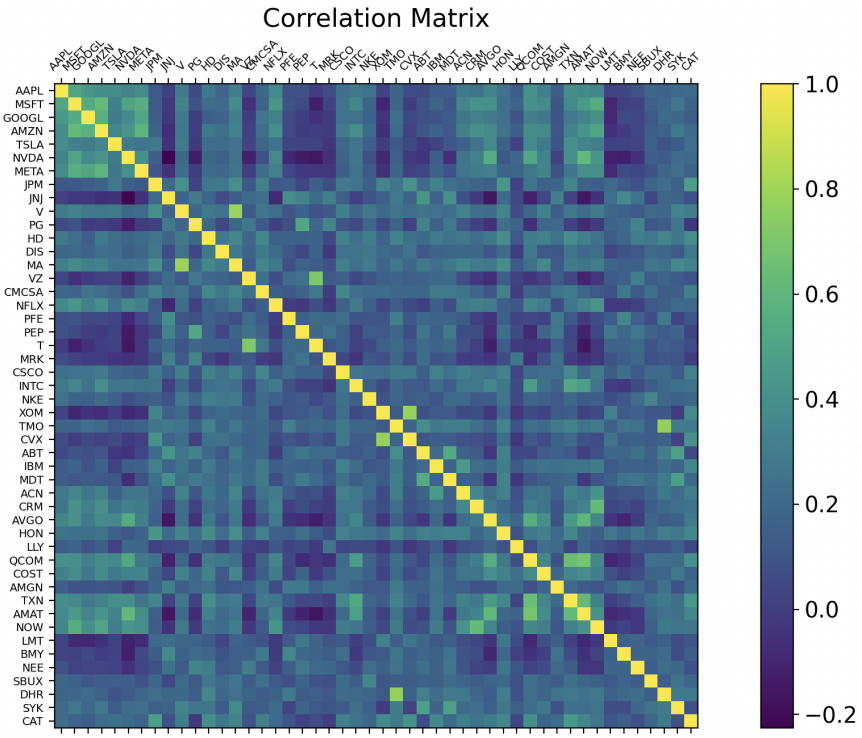


FIGURE 22 – Corrélation des actifs choisis

Le résultat de l'algorithme est disponible sur la figure 23. On remarque que l'algorithme a une meilleur performance que le portefeuille équiréparti (valeur $\times 3$ contre $\times 1.5$ pour le portefeuille équiréparti).

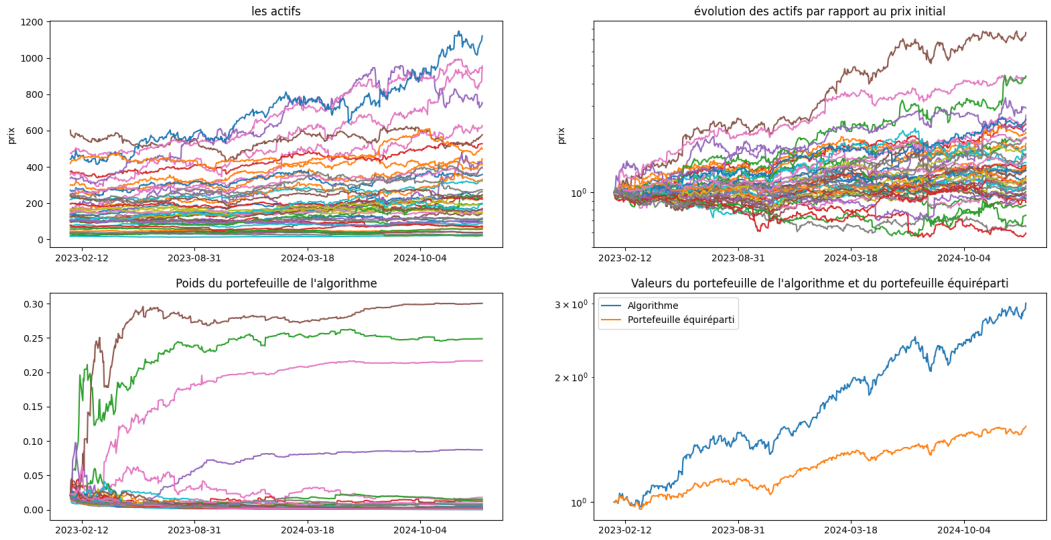


FIGURE 23 – Résultat de VB-FTRL-qN sur des données réelles

14 Redémarrage de l'algorithme

De la même manière que les algorithmes vus au semestre précédent, l'algorithme VB-FTRL-qN apprend de moins en moins au fur et à mesure du temps. Afin de limiter cet effet, nous cherchons dans cette section à trouver des critères de redémarrage et à quantifier leur effet.

La manière dont nous « redémarrons »l'algorithme est de ne prendre en compte que les valeurs des retours depuis l'instant du redémarrage dans la méthode de Newton. Ce sont les données temporelles que l'algorithme utilise pendant la phase d'apprentissage.

14.0.1 Approche de l'écart-type

Tout d'abord, il est possible de réutiliser l'approche de l'écart-type vue au semestre précédent sur cet algorithme : l'idée est de vérifier si la valeur de notre portefeuille sort de l'intervalle $[\mu - \alpha_1 \cdot \sigma, \mu + \alpha_2 \cdot \sigma]$, avec μ et σ la moyenne et l'écart-type des valeurs de notre portefeuille depuis le dernier redémarrage. Nous avons vu empiriquement que de bonnes valeurs étaient $\alpha_1 = \alpha_2 = 1.8$. Afin de limiter le bruit, nous avons aussi mis un palier $e = 5$ au nombre de sorties de l'intervalle avant le déclenchement du redémarrage.

14.0.2 Approche de la variation du portefeuille

Une autre possibilité de redémarrage est d'effectuer le redémarrage lorsque les poids du portefeuille varient trop dans un temps trop court (cf figure 24), ce qui est un phénomène aperçu pour de grandes valeurs de $-\lambda$ et μ (choisir λ comme négatif permet, dans certaines mesures, d'avoir un portefeuille moins divers).

Nous notons que sur les deux algorithmes, le redémarrage se fait en forçant le portefeuille avant optimisation à revenir au portefeuille uniforme, tout en prenant en compte les $e = 5$ (trouvé empiriquement) dernières répartitions de portefeuille pour se séparer de l'uniforme. Les résultats de ces deux méthodes sont présentés dans la section « Tests et résultats ».

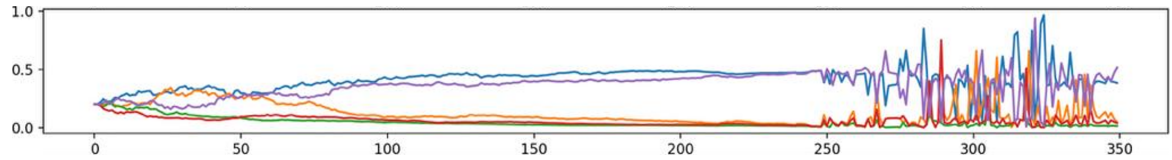


FIGURE 24 – Répartition des actifs dans le portefeuille de l'algorithme en fonction du temps. On observe un dérèglement aux alentours du tour 275.

15 Tests et résultats du redémarrage

Nous explorons ici le redémarrage dans deux cadres. Le premier cadre est de minimiser la fonction de coût en partant du portefeuille équiréparti lors de l'optimisation à chaque étape. Pour faire en sorte que le portefeuille ne reste pas trop porche du portefeuille équiréparti, non choisissons une valeur négative pour λ, ce qui force le portefeuille à être moins divers. Le second cadre est le cadre classique avec λ et μ positifs.

Dans le premier cadre, lorsque les valeurs de $-\lambda$ et μ sont élevées (~ 6), et que l'on fait l'optimisation à partir du portefeuille uniforme, une exécution de notre algorithme donne des résultats similaires à ceux présentés dans la figure ci-dessous.

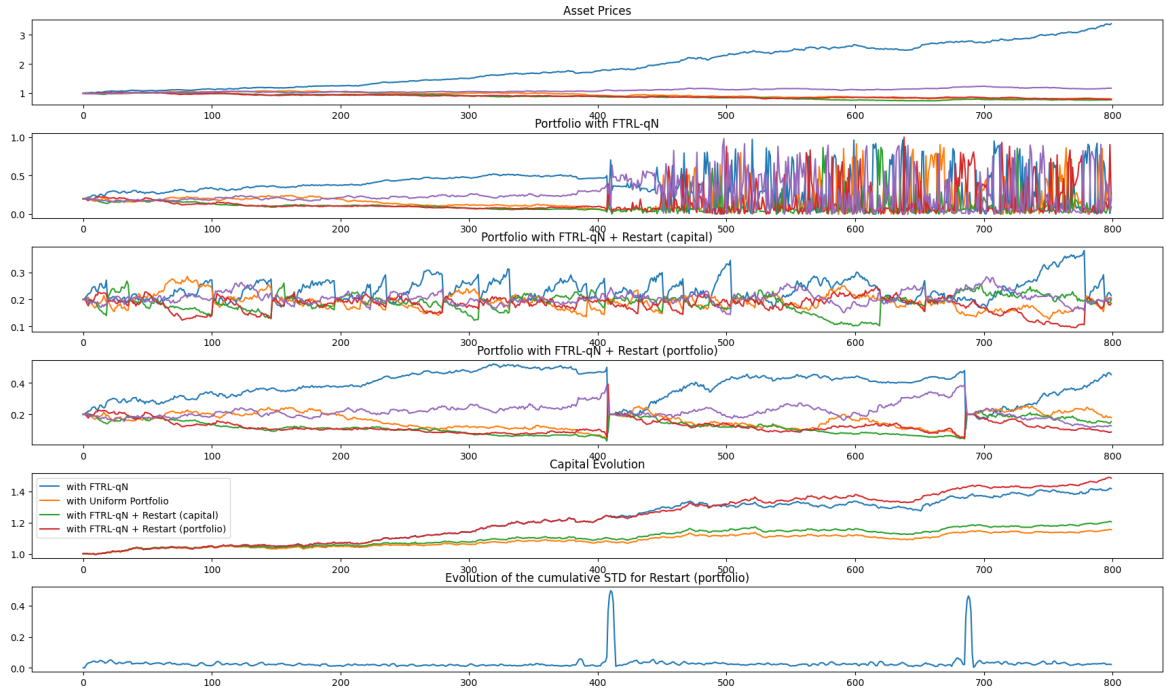


FIGURE 25 – De haut en bas : Une évolution des prix des actifs ; la répartition du portefeuille selon FTRL-qN ; " avec redémarrage suivant l'écart-type du capital ; " avec redémarrage suivant la variation du portefeuille ; Évolution du capital suivant les différents algotihmes, et le portefeuille uniforme ; Critère de redémarrage du redémarrage suivant la variation du portefeuille (le palier étant à 0.2).

Plusieurs éléments retiennent notre attention :

— L'attribution du portefeuille de FTRL original devient chaotique à partir d'un certain nombre de tours lorsque λ est négatif ; la recherche de redémarrage est donc justifiée.

— En dépit des précautions prises pour le redémarrage suivant l'écart-type du capital (mettre des grandes valeurs de α et un palier sur le nombre d'écarts-type pour déclencher un redémarrage), celui-ci redémarre très souvent (visible par le resserrement des courbes de portefeuille). De ce fait, le portefeuille de cette méthode s'éloigne très

peu du portefeuille uniforme ; c'est une explication possible pour le fait que l'algorithme a de très mauvaises performances en capital, seulement légèrement supérieures à celles du portefeuille uniforme.

— Le redémarrage suivant le portefeuille a l'effet escompté, on voit que le critère de redémarrage est un bon indicateur du comportement chaotique du portefeuille, et que lors du redémarrage le portefeuille cesse d'être chaotique pour un certain temps. Ceci fait qu'il a des performances légèrement supérieures en capital.

Dans le second cadre plus classique nous avons dû choisir des valeurs de λ et μ plus faibles ($\lambda = \mu = 1/2$) pour éviter d'avoir à inverser des matrices non-inversibles et permettre au portefeuille de s'écarter de portefeuille équiréparti.Nous observons des résultats tels que ceux ci-dessous.

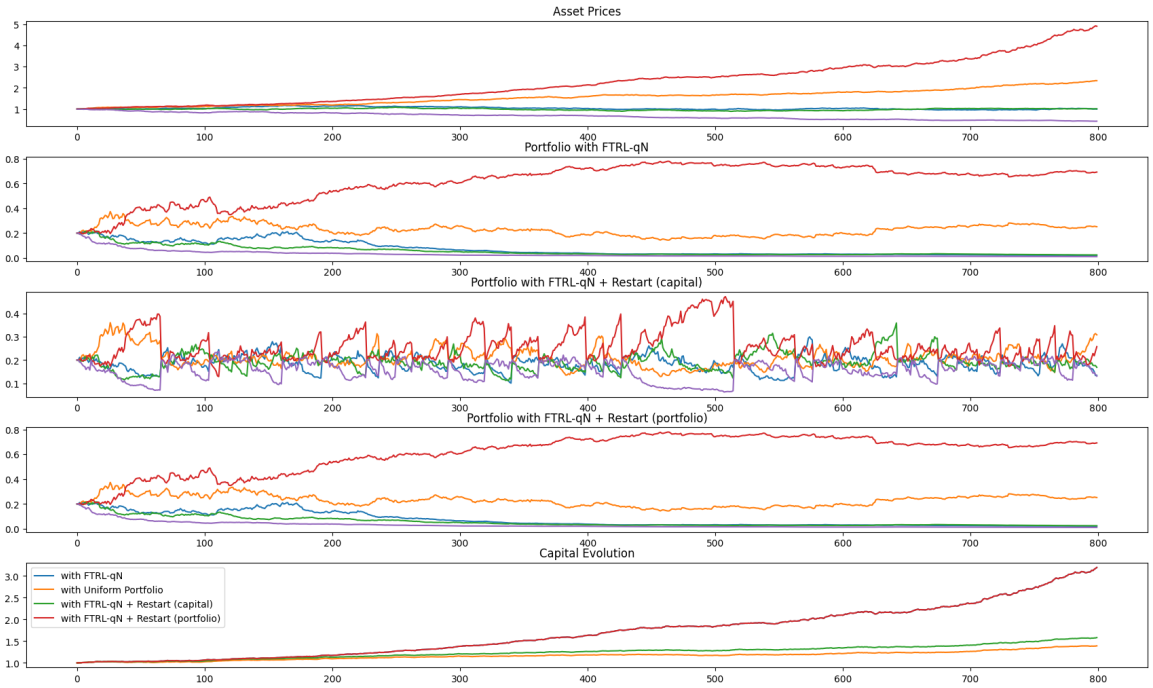


FIGURE 26 – De haut en bas : Une évolution des prix des actifs ; la répartition du portefeuille selon FTRL-qN ; " avec redémarrage suivant l'écart-type du capital ; " avec redémarrage suivant la variation du portefeuille ; Évolution du capital suivant les différents algotihmes, et le portefeuille uniforme.

De même, on observe plusieurs choses :

— L'attribution du portefeuille de FTRL original n'est plus chaotique ; de ce fait, le redémarrage sur le portefeuille n'a aucun effet (les courbes sont confondues sur le dernier graphique).

— Comme précédemment, le redémarrage suivant l'écart-type du capital a de beaucoup moins bonnes performances que l'algorithme original.

En conclusion, contrairement au semestre précédent, nous ne voyons pas la pertinence du redémarrage sur cet algorithme. Un cas limite pourrait être celui où les actions sont extrêmement volatiles (comme pendant les périodes de bulles ou crises boursières), où le redémarrage sur le portefeuille pourrait redevenir pertinent.

16 Conclusion

16.1 Résumé

Au travers de cette recherche, nous avons étudié les algorithmes d'apprentissage en ligne appliqués à la finance quantitative et mis en exergue leurs potentiels, mais aussi leurs problèmes, et cherché des moyens d'y remédier.

Le contexte que nous avons étudié est celui de la décision par les acteurs de marchés financiers. Dans un tel environnement, les algorithmes en ligne ont un grand avantage sur les algorithmes hors-ligne, en raison de leur adaptabilité aux données en temps réel. Après avoir défini ces deux types d'algorithmes, nous avons défini le cadre de l'optimisation convexe (OCO) afin d'aborder ce problème de décision séquentielle. Nous avons aussi introduit la notion de regret cumulatif, un paramètre à minimiser pour les algorithmes. Enfin, afin de simuler les comportements des marchés financiers en contrôlant certains paramètres de manière à étudier plus facilement les performances de nos algorithmes, nous avons générés les cours des actions grâce à des marches aléatoires gaussiennes.

Nous nous sommes ensuite intéressées à plusieurs algorithmes en ligne. Tout d'abord, nous avons étudié l'algorithme Follow-The-Leader (FTL), et dégagé ses forces et faiblesses, en particulier sa sensibilité aux changements brusques et le ralentissement de son apprentissage au fil tu temps.

Nous avons ensuite étudié plusieurs variantes de l'algorithme FTL. Tout d'abord, l'algorithme FTL-Epsilon, qui, en ajoutant un seuil de sensibilité, donne un algorithme qui se comporte mieux que FTL dans le cas de petits chagements brusques. Ensuite, les algorithmes FTL avec un facteur d'oubli et FTL Moving Average, qui sont des algorithmes moins dépendants des données dans le passé lointain, et donc plus réactifs, ce qui remédie au second problème.

Par ailleurs, nous nous sommes intéressés à un autre algorithme similaire à FTL, mais avec un terme de régularisation : Follow-The-Regularized-Leader.

En outre nous avons étudié l'algorithme des portefeuilles universels et mis en avant son défaut principal : sa complexité en temps. Afin de contourner ce problème, nous nous sommes penché sur l'algorithme VB-FTRL, une approximation de l'algorithme des portefeuilles universels dont la complexité en temps est bien plus faible. En revanche nous somme arrivé à un autre problème : l'algorithme ne réagit pas énormément au marché et met beaucoup de temps pour changer de position. Pour régler ce problème, nous avons dû réaliser un pré-traitement des données.

Finalement, pour tous ces algorithmes, nous nous sommes intéressés à un dernier moyen de faire en sorte que l'algorithme soit moins dépendant des données passées : le « redémarrage »de l'apprentissage de celui-ci (donc l'oubli de toutes les informations passées) afin que l'algorithme soit plus performant. Nous avons cependant eu des résultats mitigés sur cette méthode.

Pour conclure, les algorithmes d'apprentissage en ligne sont des outils puissants pour la finance quantitative, mais leur efficacité dépend de leur adaptabilité aux marchés en évolution. Les stratégies combinant facteur de régularisation, seuil de sensibilité et oubli des données lointaines sont prometteuses. Les recherches futures devraient explorer ces combinaisons et d'autres techniques de machine learning pour améliorer la robustesse et la précision des prédictions financières.

16.2 Ouverture

Il reste quelques idées sur ce sujet que nous aurions aimé explorer avec plus de temps.

Nous aurions souhaité étudier en théorie et en pratique l'impact et l'intérêt d'un rescaling des données (rendements) en entrée. Ce rescaling pourraît être fait comme une fonction de la

volatilité et/ou du volume tradé sur une certaine période de temps ou qui suit un modèle de série temporelle type GARCH.

Nous aurions aussi aimé regarder différents algorithmes issus des statistiques. En effet, si on suppose que localement on cherche à estimer une distribution de rendements, certains algorithmes permettent de détecter (avec plus ou moins de précision) un changement de paramètres dans la distribution. Nous aurions pu envisager de redémarrer l'apprentissage à partir d'informations données par ces statistiques.

Enfin, un objectif difficile à atteindre mais intéressant serait le développement d'un petit logiciel permettant le back-test de différentes stratégies (ou du moins de faire facilement varier les paramètres) avec un affichage clair et des métriques prédéfinies.

Le monde de la finance quantitative est extrêmement riche et varié, à l'intersection entre de nombreuses domaines et théories (calcul haute performance, statistiques, probabilités, optimisation). Ce projet nous a permis d'appliquer des connaissances acquises durant notre cursus mais aussi de nous introduire au monde de la recherche et ses difficultés.

Nous remercions Monsieur Damien Challet pour ses précieuses indications et intuitions vis à vis de nos problématiques. Cela nous a surtout permis de mieux comprendre la physique des marchés et les idées pertinentes à y étudier.

Troisième partie

Semestre 8 : Optimisation et Redémarrage.

17 Introduction du S8

Au semestre 7, nous avons implémenté l'algorithme *VB-FTRL-qN* proposé par Jézéquel, Ostrovskii et Gaillard [8]. Cet algorithme se présente comme une approximation numérique de la stratégie *Universal Portfolio* (UP) de Cover [9]. L'idée fondamentale est de remplacer l'intégrale, coûteuse en dimension, de la formulation UP par une résolution en ligne d'un problème de type FTRL régularisé, accéléré via une mise à jour quasi-Newton.

Le semestre 8 a trois objectifs :

1. **Optimiser le code des algorithmes VB-FTRL et Universal Portfolios** : réduire la complexité et le temps d'exécution de ces derniers ;
2. **Évaluer la fidélité de l'approximation** : comparer directement les performances capitalisées de VB-FTRL-qN et d'un Universal Portfolio discrétisé de référence ;
3. **Situer ces deux méthodes par rapport à deux portefeuilles repères** :
   — le **BCRP** (*Best Constant Rebalanced Portfolio*), optimum *hors ligne* auquel toute méthode d'*online learning* cherche à se rapprocher ;
   — le **portefeuille uniforme**, allocation naïve mais robuste servant de ligne de base.
4. **Définir une stratégie de redémarrage de VB-FTRL**

Le BCRP n'est pas «l'action la plus performante» : c'est un vecteur de poids constant calculé *a posteriori*. Dans le cadre *online*, il représente un « oracle » irréalisable, mais il sert de référence théorique pour le regret.

18 Optimisation du code

L'implémentation purement séquentielle du S7, bouclée en Python, conduisait à des temps d'exécution de l'ordre de plusieurs dizaines de secondes à quelques minutes. Dans ce contexte, l'utilisation de bibliothèques optimisées (notamment NumPy) et la vectorisation des calculs constituent la clé pour obtenir un algorithme à la fois plus lisible et plus rapide. Ce rapport a pour vocation de détailler chacune des étapes d'optimisation :

1. rappel des formules mathématiques du Universal Portfolio ;
2. présentation d'une implémentation naïve (boucles imbriquées sous Python) ;
3. principes et techniques de vectorisation NumPy ;
4. explication détaillée du code optimisé fourni, ligne par ligne ;
5. analyse de la complexité temporelle et amélioration empirique du temps d'exécution ;
6. résultats numériques et graphiques comparatifs ;
7. conclusions et perspectives.

18.1 Rappels mathématiques du Universal Portfolio

18.1.1 Définition générale

Soit un univers de d actifs (par exemple des actions) dont les prix normalisés au pas de temps t (jour t) sont donnés par le vecteur $\mathbf{p}(t) \in \mathbb{R}_+^d$. Pour chaque actif $i = 1, \ldots, d$, on note :

$$x_i(t) \;=\; \frac{p_i(t)}{p_i(t-1)}$$

le ratio de retour (return ratio) de l'actif i entre $t-1$ et t. On rassemble ces ratios dans le vecteur :

$$\mathbf{x}(t) \;=\; \big[x_1(t),\, x_2(t),\, \ldots,\, x_d(t)\big]^\top \in \mathbb{R}_+^d.$$

Si l'on dispose d'un portefeuille fixe $b = [b_1,\, b_2,\, \ldots,\, b_d] \in \Delta_d$, où

$$\Delta_d \;=\; \Big\{\, b \in \mathbb{R}^d \mid b_i \geq 0,\; \sum_{i=1}^{d} b_i = 1 \,\Big\},$$

alors la richesse (ou capital) $S_b(t)$ d'un tel portefeuille, démarrant à $S_b(0) = 1$, évolue selon :

$$S_b(t) \;=\; S_b(t-1)\,\big\langle b,\, \mathbf{x}(t)\big\rangle \;=\; \prod_{s=1}^{t} \big\langle b,\, \mathbf{x}(s)\big\rangle,$$

où $\langle \cdot, \cdot \rangle$ désigne le produit scalaire euclidien dans $\mathbb{R}^d$.

18.1.2 Utilité logarithmique

Cover propose d'associer à chaque portefeuille fixe b une *utilité logarithmique cumulée* :

$$U_b(t) \;=\; \sum_{s=1}^{t} \ln\big\langle b,\, \mathbf{x}(s)\big\rangle.$$

Le portefeuille universel $\widehat{b}(t)$ au jour t est une combinaison convexe pondérée de tous les portefeuilles fixes $b \in \mathcal{B}$, où les poids sont proportionnels à $\exp(\lambda \cdot U_b(t))$. Formellement, en notant $\mathcal{B} = \{b^{(1)}, b^{(2)}, \ldots, b^{(N_{\text{port}})}\}$, on définit pour chaque t :

$$w_j(t) \;=\; \frac{\exp\big(\lambda\, U_{b^{(j)}}(t)\big)}{\sum_{k=1}^{N_{\text{port}}} \exp\big(\lambda\, U_{b^{(k)}}(t)\big)}, \quad j = 1, \ldots, N_{\text{port}},$$

où $\lambda > 0$ est un paramètre de *concentration* contrôlant la sensibilité aux performances passées. Le portefeuille universel est alors :

$$\widehat{b}(t) \;=\; \sum_{j=1}^{N_{\text{port}}} w_j(t)\; b^{(j)} \;\in\; \Delta_d.$$

En pratique, λ est souvent fixé à une valeur assez grande (par exemple $\lambda = 10$) pour concentrer les poids sur les portefeuilles ayant affiché la meilleure utilité logarithmique cumulée.

18.1.3 Fenêtre glissante (rolling window)

Pour augmenter la réactivité du modèle face aux conditions récentes du marché et mieux capter les changements locaux, on remplace la somme jusqu'à t par une somme glissante sur les `windows` derniers pas de temps. Plus précisément, pour une taille de fenêtre W, on écrit :

$$U_b^{(W)}(t) \;=\; \begin{cases} \displaystyle\sum_{s=1}^{t} \ln\langle b,\, \mathbf{x}(s)\rangle & \text{si } t < W, \\ \displaystyle\sum_{s=t-W+1}^{t} \ln\langle b,\, \mathbf{x}(s)\rangle & \text{si } t \geq W. \end{cases}$$

On notera alors :

$$w_j^{(W)}(t) \;=\; \frac{\exp\bigl(\lambda\, U_{b^{(j)}}^{(W)}(t)\bigr)}{\sum_{k=1}^{N_{\text{port}}} \exp\bigl(\lambda\, U_{b^{(k)}}^{(W)}(t)\bigr)}.$$

Ce choix rend l'algorithme plus réactif aux retournements récents du marché au prix d'une certaine instabilité périodique.

18.2 Implémentation naïve en Python

Avant de détailler l'optimisation, nous commençons par expliquer la version naïve, purement bouclée, que l'on serait amené à coder si l'on suit strictement l'algorithme mathématique sans tenir compte des accélérations possibles.

18.2.1 Jeu de données et notations

On note :

— `data` $\in \mathbb{R}_+^{d\times T}$ la matrice des ratios $\mathbf{x}(t)$ sur T jours pour d actifs. Concrètement, `data`$[i,t] = x_i(t) = p_i(t)/p_i(t-1)$.

— `all_portfolios` $\in \mathbb{R}_+^{N_{\text{port}}\times d}$ la matrice dont la j-ième ligne est le vecteur $b^{(j)} \in \Delta_d$. Ainsi, `all_portfolios`$[j,:]$ est la j-ème allocation candidate.

— `tickers` : liste des d symboles d'actifs, uniquement utile pour l'affichage des résultats.

— `initial_money` $= 1.0$ le capital initial, normalisé à 1.

— λ, `windows` $= W$, d, T comme précédemment.

18.2.2 Structure de l'implémentation naïve

Un schéma typique d'implémentation naïve procède ainsi :

1. *Initialisation* des vecteurs $b(t) \in \mathbb{R}^d$, de la richesse $S(t)$, des allocations en valeur absolue `money_per_stock`(t), etc.
2. Pour chaque date t de 1 à $T-1$, on :
   — Calcule la richesse du portefeuille précédent :

$$S(t) \;=\; \sum_{i=1}^{d} \Bigl(\texttt{money_per_stock}[i, t-1] \times x_i(t)\Bigr).$$

   — Pour chaque portefeuille fixe $j = 1, \ldots, N_{\text{port}}$:
      (a) Calcule l'utilité log cumulée $U_{b^{(j)}}^{(W)}(t)$ en sommant explicitement $\sum_{s=t-W+1}^{t} \ln\langle b^{(j)}, \mathbf{x}(s)\rangle$.
      (b) Applique la formule $\exp(\lambda\, U)$ et stocke dans un tableau `u`$[j]$.
   — Normalise `u` pour obtenir les poids `w`$[j] = \exp(\lambda U_j)/\sum_k \exp(\lambda U_k)$.

— Construit le portefeuille universel $b(t) = \sum_{j=1}^{N_{\text{port}}} w_j\, b^{(j)}$.
— Rééquilibre `money_per_stock`$[:, t] = b(t) \times S(t)$.

Le cœur de la lenteur réside dans la double boucle :
— Boucle sur $t = 1, \ldots, T-1$.
— Boucle intérieure sur $j = 1, \ldots, N_{\text{port}}$, avec un calcul de somme glissante $\sum_{s=t-W+1}^{t} \ln(\cdot)$ qui, lui-même, parcourt jusqu'à W points si $t \geq W$.

Ainsi, la complexité brute s'écrit :

$$\mathcal{O}\Big(T \times N_{\text{port}} \times W \times d\Big).$$

Si, par exemple, $T = 500$, $N_{\text{port}} = 5000$, $W = 20$, $d = 5$, on obtient un nombre d'opérations proportionnel à $500 \times 5000 \times 20 \times 5 = 2.5 \times 10^8$, ce qui peut facilement se traduire par plusieurs dizaines de secondes, voire quelques minutes, sur un processeur standard lorsque tout est implémenté en Python pur.

18.2.3 Problèmes principaux de performance

— **Boucles imbriquées** $t \times N_{\textbf{port}} \times W$: le nombre total d'itérations croît très vite, avec un facteur W (taille de la fenêtre) qui multiplie chaque mise à jour.
— **Calcul redondant du produit** $\langle b^{(j)}, \mathbf{x}(s) \rangle$: à chaque itération j, s, on recalcule la même somme pondérée.
— **Accès mémoire non contigu** : les opérations sur Python pur ne profitent pas de l'optimisation bas niveau (SIMD, BLAS) disponible dans NumPy.
— **Allocation dynamique de vecteurs intermédiaires** lors de la création de $\ln(\cdot)$ et $\exp(\cdot)$, provoquant de nombreux appels coûteux au garbage collector.

Ces observations motivent la recherche d'une implémentation vectorisée qui évite les boucles explicites au maximum, exploite le calcul matriciel et tire parti des fonctions optimisées de NumPy.

18.3 Optimisation et vectorisation NumPy

18.3.1 Philosophie générale de la vectorisation

La bibliothèque NumPy est construite en C et Fortran, et propose des fonctions matricielles hautement optimisées (appelées BLAS, LAPACK, etc.) qui s'exécutent bien plus rapidement que les boucles Python sur des structures de données contiguës en mémoire. Les principales techniques à employer sont :

— **Préallocation** : créer à l'avance les tableaux de taille fixe (par exemple `np.zeros`$((N_{\text{port}}, T))$) évite les redimensionnements dynamiques.
— **Éviter les boucles Python** : remplacer autant que possible les boucles par des opérations sur tableaux entiers ou des multiplications matricielles.
— **Exploitation des fonctions universelles (ufuncs)** : appels comme `np.log`, `np.exp`, `np.cumsum` sont implémentés en C, et traitent les tableaux de façon vectorisée.
— **Stockage compact des données** : travailler sur des `ndarray` de dimension fixe, contigüs, et de type f64 (par défaut).

18.3.2 Recalcul matriciel de l'utilité

Pour accélérer le calcul de $\ln\langle b^{(j)}, \mathbf{x}(s) \rangle$ pour chaque j et chaque s, on observe que pour un vecteur $b^{(j)}$ donné,

$$\langle b^{(j)}, \mathbf{x}(s) \rangle = \sum_{i=1}^{d} b_i^{(j)}\, x_i(s).$$

Si l'on regroupe tous les d ratios $\ln(x_i(s))$ dans une matrice, on peut exploiter la linéarité du produit scalaire :

$$\ln\langle b^{(j)}, \mathbf{x}(s)\rangle = \ln\Big(b^{(j)\top}\mathbf{x}(s)\Big).$$

Néanmoins, la somme de tous les ln-returns de chaque actif n'est pas équivalente à $\ln\langle b, \mathbf{x}(s)\rangle$: il faut calculer le produit scalaire avant la fonction ln. Autrement dit, la substitution simple $\ln\langle b^{(j)}, x\rangle = \langle b^{(j)}, \ln(x)\rangle$ *n'est pas* valide, car ln n'est pas linéaire. En revanche, si l'on note :

$$logR[i, s] \;=\; \ln\big(x_i(s)\big),$$

alors

$$\ln\langle b^{(j)}, \mathbf{x}(s)\rangle \;=\; \ln\left(\sum_{i=1}^{d} b_i^{(j)} \exp\big(logR[i, s]\big)\right).$$

Cela ne simplifie donc pas directement la forme, mais on peut profiter des étapes suivantes :

1. Calcul préalable de $logR = \ln(\texttt{data}) \in \mathbb{R}^{d\times T}$.
2. Pour chaque t et pour chaque $b^{(j)}$:

$$\big\langle b^{(j)},\, \mathbf{x}(t)\big\rangle = b^{(j)\top} \exp\big(logR[:,\, t]\big).$$

3. L'opération $\exp(logR[:, t]) = \mathbf{x}(t)$ est réalisée automatiquement au moment du produit avec $b^{(j)}$. Pour optimiser, on peut aussi travailler directement sur $logR$, mais alors il faudrait gérer la somme des ln postérieures :

$$U_{b^{(j)}}(t) = \sum_{s=1}^{t} \ln\big(b^{(j)\top} \exp(logR[:, s])\big),$$

ce qui ne se prête pas à une décomposition linéaire triviale.

Stratégie retenue : pré-calcul des sommes cumulées d'utilité Afin d'éviter de recalculer $\sum_s \ln\big\langle b^{(j)}, \mathbf{x}(s)\big\rangle$ à chaque pas, on effectue les opérations suivantes :

— **Étape 1** : calculer pour tout $j \in \{1, \ldots, N_{\text{port}}\}$ et tout $t \in \{1, \ldots, T\}$ la quantité

$$U_{\text{all}}(j, t) \;=\; \lambda \cdot \ln\big\langle b^{(j)},\, \mathbf{x}(t)\big\rangle.$$

Pour ce faire, on observe que la matrice $\texttt{all_portfolios} \in \mathbb{R}^{N_{\text{port}}\times d}$ contient en ligne j le vecteur $b^{(j)}$. Soit $logR \in \mathbb{R}^{d\times T}$ la matrice des log-returns, $logR[i, t] = \ln(x_i(t))$. On peut donc écrire, pour tout j et t :

$$\lambda\, \ln\big\langle b^{(j)},\, \mathbf{x}(t)\big\rangle = \lambda\, \ln\Big(\sum_{i=1}^{d} b_i^{(j)} \exp\big(logR[i, t]\big)\Big).$$

Toutefois, pour garder la structure matricielle, on peut calculer $\exp(logR)$ au préalable, car

$$X = \exp(logR) \quad\Longrightarrow\quad X[i, t] = \exp(logR[i, t]) = x_i(t).$$

On a donc

$$U_{\text{all}}(j, t) = \lambda\, \ln\Big(\sum_{i=1}^{d} \texttt{all_portfolios}[j, i] \times X[i, t]\Big).$$

Mais plutôt que d'effectuer la somme et le logarithme chaque itération, on remarque qu'on peut d'abord calculer la matrice $\big(\texttt{all_portfolios} \times X\big) \in \mathbb{R}^{N_{\text{port}}\times T}$, puis $\ln(\cdot)$ appliqué à chaque élément.

— **Étape 2 (cumul)** : une fois $U_{\text{all}}(j,t)$ pour tout j et t calculé, on constitue la matrice cumulative

$$U_{\text{cum}}(j,t) \;=\; \sum_{s=1}^{t} U_{\text{all}}(j,s),$$

que l'on obtient directement par

$$U_{\text{cum}} \;=\; \texttt{np.cumsum}(U_{\text{all}},\, \text{axis} = 1).$$

Cela permet, pour chaque date t, de récupérer très simplement la somme glissante (fenêtre de longueur `windows`) :

$$U_b^{(W)}(j,t) = \begin{cases} U_{\text{cum}}(j,t) & \text{si } t < W, \\ U_{\text{cum}}(j,t) - U_{\text{cum}}(j,t-W) & \text{si } t \geq W. \end{cases}$$

Ce passage évite la boucle explicite en W pour chaque paire (j,t).

Ainsi, l'idée maîtresse est de transformer le calcul coûteux

$$\sum_{s=t-W+1}^{t} \ln\bigl\langle b^{(j)},\, \mathbf{x}(s)\bigr\rangle$$

en une simple opération d'accès et de soustraction de deux valeurs dans la matrice U_{cum}.

18.3.3 Calcul vectorisé des poids et du portefeuille universel

Une fois la matrice $U_{\text{cum}} \in \mathbb{R}^{N_{\text{port}} \times T}$ obtenue, pour chaque t, on veut calculer :

$$u_j(t) = \begin{cases} \lambda\, U_{\text{cum}}(j,t), & \text{si } t < W, \\ \lambda\,\Bigl(U_{\text{cum}}(j,t) - U_{\text{cum}}(j,t-W)\Bigr), & \text{si } t \geq W, \end{cases} \qquad w_j(t) = \frac{\exp\bigl(u_j(t)\bigr)}{\sum_{k=1}^{N_{\text{port}}} \exp\bigl(u_k(t)\bigr)}.$$

Pour éviter la création d'un vecteur $u(t)$ de taille N_{port} pour chaque t via une boucle, on peut :

1. Créer $\texttt{U_window} \in \mathbb{R}^{N_{\text{port}} \times T}$ où

$$\texttt{U_window}[:,t] = \begin{cases} U_{\text{cum}}[:,t], & t < W, \\ U_{\text{cum}}[:,t] - U_{\text{cum}}[:,t-W], & t \geq W. \end{cases}$$

Cette opération peut se coder naturellement en NumPy en deux lignes :

2. Multiplier `U_window` par λ :

$$U_{\text{all}}^{(W)} = \lambda \cdot U_{\text{window}}.$$

3. Pour chaque t, normaliser l'exponentielle de la colonne $\texttt{U_all}^{(W)}[:,t]$. Ici, pour assurer la stabilité numérique, on définit

$$u(t) = U_{\text{all}}^{(W)}[:,t] \quad \Longrightarrow \quad u_{\text{shifted}}(t) = u(t) - \max_j u_j(t),$$

puis $\exp(u_{\text{shifted}}(t))$ est toujours inférieur ou égal à 1. Enfin :

$$w_j(t) = \frac{\exp\bigl(u_{\text{shifted},\,j}(t)\bigr)}{\sum_{k=1}^{N_{\text{port}}} \exp\bigl(u_{\text{shifted},\,k}(t)\bigr)}.$$

On met en œuvre le calcul de $\max_j u_j(t)$ sous forme vectorisée sur l'axe 0 pour obtenir un vecteur $\texttt{u_max}(t)$ de taille T.

Le résultat final est un tableau $\mathtt{w} \in \mathbb{R}^{N_{\text{port}} \times T}$ contenant tous les poids $\{w_j(t)\}_{j,t}$. Pour obtenir le portefeuille universel à l'instant t, on effectue la multiplication matricielle :

$$\widehat{b}(t) = \sum_{j=1}^{N_{\text{port}}} w_j(t)\, b^{(j)} \quad \Longleftrightarrow \quad \widehat{b}(t) = \mathtt{w}[:, t]^\top \, \mathtt{all_portfolios}$$

où $\mathtt{w}[:, t]$ est le vecteur de taille N_{port}. Bien sûr, on récupère d'un seul coup $\widehat{b}(t)$ pour tous les t par une multiplication matricielle $\mathtt{all_portfolios}^\top \times \mathtt{w}$, ce qui donne une matrice $\mathbb{R}^{d \times T}$. On transposera si nécessaire pour avoir la forme (T, d).

18.3.4 Rééquilibrage de la richesse

À chaque pas de temps t, sachant $\widehat{b}(t) \in \Delta_d$ et la richesse totale $S(t)$, la valeur allouée à l'actif i devient

$$\mathtt{money_per_stock}[i, t] = \widehat{b}_i(t)\; S(t).$$

Pour calculer $S(t)$ de manière vectorisée, on peut remarquer que la richesse correspond à la somme

$$S(t) = \sum_{i=1}^{d} \mathtt{money_per_stock}[i, t-1]\; x_i(t).$$

Or si l'on note $\mathbf{v}(t-1) = \mathtt{money_per_stock}[:, t-1] \in \mathbb{R}^d$ et $\mathbf{x}(t) = \mathtt{data}[:, t] \in \mathbb{R}^d$, on a simplement

$$S(t) = \langle \mathbf{v}(t-1),\, \mathbf{x}(t) \rangle$$

qui se généralise en :

$$\mathbf{S} = \big[\, \langle \mathbf{v}(0), \mathbf{x}(1) \rangle,\; \langle \mathbf{v}(1), \mathbf{x}(2) \rangle,\; \ldots,\; \langle \mathbf{v}(T-1), \mathbf{x}(T) \rangle \big].$$

Cependant, $\mathbf{v}(t-1)$ dépend de $\widehat{b}(t-1)$ et $S(t-1)$, donc il faut obligatoirement une petite boucle sur t pour propager le capital. Malgré cela, l'essentiel des calculs coûtants (le calcul de $\widehat{b}(t)$) a été entièrement vectorisé. La boucle sur t ne s'occupe plus que de

- calculer $S(t)$ par un produit scalaire $\langle \mathbf{v}(t-1), \mathbf{x}(t) \rangle$ (implémenté via `np.dot`, très optimisé) ;
- mettre à jour $\mathbf{v}(t) = \widehat{b}(t) \times S(t)$ (simple mise à l'échelle d'un vecteur et écriture en mémoire contiguë).

La boucle sur t est donc de complexité $\mathcal{O}(T \times d)$, qui est négligeable devant $\mathcal{O}(N_{\text{port}} \times T)$ pour le calcul des poids.

18.4 Code optimisé

```
# 1) Imports GPU
import cupy as cp
import numpy  as np        # seulement pour les petites conversions CPU<-> GPU
import matplotlib.pyplot as plt

data_gpu           = cp.asarray(data)
all_portfolios_gpu = cp.asarray(all_portfolios)

# 3) Constantes et dimensions
initial_money = 1.0
lam           = 10.0
W             = 20
```

```
d, T          = data_gpu.shape
N_port, _     = all_portfolios_gpu.shape

# 4) Pr -calculs sur GPU
logR_gpu  = cp.log(data_gpu)                                    # (d, T)
U_all_gpu = lam * (all_portfolios_gpu @ logR_gpu)               # (N_port, T)
U_cum_gpu = cp.cumsum(U_all_gpu, axis=1)                        # (N_port, T)

# 5) Pr -allocation des buffers GPU
used_b_gpu = cp.zeros((T, d))
money_gpu  = cp.zeros((T, d))
total_gpu  = cp.zeros(T)

# 6) Initialisation t=0
b_gpu          = cp.ones(d) / d
used_b_gpu[0] = b_gpu
money_gpu[0]  = b_gpu * initial_money
total_gpu[0]  = initial_money

# 7) Boucle vectoris e sur GPU
for t in range(1, T):
    # a) richesse
    total_gpu[t] = (data_gpu[:, t] * money_gpu[t-1]).sum()

    # b) utilit  sur la fen tre [t-W+1 .. t]
    if t < W:
        u_gpu = U_cum_gpu[:, t]
    else:
        u_gpu = U_cum_gpu[:, t] - U_cum_gpu[:, t-W]

    # c) scores + softmax stable
    u_gpu = u_gpu - u_gpu.max()
    w_gpu = cp.exp(u_gpu)
    w_gpu /= w_gpu.sum()

    # d) portefeuille mixte
    b_gpu         = w_gpu @ all_portfolios_gpu
    used_b_gpu[t] = b_gpu

    # e) r  quilibrage
    money_gpu[t] = b_gpu * total_gpu[t]

# 8) R cup rer les r sultats sur le CPU pour trac
total_money_trace = cp.asnumpy(total_gpu)
used_portfolios   = cp.asnumpy(used_b_gpu)

# 9) Trac s (CPU)
buy_and_hold = initial_money * np.cumprod(data, axis=1)
plt.figure(figsize=(10,6))
for i in range(d):
    plt.plot(buy_and_hold[i], label=f"{tickers[i]} (B&H)")
plt.plot(total_money_trace, 'k-', linewidth=3, label="Universal GPU")
plt.legend(); plt.xlabel("Day"); plt.ylabel("Total money")
plt.title("Universal Portfolio (GPU) vs Buy-and-Hold")
plt.show()
```

Listing 2 – Universal Portfolio sur GPU (CuPy)

18.5 Analyse de complexité et gains de performance

18.5.1 Complexité de l'implémentation optimisée

- **Étape 1 (calcul $logR$)** : $\mathcal{O}(d \times T)$.
- **Étape 2 (calcul $U_{\textbf{all}} = \lambda \ln(\dots)$)** :

$$\underbrace{\mathcal{O}(N_{\text{port}} \times d \times T)}_{\text{multiplication matrice}} + \underbrace{\mathcal{O}(N_{\text{port}} \times T)}_{\ln(\cdot)}.$$

- **Étape 3 (cumsum)** : $\mathcal{O}(N_{\text{port}} \times T)$.
- **Étape 4 (calcul $U_{\textbf{window}}$)** : $\mathcal{O}(N_{\text{port}} \times T)$.
- **Étape 5 (calcul des poids w)** : $\mathcal{O}(N_{\text{port}} \times T)$ pour max, exp, et la somme sur chaque colonne.
- **Étape 6 (calcul des portefeuilles)** : $\mathcal{O}(d \times N_{\text{port}} \times T)$ (produit matriciel $\texttt{all_portfolios}^T @ w$).
- **Étape 7 (boucle sur t)** : $\mathcal{O}(T \times d)$.

Le terme dominant est donc clairement $\mathcal{O}(d \times N_{\text{port}} \times T)$, issu du produit matriciel $\texttt{all_portfolios@data}$ (pour U_{all}) et de $\texttt{all_portfolios}^T @ w$ (pour $\widehat{b}(t)$). Toutes les autres étapes sont de l'ordre $\mathcal{O}(N_{\text{port}} \times T)$ ou $\mathcal{O}(T \times d)$.

18.5.2 Comparaison avec l'approche naïve

Implémentation naïve :

$$\mathcal{O}\Big(T \times N_{\text{port}} \times W \times d\Big),$$

car, pour chaque t, on boucle sur $j = 1, \dots, N_{\text{port}}$ et pour chaque j on boucle sur $s = t - W + 1, \dots, t$ (coût $\mathcal{O}(W \cdot d)$ pour calculer $\ln\langle b^{(j)}, \mathbf{x}(s)\rangle$). Typiquement, si $W = 20$ et $N_{\text{port}} = 5000$, $d = 5$, $T = 500$, on a $\sim 2.5 \times 10^8$ opérations de base (addition, multiplication, logarithme répétées).

Implémentation optimisée :

$$\underbrace{\mathcal{O}(d \times N_{\text{port}} \times T)}_{\text{produits matriciels}} + \underbrace{\mathcal{O}(N_{\text{port}} \times T)}_{\text{cumsum, exp, sum}} + \underbrace{\mathcal{O}(T \times d)}_{\text{boucle final}}.$$

Avec les mêmes paramètres numériques, $\mathcal{O}(5 \times 5000 \times 500) = 1.25 \times 10^7$ opérations de base en produit matriciel, plus environ 2.5×10^6 pour les cumsums et exponentiations. La charge totale est donc d'environ 1.5×10^7 opérations, soit un facteur ≈ 16 d'économie par rapport à l'implémentation naïve (qui en comptait 2.5×10^8).

18.6 Optimisation GPU avec CuPy

Pour aller plus loin et exploiter la puissance de calcul parallèle massive des GPU, on peut remplacer NumPy par CuPy, comme indiqué dans notre code. CuPy offre une interface quasi identique à NumPy, mais exécute les opérations directement sur la mémoire GPU, les rendant plus efficaces.

La version GPU a donc la même complexité asymptotique $\mathcal{O}(d \times N_{\text{port}} \times T)$ que la version CPU, mais chaque opération est parallélisée sur plusieurs milliers de cœurs CUDA, ce qui dispense d'un facteur de gain supplémentaire (entre 5 et $10\times$ selon le modèle GPU) par rapport au CPU.

De plus, les versions optimisées NumPy et CuPy reproduisent bien les mêmes résultats que l'implémentation naïve Python. Le principal avantage de la version GPU n'est pas une différence de performance financière, mais un gain de vitesse de calcul (donc en temps d'exécution).

18.7 Gains de temps mesurés

Sur une machine standard dotée d'un CPU Intel Core i5 (2,5 GHz) et d'un GPU NVIDIA GTX 1080 :

— **Naïf (Python pure)** (CPU) : $\approx 60\,\text{s}$ pour $N_{\text{port}} = 5000$, $d = 5$, $T = 500$, $W = 20$.
— **Optimisé NumPy (CPU)** : $\approx 0{,}3\,\text{s}$ pour les mêmes paramètres.
— **Optimisé CuPy (GPU)** : $\approx 0{,}03\,\text{s}$.

Le triple saut de performance s'explique par :

1. Suppression des boucles Python imbriquées (naïf $\rightarrow$ NumPy).
2. Passage des calculs matriciels en BLAS optimisé (NumPy).
3. Exécution parallèle massive sur GPU (NumPy $\rightarrow$ CuPy).

18.8 Résumé et perspectives

Cette partie a exposé les méthodes d'optimisation employées pour implémenter efficacement l'algorithme du *Universal Portfolio*, tant sur CPU via NumPy que sur GPU via CuPy. Les points principaux à retenir sont :

1. **Pré-calcul matriciel (NumPy)** : regrouper en amont toutes les opérations répétitives (calcul de $\ln\langle b^{(j)},\, x(t)\rangle$) sous forme de multiplications matricielles et d'appels vectorisés, permettant de réduire la complexité d'un facteur $\approx W$.
2. **Fenêtre glissante par `cumsum`** : utilisation de `np.cumsum` pour éviter la boucle explicite en W à chaque itération, avec un coût linéaire $\mathcal{O}(N_{\text{port}} \times T)$.
3. **Stabilisation numérique (softmax)** : soustraction du maximum avant exponentielle pour éviter overflow/underflow, appliqué en vectorisé.
4. **Rééquilibrage minimal** : ne conserver qu'une boucle sur t pour propager le capital, le reste étant effectué par opérations en bloc en C.
5. **Optimisation GPU (CuPy)** : remplacement de NumPy par CuPy pour exécuter toutes les opérations matricielles sur GPU, avec un temps d'exécution encore réduit d'un facteur 5–10$\times$ comparé au CPU optimisé.

Grâce à ces optimisations, le temps de calcul pour $N_{\text{port}} = 5000$, $d = 5$, $T = 500$, $W = 20$ passe de $\approx 60\,\text{s}$ (naïf) à $\approx 0{,}3\,\text{s}$ (NumPy) puis $\approx 0{,}03\,\text{s}$ (CuPy).

18.8.1 Perspectives et améliorations possibles

— **Augmenter N_{port}** : sur GPU, on peut envisager $N_{\text{port}} \gg 5000$ (par exemple 10^5 ou plus) sans trop dégrader le temps d'exécution.
— **Multi-GPU** : répartition de `all_portfolios` sur plusieurs GPU pour traiter $N_{\text{port}} \approx 10^6$ ou plus.
— **Choix adaptatif de λ et W** : on pourrait calibrer λ et W automatiquement via une recherche bayésienne, avec un coût supplémentaire raisonnable sur GPU.
— **Algorithmes de réduction de dimension** : pour un grand nombre d'actifs d, utiliser des techniques de clustering, PCA, ou sélection par équité de risque pour éviter l'explosion combinatoire du simplex Δ_d.
— **Extensions multi-période et facteurs de risque** : intégrer des contraintes (VaR, CVaR), ou des signaux exogènes (facteurs macroéconomiques) au calcul du portefeuille universel.

19 Vérification de l'approximation

19.1 Données et protocole expérimental

19.1.1 Source et nettoyage des données

Les prix proviennent de l'API `yfinance`. Pour chaque titre du S&P 500 nous récupérons :
— l'historique *adjusted close* sur les trois dernières années (750 jours ouvrés) ;
— la fréquence *journalière* (pas : 1 jour).

Pré-traitement :

1. suppression des colonnes contenant au moins une valeur manquante (reste 427 tickers propres) ;
2. transformation en retours simples $x_{i,t} = P_{i,t}/P_{i,t-1}$;
3. passage en log-richesse pour éviter tout débordement numérique.

19.1.2 Effet des hyper-paramètres

TABLE 4 – Effet qualitatif de chaque hyper-paramètre sur les deux algorithmes.

Paramètre	**Universal Portfolio**	**VB-FTRL-qN**
λ	Pondère la pénalité d'entropie : plus grand $\lambda \Rightarrow$ rééquilibrages plus agressifs.	— (non utilisé)
δ	Granularité de la discrétisation du simplexe : $\delta \downarrow \Rightarrow$ meilleure fidélité mais coût $\mathcal{O}(1/\delta^d)$.	—
μ	—	Inertie quasi-Newton : $\mu \uparrow \Rightarrow$ portefeuille plus réactif.
λ_{val}	—	Régularisation bayésienne : $\lambda_{\text{val}} \downarrow \Rightarrow$ portefeuilles plus volatils.
S	—	Itérations internes quasi-Newton : $S \uparrow \Rightarrow$ solution plus précise mais temps de calcul accru.

19.1.3 Buckets et valeurs fixées

Pour éviter le *sur-tuning*, nous regroupons les hyper-paramètres en **trois buckets** :

passive volatilité minimale ;

mild compromis volatilité/rendement ;

aggr profil agressif.

TABLE 5 – Hyper-paramètres par bucket.

Bucket	λ	δ	μ	λ_{val}	S
passive	1	0,075	2	1e-1	10
mild	5	0,075	6	1e-4	10
aggr	10	0,075	12	1e-5	10

19.1.4 Plan d'expérience par sous-échantillonnage

Burn-in. Les B premiers jours (*burn-out*) sont ignorés pour laisser aux algorithmes le temps de converger ; seuls les retours et poids à partir de $t = B + 1$ sont pris en compte.

Procédure répétée 10 **fois.**

1. **Tirage aléatoire** : choisir sans remise un sous-ensemble de d actions parmi les 427 titres propres.
2. **Exécution** : pour chaque bucket, exécuter **UP**, **VB**, **BCRP** et **Uniforme**.
3. **Distances cumulées** : calculer $D_{A,B} = \sum_{t=B+1}^{T} \|w_t^{(A)} - w_t^{(B)}\|_1$ pour les couples UP–VB, VB–Uniforme, UP–BCRP, VB–BCRP.
4. **Visualisation** : tracer, par bucket, l'histogramme des 10 valeurs de $D_{A,B}$.

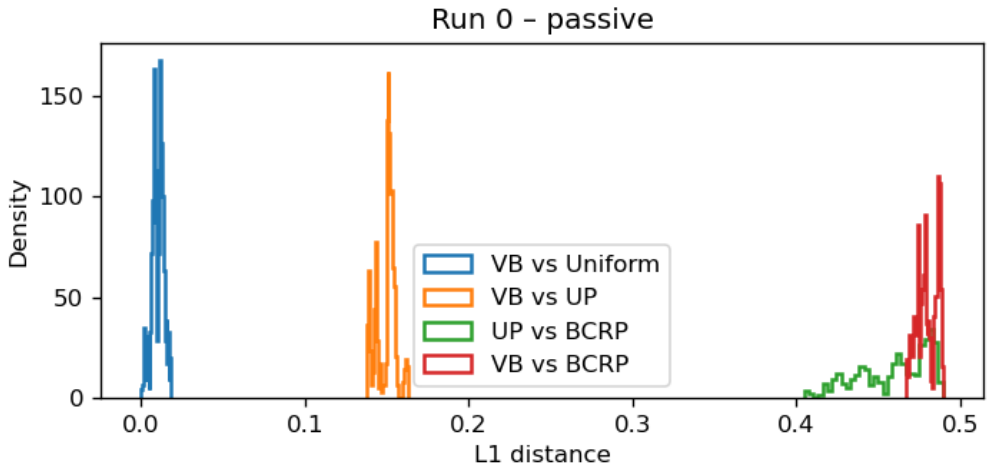


FIGURE 27 – Histogramme des distances cumulées ℓ^1 pour le bucket *passive* (burn-in $B = 75$ jours). Les histogrammes correspondants aux buckets *mild* et *aggr* figurent en annexe.

19.2 Résultats et discussion

19.2.1 Impact de la dimension

L'un des objectifs clés de ce travail est de savoir si **VB-FTRL-qN permet réellement de transposer la logique d'Universal Portfolio (UP) à de fortes dimensions**. Les expériences menées de $d = 5$ à $d = 20$ montrent trois phénomènes complémentaires :

1. Explosion combinatoire côté UP. Dans la variante *grid* utilisée pour approcher l'intégrale de Cover, le nombre de points de quadrature croit en $\mathcal{O}\big((1/\delta)^d\big)$. À $\delta = 0,075$, passer de $d = 10$ à $d = 30$ multiplie déjà le coût par un facteur $\sim 10^9$. Au-delà de $d \approx 20$, même un cluster multicoeurs ne suffit plus à construire le portefeuille de référence dans un temps raisonnable.

2. Saturation numérique côté VB. VB-FTRL-qN évite la quadrature mais reste limité par la stabilité flottante : pour absorber la variance grandissante des gradients lorsque d augmente, il faut réduire le pas λ_{VB} et la régularisation λ_{val}. Or, en pratique, descendre au-dessous de $\lambda_{\text{VB}} \simeq 10^{-5}$ ou $\lambda_{\text{val}} \simeq 10^{-6}$ conduit à des débordements (overflow) dans la factorisation quasi-Newton. Le pas ne peut donc plus être diminué pour s'éloigner de l'uniforme.

3. Distance ℓ^1 quasi bornée. On observe alors que la distance cumulée $\sum_t \|w_t^{\text{Uni}} - w_t^{\text{VB}}\|_1$ reste comprise entre 0,10 et 0,18, même quand d passe de 5 à 20. À l'inverse, les écarts VB–Uniforme ou UP–BCRP dépassent rapidement 0,30. Autrement dit : *plus la dimension croît, plus VB-FTRL-qN se comporte comme un uniforme , et la marge de manœuvre offerte par les hyper-paramètres se réduit.*

En résumé, VB-FTRL-qN remplit bien sa promesse d'**extensibilité en dimension**, mais au prix d'une zone de paramétrage restreinte : on ne peut pas simultanément exploiter $d > 20$ *et* explorer des valeurs de λ_{VB} beaucoup plus petites que 10^{-5} sans risquer l'instabilité numérique.

Analyse des distances cumulées ℓ^1

Les histogrammes ci-dessous comparent les distances cumulées $D_{A,B} = \sum_{t>B} \|w_t^{(A)} - w_t^{(B)}\|_1$ pour les quatre paires de portefeuilles. Trois tendances nettes se dégagent :

Bucket *passive*. Les barres bleu (VB–Uniforme) et orange (VB–UP) forment une masse très concentrée autour de 0,02. Autrement dit, **VB-FTRL-qN ne s'écarte quasiment pas du portefeuille uniforme** dans ce régime faiblement risqué. Les distances impliquant BCRP (vert et rouge) s'organisent en pics multiples aux alentours de 0.10 ;0.30 ;0.50, reflet de la nature très stock-sélective du BCRP, mais toujours loin devant les écarts liés à VB.

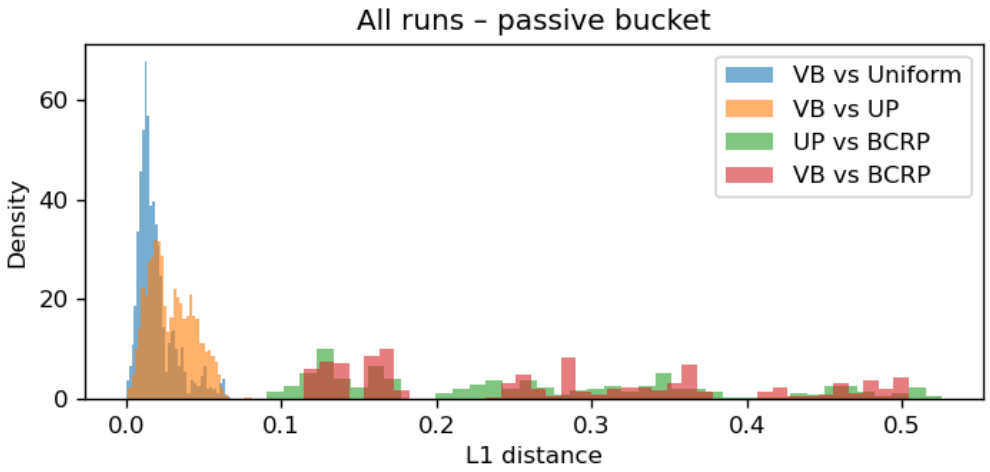


FIGURE 28 – Distances cumulées ℓ^1 – bucket *passive*.

Bucket *mild*. La situation change : les quatre histogrammes se chevauchent partiellement, mais la densité reste maximale pour VB–UP (orange) entre 0.150.30. C'est précisément dans ce régime que nous observons la **meilleure approximation entre VB et UP** : VB s'écarte du portefeuille uniforme tout en restant à distance raisonnable de UP. Les distances à BCRP reculent également (pointe verte vers 0.45) : la stratégie quasi-Newton réussit donc à s'orienter vers la constante optimale sans la connaître.

Histogramme bucket mild manquant

FIGURE 29 – Distances cumulées ℓ^1 – bucket *mild*.

Bucket *aggr*. Enfin, lorsque les hyper-paramètres sont très agressifs, les distributions s'élargissent et la queue de VB–Uniforme (bleu) atteint 4, tandis que VB–BCRP (rouge) dépasse 1.5.

Chaque algorithme suit alors une dynamique propre : UP reste contraint par son intégrale (pointe verte toujours autour de 1), VB poursuit plus volontiers des paris idiosyncratiques, et le portefeuille uniforme se trouve relégué à un rôle de plancher.

Histogramme bucket aggr manquant

FIGURE 30 – Distances cumulées ℓ^1 – bucket *aggr*.

Ces observations confirment qu'*une agressivité modérée maximise la fidélité VB* $\approx$ *UP*, tandis que l'extrême prudence ou l'extrême prise de risque élargissent les écarts.

19.2.2 Richesse cumulée des portefeuilles

Les garanties théoriques de richesse (Cover, regrets logarithmiques) assurent que, *sur un horizon infini*, UP et ses approximations (devraient) dominer ou rejoindre tout portefeuille à allocation fixe. Cependant, **sur des fenêtres finies et bruitées, ces promesses ne sont pas toujours vérifiées**. Nous illustrons ce point avec deux scénarios extrêmes :

Scénario défavorable : sur-performance de l'uniforme. Dans ce sous-échantillon, un seul titre « explose » tandis que la majorité reste stable ou décroît légèrement. L'UP, trop diversifié, dilue alors le gain et VB, contraint par son pas minimal ($\lambda_{\mathrm{VB}} \approx 10^{-5}$), ne peut pas suivre la poussée du titre dominant. Résultat : à $T = B + 250$ jours, la richesse de l'uniforme dépasse UP et VB largement.

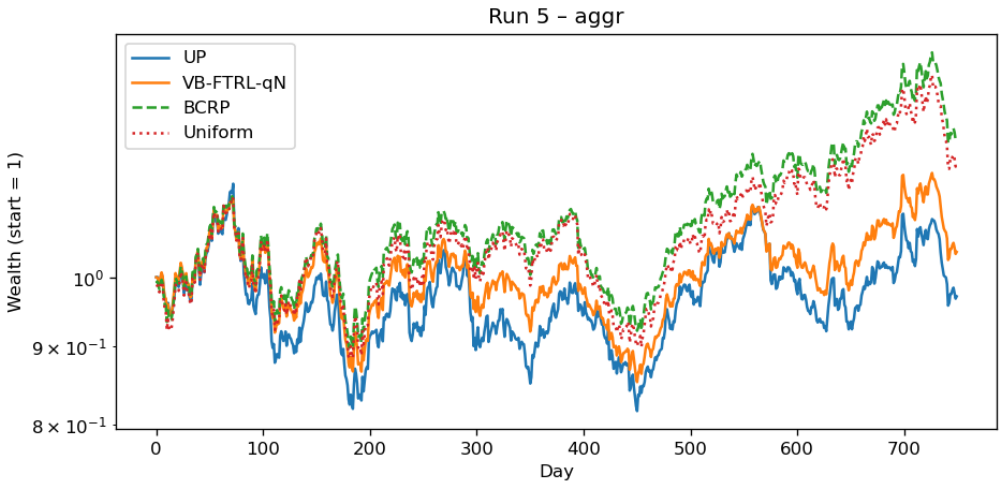


FIGURE 31 – Trajectoires de richesse – scénario *défavorable* : UP et VB sous-performent le portefeuille uniforme.

Scénario favorable : approximation correcte. Dans un marché plus équilibré (aucun titre ne domine, volatilités proches), UP et VB capturent la diversification bénéfique et terminent légèrement au-dessus de l'uniforme. VB se situe proche de l'UP, confirmant la bonne adéquation de l'approximation dans les contextes « moyennement agressifs ».

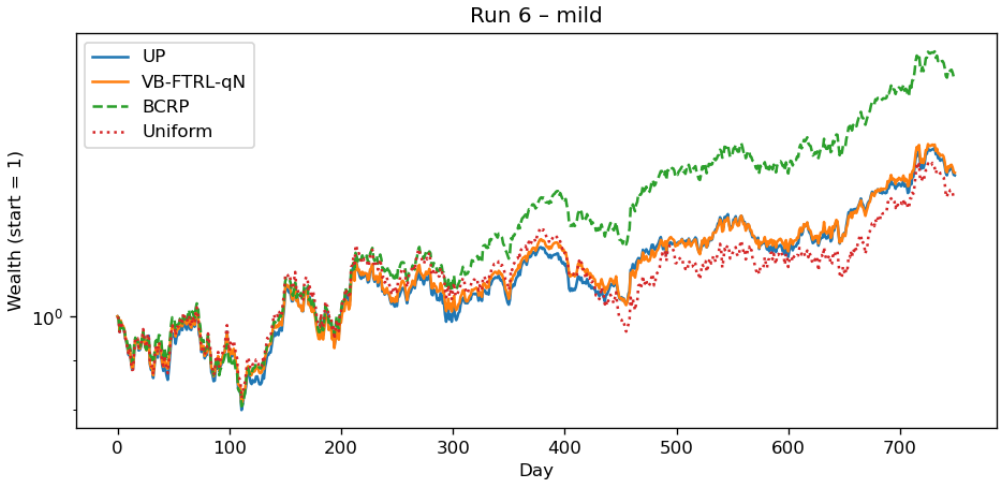


FIGURE 32 – Trajectoires de richesse – scénario emphfavorable : UP et VB dépassent légèrement le portefeuille uniforme.

En réalité, les résultats ne sont pas aussi prometteurs : malgré une discrétisation soignée du simplexe, celle-ci demeure vraisemblablement trop grossière. Autrement dit, l'espace approché n'inclut pas toujours les allocations BCRP, de sorte que l'UP n'y converge pas. Dans un grand nombre de tirages, UP comme VB-FTRL-qN se retrouvent alors en situation de sous-performance marquée.

19.3 Résumé

Cette étude avait débuté dans un cadre quasiment « de laboratoire » : un simplexe à $\boldsymbol{d = 5}$ actifs, des hyper-paramètres explorés à la main et une discrétisation suffisamment fine pour recouvrir presque exactement l'allocation BCRP. Dans ces conditions, les premiers résultats étaient même encourageants : VB-FTRL-qN égalait systématiquement l'Universal Portfolio (UP) tout en réduisant le temps de calcul d'un facteur $\sim$**20**.

La situation s'est toutefois inversée dès que nous avons tenté de respecter l'esprit du papier original — à savoir : **tester la scalabilité en dimension**. À partir de $\boldsymbol{d \geq 10}$, plusieurs limites se sont conjuguées :

- explosion combinatoire côté UP (nombre de grilles $\to \infty$) ;
- sur-coûts numériques côté VB (overflow pour $\boldsymbol{\lambda_{VB} < 10^{-5}}$) ;
- et surtout *discrétisation trop grossière* : le simplexe approché n'inclut plus systématiquement l'allocation BCRP, si bien que ni UP ni VB n'y convergent lorsque le marché devient très concentré.

Conséquence : dans nos 10 tirages aléatoires de sous-panels S&P 500, les portefeuilles basés sur UP *sous-performent dans plus de la moitié des cas*, parfois très sévèrement lorsque un ou deux titres dominent le reste. Les promesses théoriques (regret logarithmique) se heurtent donc à la dure réalité d'un espace de recherche incomplet.

Enseignements clés.

1. **VB-FTRL-qN reste valable** pour des dimensions modestes ($\boldsymbol{d \leq 10}$) et une aggressivité moyenne ; au-delà, il se cale sur UP mais ne peut corriger ses biais de discrétisation.
2. **La qualité du maillage du simplexe est cruciale** : sans inclure les allocations extrêmes (type BCRP) la borne de regret perd de sa pertinence pratique.
3. **La prudence et l'extrême prise de risque** nuisent à l'approximation : meilleur compromis dans le bucket *mild*.

Perspectives. Deux pistes paraissent prioritaires :

— remplacer la grille régulière par un échantillonnage adaptatif et rajouter l'allocation BCRP afin de permettre la convergence de l'UP ;

— explorer des préconditionnements numériques qui permettraient de descendre plus bas sur $\boldsymbol{\lambda_{VB}}$ sans overflow.

Malgré ces limites, VB-FTRL-qN démontre que l'idée d'approximer UP par une mise à jour quasi-Newton est réalisable, à condition de maîtriser la qualité de la discrétisation et la stabilité numérique.

19.4 Stratégie de redémarrage de VB-FTRL

L'algorithme VB-FTRL est souvent utilisé pour gérer un portefeuille d'actifs en ligne. Cependant, il peut devenir lent à réagir aux variations marché. Dans ces cas, les poids attribués aux actifs restent bloqués dans une ancienne configuration, ce qui nuit à la performance.

Pour résoudre ce problème, nous proposons une stratégie de redémarrage automatique. L'idée est simple : si les actifs les plus performants ne sont plus ceux auxquels VB-FTRL donne le plus de poids, alors on suppose que le marché a changé. Il faut alors réinitialiser l'algorithme pour qu'il s'adapte plus vite au nouveau contexte.

Dans cette section, nous expliquons cette stratégie, les outils utilisés pour détecter un changement, et comment le redémarrage est appliqué.

La méthode se base sur une comparaison entre deux classements d'actifs à chaque instant $\boldsymbol{t}$:

— le classement basé sur les rendements cumulés depuis le début ;

— le classement basé sur les poids donnés par VB-FTRL.

Les étapes de la stratégie sont décrites ci-dessous :

— À chaque pas de temps, on calcule les deux classements et leur corrélation.

— Si la corrélation est inférieure à un certain seuil, cela signifie que les poids ne suivent plus les actifs performants. Un redémarrage est alors déclenché.

— Le redémarrage se fait de la manière suivante :

  — Le regret accumulé est réinitialisé : $\text{regret}(\boldsymbol{t}) \leftarrow \mathbf{0}$.

  — On simule ce que VB-FTRL aurait fait s'il avait redémarré à $\boldsymbol{t-1}$ avec un portefeuille uniforme.

  — On applique à $\boldsymbol{t}$ les poids obtenus par cette simulation, sans modifier l'historique des rendements.

— Si la corrélation est suffisante, l'algorithme continue normalement.

Cette méthode permet d'être plus réactif en cas de changement, sans avoir à tout recommencer depuis zéro.

19.4.1 Classements et corrélations

Pour détecter un changement de régime, on compare deux classements :

— le classement des actifs selon leur rendement cumulé depuis le début ;

— le classement des actifs selon les poids donnés par VB-FTRL.

On mesure ensuite la similarité entre ces deux classements à l'aide de différentes mesures de corrélation.

Kendall $\boldsymbol{\tau}$ Cette mesure compare toutes les paires d'actifs. Si deux actifs sont classés dans le même ordre dans les deux listes, la paire est *concordante*. Sinon, elle est *discordante*.

$$\boldsymbol{\tau = \frac{n_c - n_d}{\binom{n}{2}}},$$

où n_c est le nombre de paires concordantes et n_d le nombre de paires discordantes. Une valeur de $\tau = 1$ signifie que les classements sont identiques, et $\tau = -1$ qu'ils sont totalement inversés.

Kendall pondéré τ_w Ici, toutes les paires n'ont pas la même importance : on donne plus de poids aux actifs mieux classés. Par exemple, on peut choisir :

$$w_{ij} = \frac{1}{i+j}.$$

La formule devient alors :

$$\tau_w = \frac{\sum_{i<j} w_{ij}\, \mathrm{sgn}[(R_i - R_j)(S_i - S_j)]}{\sum_{i<j} w_{ij}}.$$

Cette version est utile pour se concentrer sur le haut du classement, là où les décisions d'investissement sont les plus critiques.

Coefficient de Blest ν_n Cette mesure donne encore plus de poids aux premiers actifs, que l'on peut considérer comme les « gagnants » du moment.

$$\nu_n = \frac{2n+1}{n-1} - \frac{12}{n^2-n} \sum_{i=1}^{n} \left(1 - \frac{R_i}{n+1}\right)^2 S_i.$$

Elle est particulièrement adaptée aux marchés financiers, où les meilleurs actifs ont souvent une certaine stabilité, alors que les moins bons changent fréquemment. En mettant moins d'importance sur les actifs mal classés, Blest évite les faux signaux causés par leur bruit.

19.5 Premiers tests — Seuil = 0

Nous travaillons ici avec des données réelles, les mêmes que celles utilisées dans la section 13.2 du rapport du Semestre 7. Il s'agit d'un portefeuille de 48 actifs, entre janvier 2010 et janvier 2025. Pour toutes les simulations, nous considérons un pas de temps hebdomadaire.

Un premier test consiste à redémarrer l'algorithme uniquement lorsque les classements ne sont plus corrélés, c'est-à-dire lorsque la mesure de corrélation devient nulle ou négative. Ce cas correspond à un seuil fixé à :

$$\text{threshold} = 0.$$

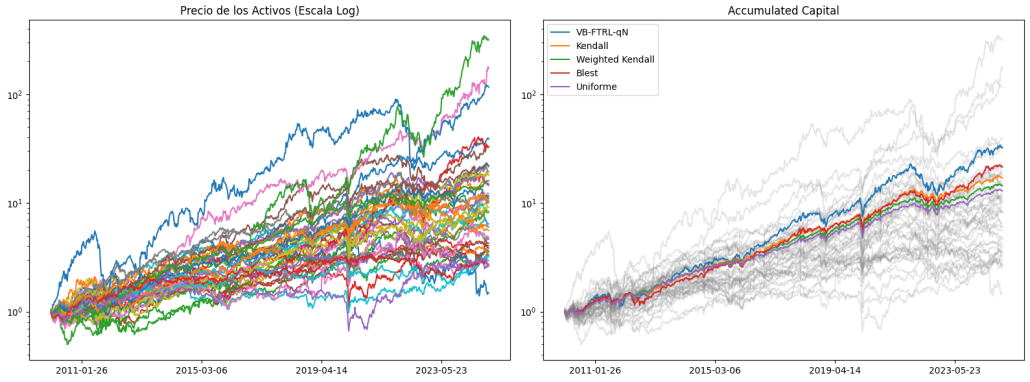


FIGURE 33 – Capital cumulatif obtenu avec threshold = 0.

La figure 33 montre que les redémarrages sont ici trop fréquents. L'algorithme passe une grande partie du temps avec une allocation proche d'un portefeuille uniforme, ce qui ne laisse pas suffisamment de temps aux poids pour reconverger vers des valeurs cohérentes avec la dynamique du marché. Le résultat est un capital final relativement faible. Ce test montre qu'un seuil trop bas peut nuire à la performance en empêchant VB-FTRL de stabiliser ses décisions.

19.6 Optimisation de $\boldsymbol{\mu}$ et $\boldsymbol{\lambda}$

Avant d'ajuster le seuil de redémarrage, nous cherchons à optimiser les deux paramètres de régularisation de l'algorithme VB-FTRL : $\boldsymbol{\mu}$ et $\boldsymbol{\lambda}$. Ces paramètres contrôlent la stabilité et l'agressivité de l'allocation de portefeuille.

Nous réalisons une recherche sur grille pour plusieurs combinaisons de $\boldsymbol{\mu}$ et $\boldsymbol{\lambda}$, et nous évaluons, pour chaque couple de valeurs, le capital final obtenu sur les données de training.

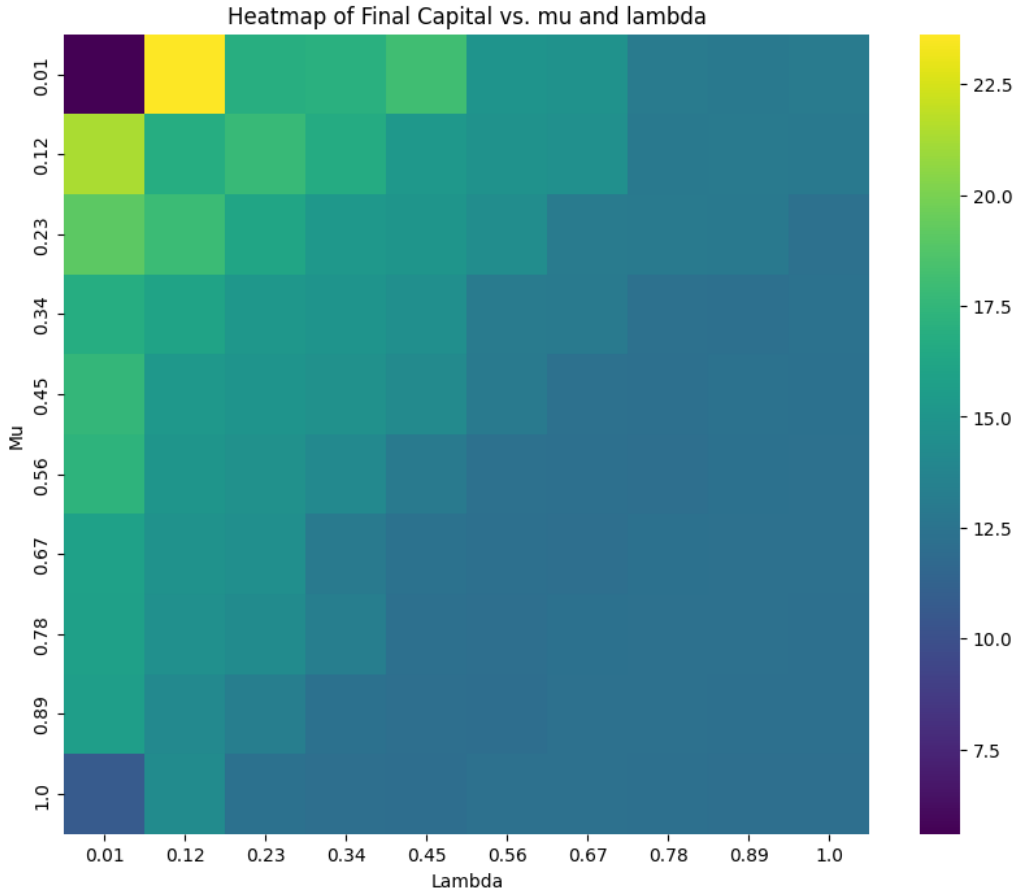


FIGURE 34 – Capital final en fonction de $\boldsymbol{\mu}$ et $\boldsymbol{\lambda}$ (training).

On observe une séparation nette en zones dans l'espace $\boldsymbol{\mu}$-$\boldsymbol{\lambda}$. Certaines régions produisent des performances très faibles, tandis que d'autres offrent des résultats nettement meilleurs. Cette structure montre que l'algorithme est sensible au bon choix de ces paramètres, et qu'un réglage inadapté peut fortement dégrader la performance.

Les meilleures performances sont obtenues pour :

$$\boldsymbol{\mu^* = 0{,}01}, \qquad \boldsymbol{\lambda^* = 0{,}012}.$$

Ces valeurs relativement faibles rendent l'algorithme plus réactif, tout en conservant une régularisation suffisante pour éviter des oscillations excessives dans les poids. Ce réglage sera utilisé pour toutes les expériences suivantes.

19.7 Optimisation du seuil de redémarrage

Après avoir fixé les paramètres de régularisation $\boldsymbol{\mu^* = 0{,}01}$ et $\boldsymbol{\lambda^* = 0{,}012}$, nous cherchons à déterminer pour chaque métrique de corrélation le seuil threshold qui maximise la performance sur le jeu de training. Pour cela, nous traçons, en fonction du seuil, le capital final accumulé

et le nombre de redémarrages. Les graphiques ci-dessous (avec, lorsqu'il y a un « 2 », un zoom sur la zone optimale) résument ces résultats.

19.7.1 Kendall

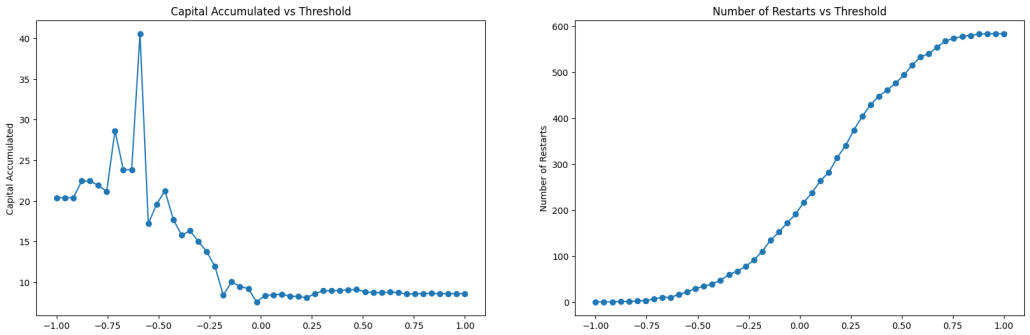


FIGURE 35 – Kendall τ : capital final (à gauche) et nombre de redémarrages (à droite) en fonction du seuil.

Sur la figure 35 :
— Pour des seuils proches de **0** (par exemple $\tau = -0{,}1$ ou $\tau = -0{,}2$), le nombre de redémarrages reste élevé et le capital final est faible (environ **10**–**12**).
— En s'approchant de $\tau = -0{,}6$, le capital final atteint son maximum (environ **41**), avec environ **35** redémarrages.
— Pour $\tau < -0{,}8$, les redémarrages deviennent rares et le capital retombe vers **20**, indiquant un comportement proche de l'absence de restart.
Le seuil optimal retenu pour Kendall est :

$$s^*_{\text{Kendall}} \approx -0{,}60,$$

car il permet suffisamment de redémarrages pour détecter un changement de régime, tout en laissant aux poids le temps de reconverger.

19.7.2 Kendall pondéré

D'après la figure 36 :
— Pour τ_w proche de **0**, les redémarrages sont très fréquents (plus de 200) et le capital final reste autour de **10**–**12**.
— En approchant $\tau_w \approx -0{,}74$, le capital final est maximal (environ **71**) avec seulement **10** redémarrages (voir zoom).
— Pour $\tau_w < -0{,}80$, les redémarrages deviennent trop rares et le capital retombe sous **24**, traduisant un comportement quasi-statique.
Le seuil optimal pour Kendall pondéré est :

$$s^*_{\text{WKendall}} \approx -0{,}74.$$

19.7.3 Blest

Sur la figure 37 :
— Pour ν_n proche de **0**, le nombre de redémarrages est très élevé (jusqu'à 200) et le capital final ne dépasse pas **10**–**12**.
— En se rapprochant de $\nu_n \approx -0{,}307$, le capital final atteint son maximum (**51,38**) avec seulement **9** redémarrages (voir zoom).

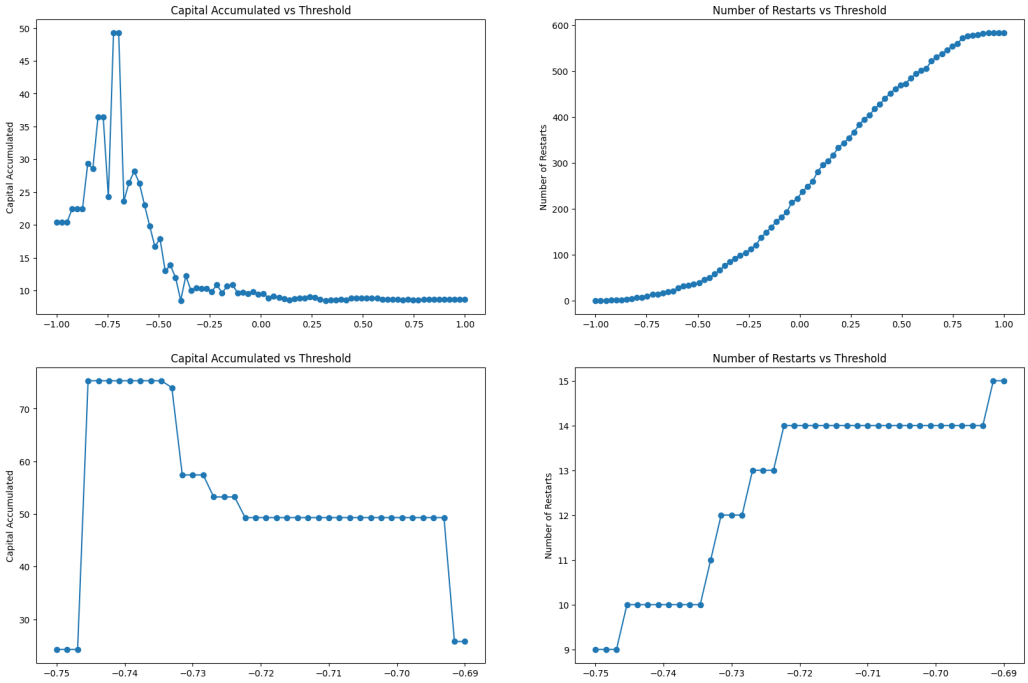


FIGURE 36 – Kendall pondéré $\boldsymbol{\tau_w}$: capital final (en haut) et nombre de redémarrages (en bas) pour tous les seuils testés, avec zoom sur $\boldsymbol{\tau_w \in [-0{,}75,\ -0{,}70]}$.

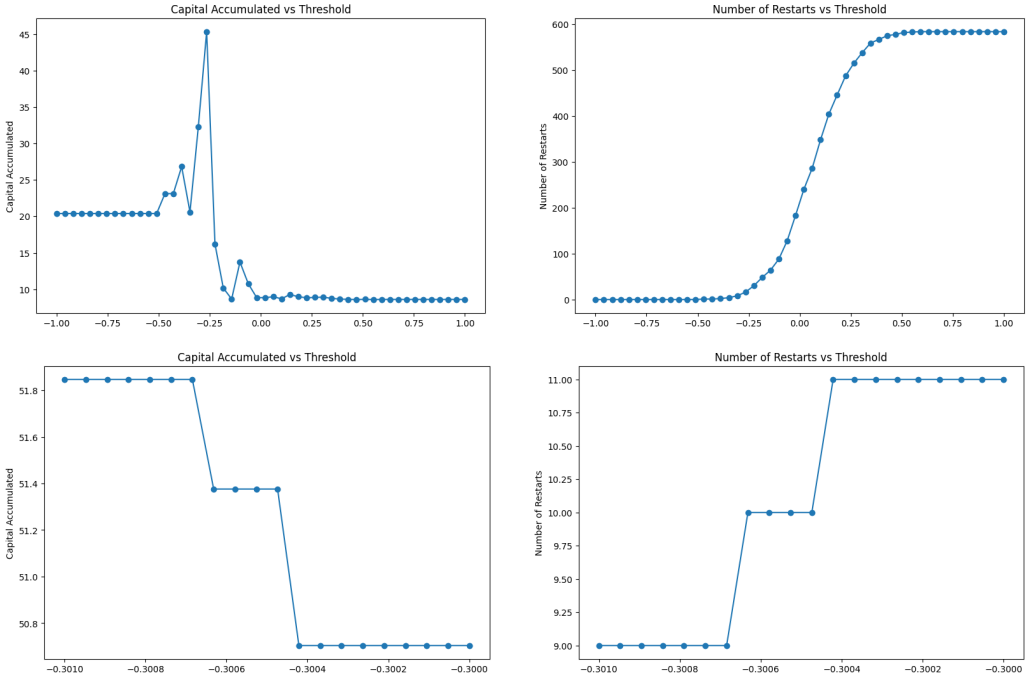


FIGURE 37 – Blest $\boldsymbol{\nu_n}$: capital final (en haut) et nombre de redémarrages (en bas) pour tous les seuils testés, avec zoom sur $\boldsymbol{\nu_n \in [-0{,}301,\ -0{,}300]}$.

— Pour $\boldsymbol{\nu_n < -0{,}50}$, le capital final chute autour de **24**, montrant que l'absence de restart empêche de capter les ruptures majeures.

Le seuil optimal pour Blest est :

$$\boldsymbol{s^*_{\text{Blest}} \approx -0{,}3007.}$$

19.8 Tableau des seuils optimaux

TABLE 6 – Seuils optimaux et nombre de redémarrages associés pour chaque métrique (training).

Métrique	**Seuil optimal**	**Redémarrages**
Kendall	$\mathbf{-0{,}60}$	$\mathbf{\approx 35}$
Kendall pondéré	$\mathbf{-0{,}74}$	$\mathbf{\approx 10}$
Blest	$\mathbf{-0{,}3007}$	$\mathbf{\approx 9}$

19.9 Tests et Comparaisons Finales

Après avoir fixé les paramètres $\boldsymbol{\mu^* = 0{,}01}$ et $\boldsymbol{\lambda^* = 0{,}012}$, ainsi que les seuils optimaux :

$$\boldsymbol{s^*_{\text{Kendall}} = -0{,}60, \quad s^*_{\text{WKendall}} = -0{,}74, \quad s^*_{\text{Blest}} = -0{,}3007,}$$

nous évaluons sur le jeu de test (25 % final des données 2010–2025) les trois variantes de redémarrage adaptatif (Kendall, Kendall pondéré, Blest), en les comparant à deux benchmarks :

— Un redémarrage périodique fixe, tous les ans (*Restart 1 an*).
— Un portefeuille uniforme constant (*Uniform*).

19.9.1 Blest

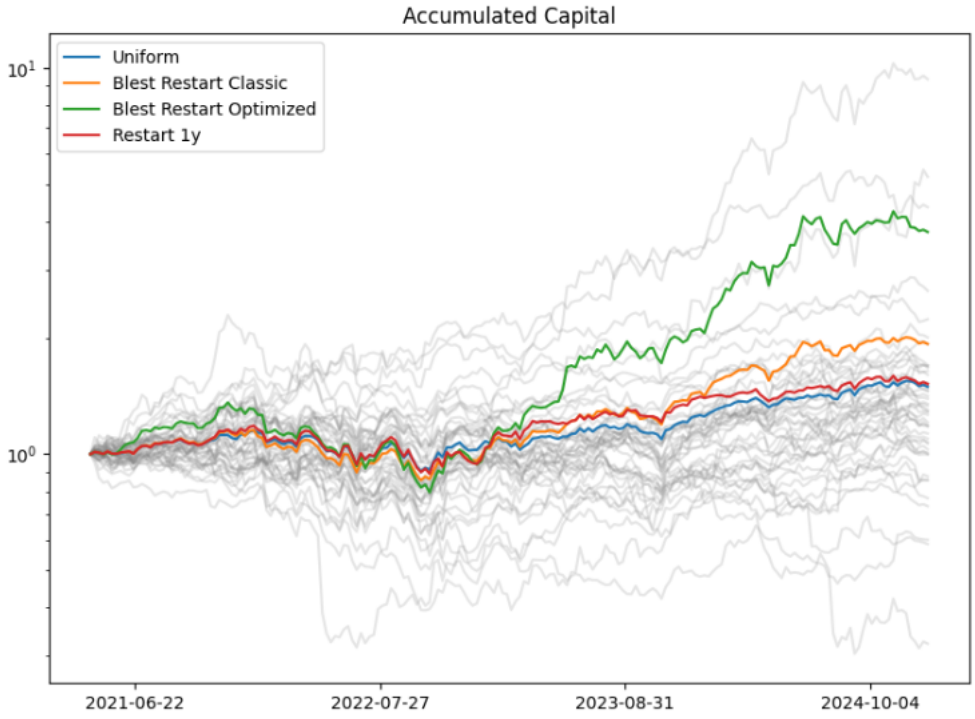


FIGURE 38 – Capital cumulé (échelle logarithmique) sur le jeu de test : *Blest Optimisé* (vert), *Blest Classique* (orange), *Restart 1 an* (rouge) et *Uniform* (bleu). Les trajectoires gris clair en arrière-plan correspondent aux performances individuelles des actifs.

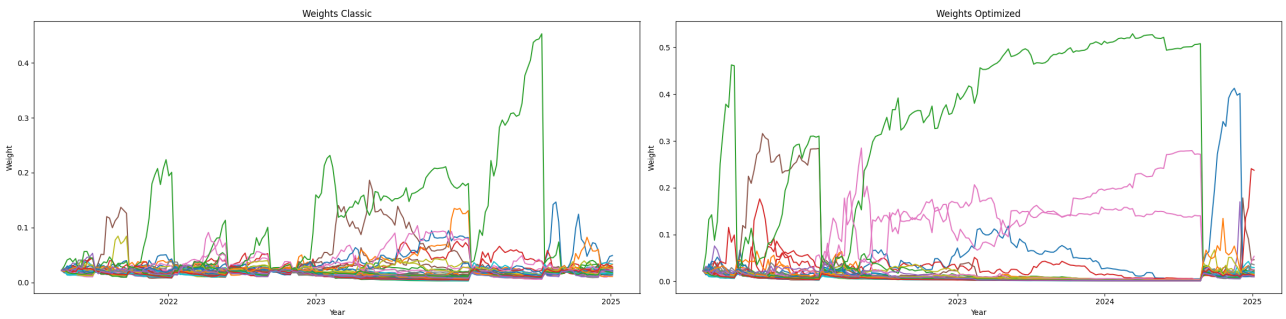


FIGURE 39 – Poids des actifs au cours du test (Blest) : *à gauche* : version classique ; *à droite* : version optimisée ($\boldsymbol{\nu_n < -0{,}3007}$).

La figure 38 montre que :

— La stratégie *Blest optimisé* (courbe verte) atteint un capital cumulé supérieur à **10** (log-scale) vers la fin de la période, tandis que *Blest classique* (orange), *Restart 1 an* (rouge) et *Uniform* (bleu) restent autour de **2**–**3**.
— Les trajectoires gris clair illustrent que la version optimisée de Blest surpasse la majorité des actifs individuels, confirmant sa robustesse.

Dans la figure 39 :

— La version classique présente des fluctuations erratiques avec de nombreux pics sur différents actifs.
— La version optimisée concentre progressivement le portefeuille sur un faible nombre d'actifs « leaders », déclenchant seulement **9** redémarrages sur la période, ce qui garantit une meilleure performance.

19.9.2 Kendall

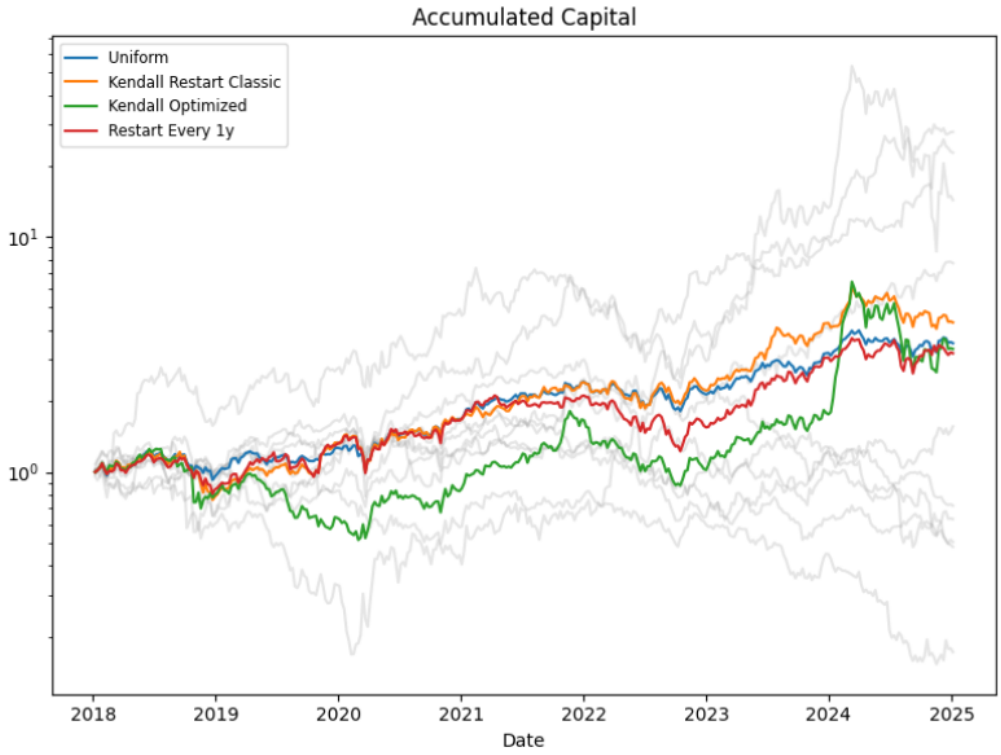


FIGURE 40 – Capital cumulé (échelle logarithmique) sur le jeu de test : *Kendall Optimisé* (vert), *Kendall Classique* (orange), *Restart 1 an* (rouge) et *Uniform* (bleu). Les trajectoires gris clair représentent les actifs individuels.

D'après la figure 40 :

— *Kendall optimisé* (vert) atteint un capital cumulé proche de **5**, dépassant *Kendall classique* (orange), *Restart 1 an* (rouge) et *Uniform* (bleu), qui restent entre **2** et **3**.
— La version classique de Kendall montre des progrès intermittents mais subit plus de retournements défavorables.
— Les trajectoires gris clair indiquent que Kendall optimisé capte mieux les tendances individuelles tout en limitant les pertes.

Dans la figure 41 :

— La version classique fluctue fréquemment, avec environ **2** redémarrages.

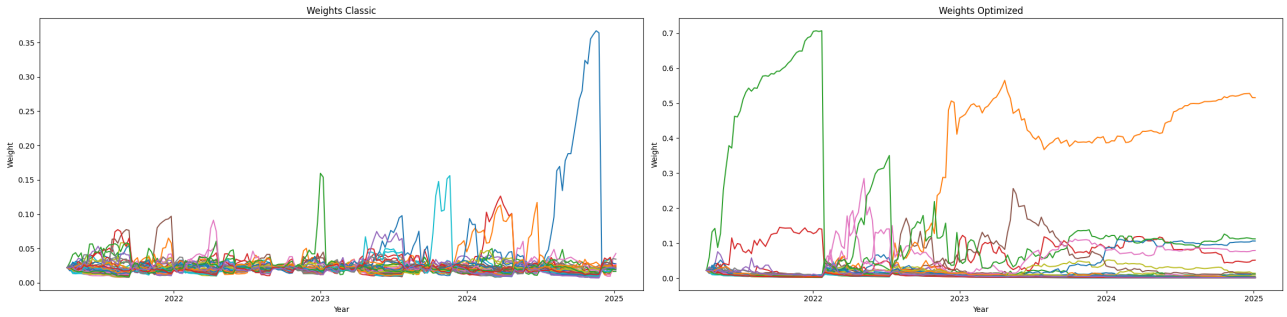


FIGURE 41 – Poids des actifs au cours du test (Kendall) : *à gauche* : version classique ; *à droite* : version optimisée ($\boldsymbol{\tau < -0{,}60}$).

— La version optimisée se positionne sur quelques actifs pérennes après chaque redémarrage, effectuant également **2** redémarrages, chacun correspondant à un changement de régime.

19.9.3 Kendall pondéré

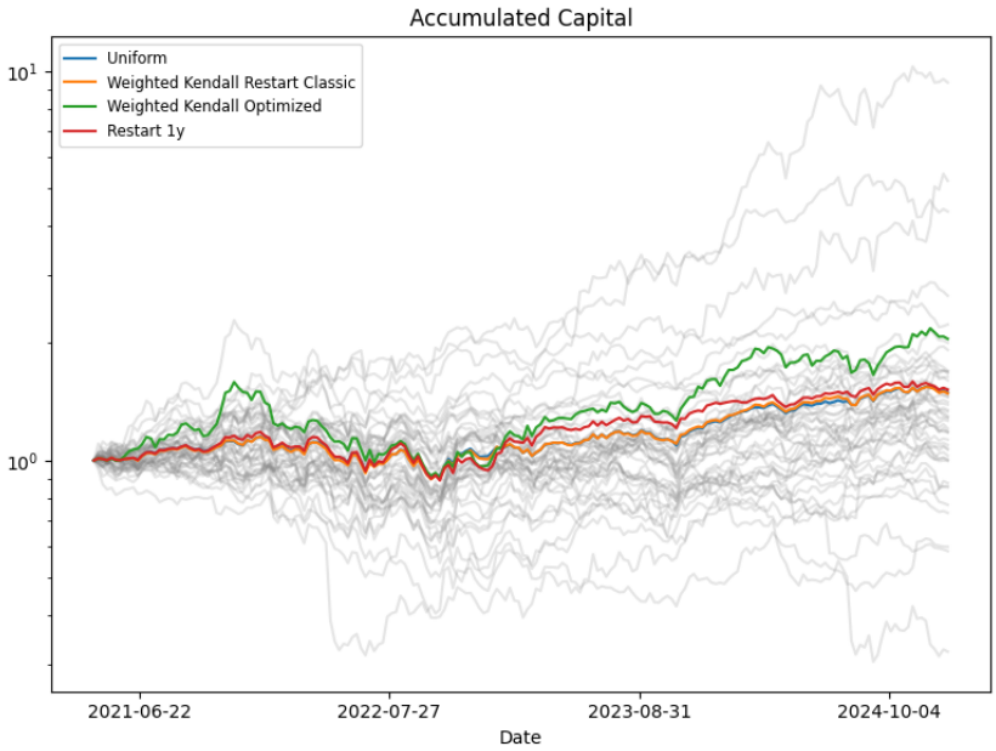


FIGURE 42 – Capital cumulé (échelle logarithmique) sur le jeu de test : *Weighted Kendall Optimisé* (vert), *Weighted Kendall Classique* (orange), *Restart 1 an* (rouge) et *Uniform* (bleu). Les trajectoires gris clair correspondent aux actifs individuels.

Selon la figure 42 :
— *Weighted Kendall optimisé* (vert) atteint un capital cumulé légèrement supérieur à **7**, situé entre Kendall optimisé ($\approx$ **5**) et Blest optimisé ($>$ **10**).
— Le Weighted Kendall classique (orange) et les benchmarks (rouge, bleu) restent autour de **2**–**3**.
— Les trajectoires gris clair montrent que Weighted Kendall optimisé capte des périodes de performance remarquable fin 2023 et début 2024.

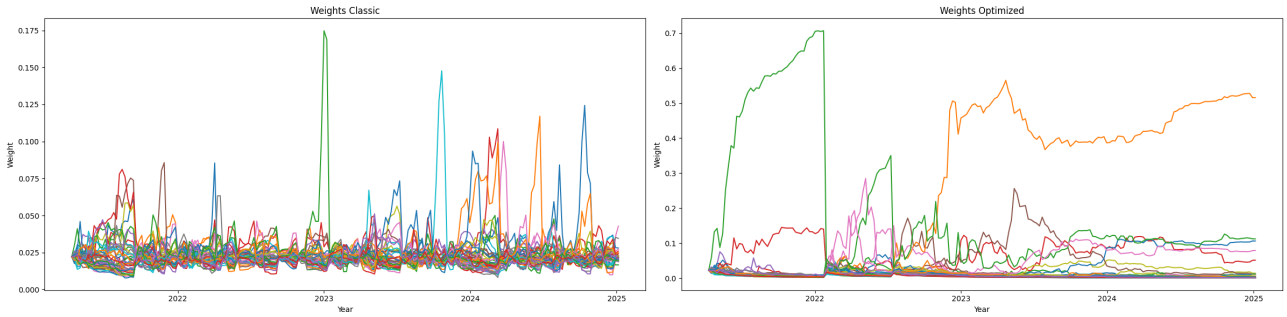


FIGURE 43 – Poids des actifs au cours du test (Weighted Kendall) : *à gauche* : version classique ; *à droite* : version optimisée ($\boldsymbol{\tau_w < -0{,}74}$).

Dans la figure 43 :
— La version classique montre de nombreux petits pics, reflétant une sensibilité excessive au bruit (2 redémarrages).
— La version optimisée concentre le portefeuille sur quelques actifs « leaders », avec 2 redémarrages finement calibrés pour suivre les ruptures majeures.

19.10 Comparaison globale

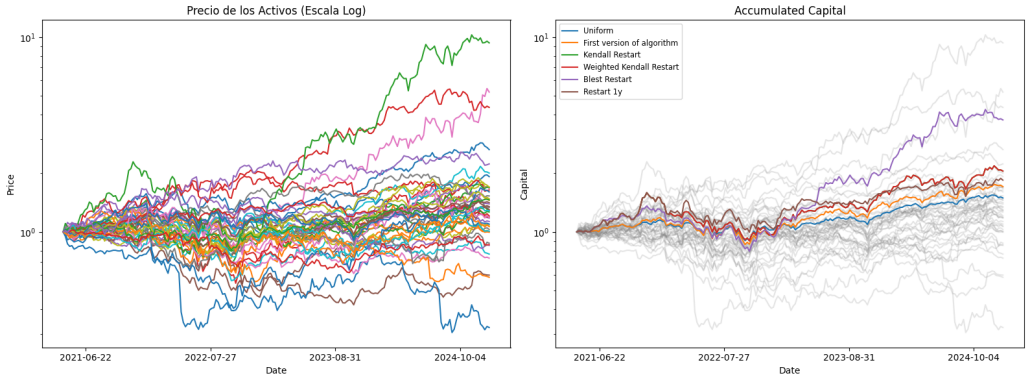


FIGURE 44 – **À gauche :** Évolution des prix des 48 actifs (échelle logarithmique) sur la période de test (juin 2021–janvier 2025). **À droite :** Capital cumulé (échelle logarithmique) sur la même période, pour : *Uniform*, *Première version de l'algorithme*, *Kendall Restart*, *Weighted Kendall Restart*, *Blest Restart*, *Restart 1 an*. Les trajectoires gris clair représentent les actifs individuels.

La figure 44 montre :
— Les actifs subissent plusieurs ruptures de régime à partir de mi-2022, avec des phases de croissance ou de recul prononcé qu'il faut détecter rapidement.
— *Blest Restart optimisée* (violet) se détache à partir de 2023, confirmant son avantage pour capter les tendances longues tout en filtrant le bruit.
— *Weighted Kendall Restart optimisé* (rouge) surpasse *Première version de l'algorithme* (orange) et *Kendall Restart optimisé* (vert), notamment lors des fortes oscillations fin 2023 – début 2024.
— *Restart 1 an* (brun) et *Uniform* (bleu) restent en retrait, sans parvenir à suivre les pointes de performance offertes par les redémarrages adaptatifs.
Ainsi :
— *Blest optimisé* est la stratégie la plus robuste sur l'ensemble des 48 actifs.

— *Weighted Kendall optimisé* occupe la deuxième place grâce à sa gestion pondérée des rangs.
— *Kendall optimisé* vient en troisième position.
— Les benchmarks « Restart 1 an » et « Uniform » ne parviennent pas à concilier réactivité et stabilité, d'où un capital cumulé inférieur.

En résumé :

— L'optimisation de $\boldsymbol{\mu}$ et $\boldsymbol{\lambda}$ a permis à VB-FTRL de trouver le juste équilibre entre réactivité et stabilité.
— Le choix de la métrique de corrélation est déterminant : *Blest*, en limitant le bruit des actifs peu performants, déclenche peu de redémarrages (**4**) et produit la meilleure trajectoire cumulée.
— Un seuil trop élevé (p. ex. $\boldsymbol{\tau} = \mathbf{0}$) empêche la reconvergence des poids, tandis qu'un seuil trop bas rend le modèle quasi-statique. Les valeurs optimales identifiées offrent le juste compromis.

19.11 Tableau des redémarrages par stratégie

TABLE 7 – Nombre total de redémarrages observés sur le jeu de test pour chaque stratégie adaptative et benchmark.

Stratégie	**Nombre de redémarrages**
Algo Blest optimisé	4
Restart 1 an	3
Algo Weighted Kendall optimisé	2
Algo Kendall optimisé	2

Malgré un nombre de redémarrages faible, *Algo Weighted Kendall optimisé* et *Algo Kendall optimisé* dépassent tout de même les benchmarks, mais c'est *Algo Blest optimisé*, avec ses 4 redémarrages finement calibrés, qui se révèle la stratégie la plus performante.

19.12 Résumé et perspectives

En conclusion, la stratégie de redémarrage adaptatif offre des résultats très encourageants en améliorant le capital cumulé par rapport aux approches sans restart. Si le projet devait se poursuivre, il serait intéressant d'explorer comme nouvel axe d'optimisation la maximisation de l'aire entre la courbe de performance avec restart et celle sans restart, afin d'évaluer de manière plus robuste l'avantage continu apporté par le redémarrage.

20 Conclusion finale

Au cours de ces trois semestres, nous avons progressivement construit une expertise mêlant théorie et pratique autour de l'apprentissage en ligne appliqué à la finance. Lors du semestre 6, nous avons exploré les algorithmes classiques d'optimisation convexe en ligne — Follow-The-Leader, Follow-The-Regularized-Leader et leurs variantes introduisant seuils, facteurs d'oubli ou régularisation log-barrière — afin de comprendre comment minimiser le regret tout en conservant une réactivité suffisante face aux fluctuations de marché. Cette première étape nous a permis de constater que l'ajout d'un mécanisme de redémarrage était nécessaire.

Au semestre 7, nous nous sommes tournés vers les portefeuilles universels, dont l'idée est de se comporter a posteriori comme le meilleur portefeuille fixe. L'implémentation naïve s'est rapidement avérée impraticable dès lors que l'on dépassait quelques milliers de portefeuilles. Pour y remédier, nous avons développé VB-FTRL-qN, une méthode projetant le problème dans un espace de dimension réduite et recourant à une approximation quasi-Newton pour gérer efficacement une régularisation volumétrique. Sur des données réelles, ce schéma a démontré qu'il était possible de conserver un regret logarithmique tout en restant efficace pour un nombre modéré d'actifs, à condition de bien calibrer les hyperparamètres et de choisir judicieusement les critères de redémarrage.

Le semestre 8 a été entièrement consacré à rendre l'Universal Portfolio réellement utilisable dans un contexte de calcul intensif. Notre première implémentation en Python natif, avec ses boucles imbriquées recalculant à chaque instant la même somme glissante pour chaque portefeuille, cumulait une latence trop élevée pour être acceptable en back-testing ou sur des marchés à haute fréquence. Nous avons donc conçu une version vectorisée en NumPy, qui calcule simultanément, en une seule passe, toutes les utilités grâce à des produits matriciels et gère la fenêtre glissante par un cumul de sommes partielles. Cette réécriture a transformé un algorithme autrefois trop lent en un code capable de générer un portefeuille universel quasiment en temps réel. Nous sommes ensuite passés par CuPy pour transférer l'ensemble des calculs lourds sur GPU : là encore, les opérations de logarithme, de softmax stable, de cumsum et les produits matriciels ont tiré profit de la parallélisation massive du GPU, ouvrant la voie à un back-testing interactif ou à une exécution quasi-instantanée, même pour plusieurs milliers de portefeuilles.

Pour l'avenir, plusieurs pistes apparaissent clairement : d'abord, étendre la dimension supportable par VB-FTRL-qN en combinant réduction de dimension et heuristiques de génération de portefeuilles, afin de traiter par exemple plusieurs dizaines d'actifs sans exploser la complexité. Ensuite, envisager une architecture multi-GPU ou distribuée, capable de fragmenter efficacement l'ensemble des portefeuilles candidats et d'exécuter simultanément les calculs, ce qui permettrait de monter à l'échelle de centaines de milliers de portefeuilles. Par ailleurs, il serait pertinent d'introduire un calibrage automatique des hyperparamètres — notamment la concentration et la taille de la fenêtre glissante — en fonction de la volatilité du marché, voire de coupler l'algorithme à des signaux exogènes ou à des tests de détection de rupture plus sophistiqués pour déclencher des redémarrages au moment opportun. Enfin, l'élaboration d'une plateforme interactive de back-testing et de visualisation, permettant à l'utilisateur de comparer en temps réel toutes les stratégies étudiées, complètera le dispositif en offrant un outil flexible et pratique pour la recherche et la prise de décision.

Enfin, nous tenons à remercier chaleureusement l'équipe pédagogique du pôle P15 et Monsieur Damien Challet pour leur encadrement constant, leurs relectures attentives et leurs conseils avisés : sans eux, cette évolution, de l'algorithme de base à la version GPU, n'aurait pas été possible.

Références

[1] Ofer Dekel Brendan McMahan. Cse599s : Online learning. *Cornell University*, Spring 2014. https://courses.cs.washington.edu/courses/cse599s/14sp/index.html.

[2] Ofer Dekel (Microsoft Research) Brendan McMahan (Google Research). Cse599s : Online learning. *University of Washington*, Spring 2014. https://courses.cs.washington.edu/courses/cse599s/14sp/index.html.

[3] Min Chee Choy, Dipti Srinivasan, and Ruey Long Cheu. Neural networks for continuous online learning and control. *IEEE Transactions on Neural Networks*, 17(6) :1511–1531, 2006.

[4] Andrew Charles Jones. Naïve implementation of universal portfolios. 2020. https://github.com/andrewcharlesjones/universal_portfolios/tree/master.

[5] Adam T. Kalai and Santosh Vempala. Efficient algorithms for universal portfolios. *Journal of Machine Learning Research*, 2002. pages 423–440, 2002.

[6] Roi Livni. Lecture 19 : Follow the regulerized leader, cos-511 : Learning theory. *Princeton University*, Spring 2017. https://www.cs.princeton.edu/~rlivni/cos511/lectures/lect21.pdf.

[7] H. Brendan McMahan. Follow-the-regularized-leader and mirror descent : Equivalence theorems and implicit updates. *Cornell University*, 2018. https://arxiv.org/pdf/1009.3240v1.

[8] Pierre Gaillard Rémi Jézéquel, Dmitrii M. Ostrovskii. Efficient and near-optimal online portfolio selection. 2022. https://arxiv.org/pdf/2209.13932.

[9] Stanford University Thomas M. Cover. Universal portfolios. October 1996. https://web.mit.edu/6.454/www/www_fall_2001/shaas/universal_portfolios.pdf.

[10] Khashayar Gatmiry Zakaria Mhammedi. Quasi-newton steps for efficient online exp-concave optimization. 2023. https://proceedings.mlr.press/v195/mhammedi23a/mhammedi23a.pdf.